UNIVERSITE DE YAOUNDE I
FACULTE DES SCIENCES
DEPARTEMENT D'INFORMATIQUE

THE UNIVERSITY OF YAOUNDE I
FACULTY OF SCIENCES
DEPARTMENT OF COMPUTER SCIENCE

Migration d'une Interface Homme Machine de l'architecture MVC vers l'architecture MVC 2: Cas d'étude l'application web de l'ERP ADempiere

Mémoire

Présenté et soutenu par :

FOUOMENE PEWO Daniel Rene
Maître ès sciences
Matricule : 07U483

En vue de l'obtention du

Diplôme de Master II Recherche en Informatique Fondamentale

Composition du jury

Président: **Pr. Claude TANGHA**, Maître de Conférences, Université de Yaoundé 1

Examinateur: **Dr. Roger ATSA ETOUNDI**, Chargé de cours, Université de Yaoundé 1

Rapporteurs: **Pr. Marcel FOUDA NDJODO**, Maître de Conférences, Université de Yaoundé 1

Mr. Prosper MENDOUGA, Ingénieur, DG KoosseryTech CMR

Année Académique 2011 - 2012

DEDICACES

A mon papa PEWO Jacques

A ma maman MELI FOUOMENE Celine

A la mémoire de maman WATCHUENG Sidonie

REMERCIEMENTS

- À tous *les enseignants du Département d'informatique de la Faculté des sciences de l'Université de Yaoundé I* pour tous les enseignements reçus durant ma formation.
- Pr. Marcel FOUDA NDJODO, Chef de Département Informatique et des Technologies Educatives à L'ENS de Yaoundé, pour avoir cru en moi, pour le suivi constant, ses conseils et ses suggestions réguliers qui m'ont encouragé et m'ont permis de mener à terme ce travail.
- Pr. Claude TANGHA, Chef de Département Informatique à L'ENSP de Yaoundé, pour m'avoir fait l'honneur de présider ce jury.
- Dr. Roger ATSA ETOUNDI, Chargé de cours à la faculté des sciences de l'université de Yaoundé I, pour m'avoir fait l'honneur d'être membre de ce jury.
- La société KOOSSERY TECHNOLOGY CMR par le biais de son Directeur Mr Prosper MENDOUGA, pour m'avoir accueilli en son sein et permis de réaliser ce stage académique dans des conditions agréables.
- Mes encadreurs au sein de l'entreprise, Mr Prosper MENDOUGA et Mr Baké Jc BAKENEGHE. Vos remarques, votre objectivité, vos conseils et votre sens de l'organisation m'ont été précieux.
- Ing. Valery TANDZON pour sa disponibilité et ses précieux conseils en tant que team leader du pool ERP et ami.
- Ing. Cedrick ESSALE ainsi qu'au personnel de l'entreprise qui a été accueillant et a sacrifié de précieuses minutes pour m'aider dans mon travail.
- Mes camarades de stage Beauclair Nanfack, Alain Mouafo, Pascal Feutseu, Claude Ngotoum, Lamine Mouchili, thierry Tagnekom, avec qui j'ai partagé des bons et des difficiles moments, et qui ont rendu ma période de stage agréable.
- Mes frères et sœurs Joseph, Nelson, Moïse, Carole, Darole, Staëlle et Elisabeth qui sont pour moi une source de courage et détermination.
- Mes oncles et Tante Tia Thomas, Kemta Jean Piere, FOM Georges et Kemta Véronique.
- Mes cousins Pewo Bruno, Yonta Tia Eugene.
- À toute ma famille.
- A tous ceux qui n'apparaissent pas dans ces quelques lignes et qui font partie de ma vie.

ABREVIATIONS

Terme	Signification
AJAX	Asynchronous Javascript and XML
API	Application Programming Interface
ASP	Active Server Pages
Eclipse RPC	Eclipse Rich Client Platform
EJB	Enterprise JavaBeans
ERP	Entreprise Resource Planning
GED	Gestion Electronique Documentaire
HTML	Hypertext Markup Language
IHM	Interface Homme Machine
Java EE	Java Enterprise Edition
JMS	Java Message Service
JPA	Java Persistence API
JSP	Java Server Pages
MDB	Message Driven-Bean
MVC	Modèle, Vue, Contrôleur
MVC 2	Modèle, Vue, Contrôleur (avec contrôleur unique)
OTAN	Organisation du Traité de l'Atlantique Nord
PC	Personal Computer
PDA	Personal Digital Assistant
SI	Système d'information
SQL	Structured Query Language
XML	Extensible Markup Language

RESUME

Le MVC (Modèle-Vue-Contrôleur) est une architecture qui organise l'interface homme-machine (IHM) d'une application logicielle. Il divise l'IHM en un modèle (modèle de données), une vue (présentation, interface utilisateur) et un contrôleur (logique de contrôle, gestion des événements, synchronisation), chacun ayant un rôle précis dans l'interface. Le MVC, très pratique, peut cependant, se révéler lourd à mettre en place à cause de la multitude de contrôleurs à implémenter. Afin de simplifier la réalisation d'une telle architecture, une nouvelle version a été introduite : le MVC 2. C'est exactement la même architecture à la différence qu'il n'y a plus qu'un seul contrôleur qui se charge de rediriger la requête vers le bon traitement. Il se pose donc le problème de la migration de ces IHM qui étaient basées sur l'architecture MVC vers l'architecture MVC 2. Dans ce mémoire, nous proposons une méthodologie de migration de ces IHM. Cette méthodologie est fortement basée sur les techniques de refactoring du génie logiciel. Comme cas d'étude, nous illustrons cette méthodologie sur l'application web de l'ERP ADempiere.

Mots-clés : Génie logiciel, Refactoring, Interface Homme Machine(IHM), MVC, MVC 2

ABSTRACT

The MVC (Model-View-Controller) is an architecture which organizes the Human Computer Interaction (HCI) of a software application. It divides the HCM in a model (data models), view (presentation, user interface) and a controller (control logic, event handling, synchronization), each having a specific role in the interface. Although MVC is more, convenient in practice, it may however, be difficult to implement because of the large numbers of controllers. In order to simplify the realization of such architecture, a new version was introduced: named MVC 2. It has exactly the same architecture with MVC, but is based only on one controller which is responsible to redirect the request to the right treatment. This raises the problem of migration of HCMs from MVC to MVC 2. This report aims at solving this problem by proposing a methodology for migration of HCM from MVC to MVC 2 architecture. This methodology is strongly based on refactoring techniques of software engineering. As a use case, we illustrate this methodology with the web application ADempiere ERP.

Keywords: Software Engineering, Refactoring, Human Computer Interaction (HCI), MVC, MVC 2

LISTE DES FIGURES

Figure 1 - Le processus de maintenance [Retaille 2005]....8
Figure 2 - L'interface, un intermédiaire entre l'homme et la machine [Baron 2003]....11
Figure 3 - Modèles intervenant dans le cycle de développement en V pour les systèmes interactifs [Baron 2003]....12
Figure 4 - Le modèle MVC [Zango et Ouattara 2006]....15
Figure 5 - L'architecture MVC dans une application....19
Figure 6 - L'architecture MVC 2 dans une application....20
Figure 7- Renommage d'éléments de code [Retaille 2005]....23
Figure 8 - Extraction d'une méthode [Retaille 2005]....25
Figure 9 - Extraction d'interface [Retaille 2005]....25
Figure 10 - Changement de signature d'une méthode [Retaille 2005]....26
Figure 11 - Généralisation d'un type [Retaille 2005]....28
Figure 12 - Refonte avec le design pattern façade [Retaille 2005]....30
Figure 13 - a - IHM basée sur MVC....31
Figure 14 - Architecture de Struts2 [Jérôme L. 2009]....42
Figure 15 - Architecture de l'application web de l'ERP ADempiere basée sur MVC....45
Figure 16 - Extrait de la structure de la classe d'action WindowAction....47
Figure 17 - Extrait du fichier struts.xml....48
Figure 18 - Architecture de l'application web de l'ERP KoosseryADempiere basée sur MVC 2....49
Figure 19 - Fonctionnalité de L'ERP KoosseryADempiere....50
Figure 20 - Le menu Refactor d'Eclipse....51
Figure 21 - Architecture de KoosseryADempiere.web....52
Figure 22 - Arborescence du projet koosseryADempiere.web....54
Figure 23 - Extrait du fichier pom.xml....56
Figure 24 - Extrait du fichier web.xml....57
Figure 25 - Les classes d'action....60
Figure 26 - La classe d'action KoosseryAdempiereBaseAction....60
Figure 27 - Extrait du fichier struts.xml....62
Figure 28 - Package login....67
Figure 29 - Extrait du Package de la vue window228.jsp générée par l'Application Dictionnary....69

SOMMAIRE

Dédicaces i
Remerciements ii
Abréviations iii
Résumé iv
Abstract iv
Liste des figures v
Sommaire vi
INTRODUCTION GENERALE 1
CHAPITRE I : DEFINITION DES CONCEPTS 5
I.1. Le génie logiciel 5
I.2. Le refactoring 9
I.3. L'Interface Homme Machine 11
I.4. Architecture d'Interface Homme Machine 13
I.5. L'architecture MVC 15
I.6. L'architecture MVC 2 19
I.7. Migration informatique 21
CHAPITRE II : METHODOLOGIE DE MIGRATION 23
II.1. Les outils scientifiques utilisés lors de la migration 23
II.2. Présentation de la méthodologie de migration 30
II.3. Illustration sur l'application web de l'ERP ADempiere 32
CHAPITRE III : REALISATION DE L'APPLICATION WEB DE L'ERP KoosseryADempiere 50
III.1. Présentation de L'ERP KoosseryADempiere 50
III.2. Environnement de développement et outils utilisés 51
III.3. Architecture de l'application web de l'ERP KoosseryADempiere 52
III.4. Arborescence du projet *KoosseryADempiere.web* 53
III.5. Migration de l'application web de l'ERP ADempiere 55
CONCLUSION GENERALE 70
Référence bibiographiques 71
Annexes 73
Table des matières 77

INTRODUCTION GENERALE

Ce mémoire est le fruit du travail mené durant le stage académique au sein des équipes de KOOSSERY TECHNOLOGY CMR, du 07 Mars au 30 Décembre 2011. KOOSSERY TECHNOLOGY CMR, est une entreprise spécialisée en conseil et ingénierie informatique, filiale du Groupe KOOSSERY TECHNOLOGY France et basée à Douala. Profitant des avancées actuelles en technologies de l'information, Koossery Technology Cameroun améliore sans cesse la pratique de ses missions. Ses solutions inspirées en permanence de l'univers des Innovations et des 'Best Practices ' aux standards mondiaux, lui permettent de porter son approche partenariale et sa vision agile. Koossery Technology développe ses solutions autour de quatre principaux métiers : l'Audit conseil(SI), le management des infrastructures techniques (systèmes et réseaux), le développement d'applications spécifiques et l'intégration de Business solutions (ERP, GED, Applications de Gestion des établissements de micro finance).

La diffusion de l'information par l'intermédiaire d'applications Web en modes Intranet, Extranet et Internet se généralise actuellement. Les avantages de ces applications ne sont plus aujourd'hui à démontrer et l'adoption massive par les systèmes d'information(SI) entreprises est un fait indéniable. Nous remarquons, par exemple, la forte émergence d'applications et de systèmes collaboratifs de gestion de contenus en ligne, de syndication, d'importation et d'exportation de données. En plus de développer ces nouvelles applications métiers, les entreprises spécialisées dans le développement d'applications Web doivent maintenant faire face aux demandes de transformations ou de pseudo migration des applications de type Desktops vers le Web [Kadri 2010]. Une application Desktop (« application de bureau ») étant un logiciel applicatif qui affiche son interface graphique dans un environnement de bureau.

Ainsi, des milliers d'applications de différents domaines sont appelées à être déployées sur le Web. Dans un futur proche, il ne sera plus nécessaire d'installer de logiciels sur nos ordinateurs ni même d'y stocker des fichiers. Chaque application ou presque a son pendant en ligne, qu'il s'agisse de retoucher une image, de faire du montage vidéo, de convertir des fichiers, de créer des documents, etc. Le web devient ainsi incontournable, les utilisateurs veulent pouvoir accéder par exemple à leur progiciel de gestion intégré (PGI ou en anglais ERP: Entreprise Resource Planning) de n'importe quel poste de travail, téléphones et PDA (Personal Digital Assistant) y compris, sans rien installer [Smile 2008].

Les applications Web sont actuellement en train de remplacer les applications traditionnelles de type Desktop. L'intérêt de cette migration est double [Kadri 2010]. Tout d'abord, beaucoup d'utilisateurs sont à l'aise et préfèrent la navigation sur le web. Grâce à de nouvelles technologies (AJAX, Flash ou Eclipse RPC) les développeurs d'applications Web peuvent dorénavant concevoir des applications ayant les mêmes aspects et offrant les mêmes avantages et fonctionnalités que les applications Desktops. Enfin, deuxième intérêt, les administrateurs de système se retrouvent à gérer quelques serveurs à la place d'une multitude de machines. Les applications ne sont plus déployées sur les postes clients.

Plusieurs technologies, avec leurs outils associés, sont à ce jour proposées pour le développement des applications Web (AJAX, Flash ou Eclipse RPC). Ainsi la partie Interface homme machine(IHM) des applications Web ne se limite plus à la simple mise en place d'une charte graphique. Elle intègre de plus en plus de traitements avec pour objectif d'augmenter la convivialité et de limiter le nombre des transactions [Kadri 2010]. Cependant ces technologies ne sont destinées qu'à mener à bien l'étape de codage. Une réflexion sur l'architecture de l'application Web est rarement un sujet de préoccupation. En conséquence, certaine application Web pose de sérieux problèmes de maintenance et d'évolution. C'est le cas de l'application web de l'ERP ADempiere [ADempiereWIKI] qui résulte de la migration de son application de type Desktops.

D'une manière générale, une application peut être découpée en trois niveaux d'abstraction clairement définis [Dembski 2005] :

- La *couche présentation*, ou encore IHM, permet l'interaction entre l'utilisateur final de l'application et le reste du système. Cette couche gère la souris, les saisies clavier. Elle doit être le plus ergonomique possible.
- La *logique applicative*, les traitements décrivant les travaux à réaliser par l'application. Ils peuvent être découpés en deux familles :
    - Les traitements locaux, regroupant les contrôles effectués au niveau du dialogue avec l'IHM.
    - Les traitements globaux, constituant l'application elle-même. Cette couche, appelée Business Logic ou couche métier, contient les règles internes de l'application.
- Les *données*, ou plus exactement l'accès aux données, regroupant l'ensemble des mécanismes permettant la gestion des informations stockées par l'application.

Cependant l'application web l'ERP ADempiere est basée sur l'architecture MVC (Modèle/Vue/Contrôleur) ayant pour inconvénient une multiplication du nombre de contrôleurs nécessaire à l'application web : l'implémentation de plusieurs contrôleurs nécessite beaucoup de code à produire ; rendant l'application web pas évidente pour une prise en main aisée par un développeur. De plus dans l'application web de l'ERP ADempiere nous avons du mal à distinguer clairement la couche présentation, la couche métier et les données : elles restent trop fortement liées entre elles; rendant ainsi l'application web difficilement maintenable et évolutive.

KOOSSERY TECHNOLOGY CMR dans le souci d'offrir une application web flexible, adaptée et pas onéreux à ses partenaires, décide donc de baser l'application web de l'ERP ADempiere sur le framework de développement d'application web Struts 2, permettant de rendre l'application web plus facile à maintenir et à étendre (ajout de nouvelles fonctionnalités); travail qui a été effectué pendant le stage académique.

Comme le framework Struts 2 implémente l'architecture MVC 2 (Modèle, Vue, Contrôleur avec contrôleur unique), baser l'application web de l'ERP ADempiere sur le framework Struts 2, revient en faisant une abstraction, à migrer l'application web de l'ERP ADempiere vers l'architecture MVC 2.

En définitive le problème de ce mémoire est : « Comment migrer une Interface Homme Machine de l'architecture MVC vers l'architecture MVC 2 ? »

Nous envisageons obtenir de l'étude une méthodologie de migration de l'architecture MVC d'une IHM vers l'architecture MVC 2. L'IHM basée sur l'architecture MVC 2 permettra : une prise en main de l'IHM aisée par un développeur, de simplifier la maintenance de l'IHM et une meilleure évolutivité de l'IHM.

Pour migrer l'IHM de l'architecture MVC vers l'architecture MVC 2, nous procéderons par un refactoring de l'IHM en mettant œuvre certaines techniques de refactoring du domaine génie logiciel. En effet le refactoring est une activité de maintenance du domaine de génie logiciel d'un genre particulier. Il s'agit non pas d'une tâche de correction des anomalies ou de réalisation de petites évolutions pour les utilisateurs mais d'une activité plus proche de la maintenance préventive et adaptative dans le sens où elle n'est pas directement visible de l'utilisateur.

Ce mémoire est organisé sur trois chapitres :

Le premier chapitre dans lequel nous allons définir les différents concepts qui interviennent dans le sujet, le génie logiciel domaine dans lequel se situe notre travail, le refactoring, l'IHM, architecture d'IHM, l'architecture MVC, l'architecture MVC 2 et la migration informatique.

Le second chapitre présente notre méthodologie de migration, les outils scientifiques utilisés et l'illustration sur l'application web de l'ERP ADempiere.

Le troisième chapitre entièrement consacré à l'étude de cas : est la réalisation de l'application web de l'ERP KoosseryADempiere.

CHAPITRE I : DEFINITION DES CONCEPTS

Ce chapitre vise à donner une vue générale sur les notions nécessaires à la compréhension du travail développé dans ce mémoire. Il aborde successivement les points suivants, le génie logiciel domaine dans lequel se situe notre travail, le refactoring, l'IHM, architecture d'IHM, l'architecture MVC, l'architecture MVC 2 et la migration informatique.

I.1. Le génie logiciel

L'informatisation est le phénomène le plus important de notre époque. Elle s'immisce maintenant dans la plus part des objets de la vie courante et ce, que ce soit dans l'objet proprement dit (Par exemple, aujourd'hui, 90% des nouvelles fonctionnalités des automobiles sont apportées par l'électronique et l'informatique embarquées. Il y a, ou aura à terme, du logiciel partout : ampoules électriques, four à micro-ondes, tissus des vêtements, stylos, livres, etc.), ou bien dans le processus de conception ou de fabrication de cet objet [Laurent A. 2006].

Actuellement, l'informatique est au cœur de toutes les grandes entreprises. Le système d'information d'une entreprise est définit comme l'ensemble des ressources matérielles, humaines et informatiques contribuant à la collecte, la mémorisation, la recherche, la communication et l'utilisation des données nécessaires et suffisantes pour un pilotage des opérations au sein du système de l'entreprise [Karim 2000]. Depuis quelques années, la fabrication du matériel est assurée par quelques fabricants seulement. Ce matériel est relativement fiable et le marché est standardisé. Les problèmes liés à l'informatique sont essentiellement des problèmes de logiciel.

I.1.1. Les logiciels

Un logiciel est un ensemble d'informations relatives à des traitements effectués automatiquement par un appareil informatique. Y sont incluses les instructions de traitement, regroupées sous forme de programmes, des données et de la documentation. Le tout est stocké sous forme d'un ensemble de fichiers dans une mémoire (exemple : logiciel de comptabilité, logiciel de gestion des prêts).

Les logiciels, suivant leur taille, peuvent être développés par une petite équipe, ou un ensemble d'équipes coordonnées. Le développement de grands logiciels par de grandes

équipes pose d'importants problèmes de conception et de coordination. Or, le développement d'un logiciel est une phase absolument cruciale qui monopolise l'essentiel du coût d'un produit et conditionne sa réussite et sa pérennité [Laurent A. 2006].

En 1995, une étude du Standish Group dressait un tableau accablant de la conduite des projets informatiques. Reposant sur un échantillon représentatif de 365 entreprises, totalisant 8380 applications, cette étude établissait que [Laurent A. 2006]:

- 16, 2% seulement des projets étaient conformes aux prévisions initiales,
- 52, 7% avaient subi des dépassements en coût et délai d'un facteur 2 à 3 avec diminution du nombre des fonctions offertes,
- 31, 1% ont été purement abandonnés durant leur développement.

Pour les grandes entreprises (qui lancent proportionnellement davantage de gros projets), le taux de succès est de 9% seulement, 37% des projets sont arrêtés en cours de réalisation, 50% aboutissent hors délai et hors budget [Laurent A. 2006].

L'examen des causes de succès et d'échec est instructif : la plupart des échecs proviennent non de l'informatique, mais de la maîtrise d'ouvrage, en comprenant sous ce terme à la fois les dirigeants et les concepteurs des métiers. Pour ces raisons, le développement de logiciels dans un contexte professionnel suit souvent des règles strictes encadrant la conception et permettant le travail en groupe et la maintenance du code. Ainsi, une nouvelle discipline est née [Laurent A. 2006] : le génie logiciel.

I.1.2. Le génie logiciel

Le génie logiciel est un domaine de recherche qui a été défini (fait rare) du 7 au 11 octobre 1968, à Garmisch-Partenkirchen, sous le parrainage de l'OTAN. Il a pour objectif de répondre à un problème qui s'énonçait en deux constatations : d'une part le logiciel n'était pas fiable, d'autre part, il était incroyablement difficile de réaliser dans des délais prévus des logiciels satisfaisant leur cahier des charges [Laurent A. 2006].

L'objectif du génie logiciel est d'optimiser le coût de développement du logiciel. L'importance d'une approche méthodologique s'est imposée à la suite de la crise de l'industrie du logiciel à la fin des années 1970. Cette crise de l'industrie du logiciel était principalement due à [Laurent A. 2006]:

- L'augmentation des coûts ;
- Les difficultés de maintenance et d'évolution ;

- La non fiabilité ;
- Le non-respect des spécifications ;
- Le non-respect des délais.

La maintenance est devenue une facette très importante du cycle de vie d'un logiciel. En effet, une enquête effectuée aux USA en 1986 auprès de 55 entreprises révèle que 53% du budget total d'un logiciel est affecté à la maintenance. Ce coût est réparti comme suit [Laurent A. 2006]:

- 34% maintenance évolutive (modification des spécifications initiales) ;
- 10% maintenance adaptative (nouvel environnement, nouveaux utilisateurs) ;
- 17% maintenance corrective (correction des bogues) ;
- 16% maintenance perfective (améliorer les performances sans changer les spécifications) ;
- 6% assistance aux utilisateurs ;
- 6% contrôle qualité ;
- 7% organisation/suivi ;
- 4% divers.

Pour apporter une réponse à tous ces problèmes, le génie logiciel s'intéresse particulièrement à la manière dont le code source d'un logiciel est spécifié puis produit. Ainsi, le génie logiciel touche au cycle de vie des logiciels [Laurent A. 2006] :

- L'analyse du besoin,
- L'élaboration des spécifications,
- La conceptualisation,
- Le développement,
- La phase de test,
- La maintenance.

Les projets relatifs à l'ingénierie logicielle sont généralement de grande envergure et dépassent souvent les 10000 lignes de code. C'est pourquoi ces projets nécessitent une équipe de développement bien structurée. La gestion de projet se retrouve naturellement intimement liée au génie logiciel [Laurent A. 2006].

I.1.3. Le rôle de la maintenance dans l'évolution logicielle

La maintenance est une activité majeure de l'ingénierie logicielle. C'est en partie grâce à elle que le logiciel respecte le niveau d'exigence des utilisateurs et permet, dans une certaine

mesure, que les investissements consacrés au logiciel soient rentabilisés. D'une manière générale, la maintenance est un processus qui se déroule entre deux versions majeures d'un logiciel. Ce processus produit des correctifs ou de petites évolutions, qui sont soit diffusé en production au fil de l'eau, soit regroupés sous forme de versions mineures [Retaille 2005]. La figure 1 illustre les différentes étapes du processus de maintenance pour la gestion des anomalies et des évolutions mineures.

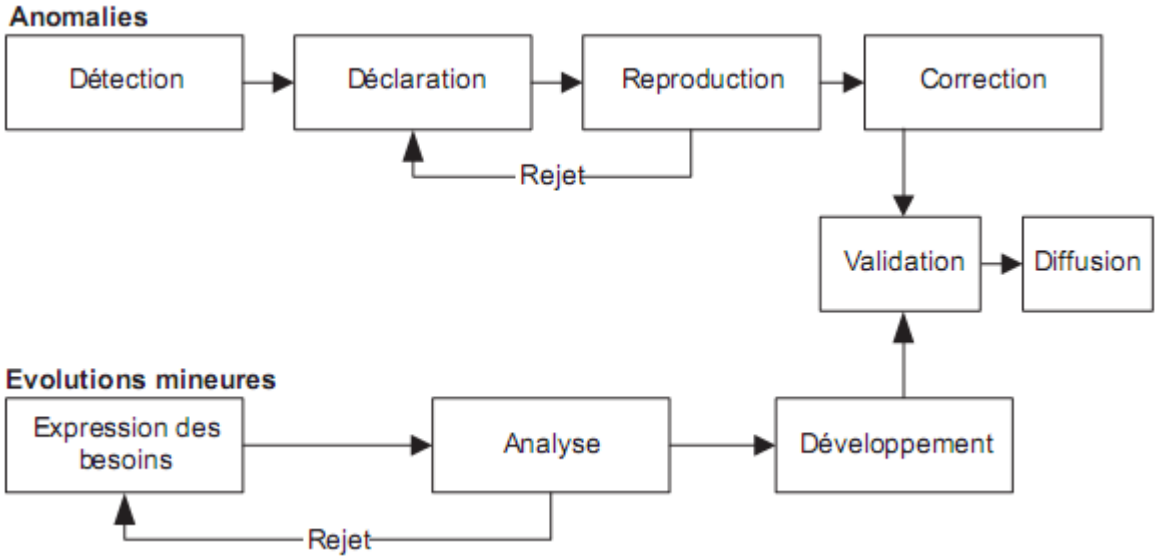


Figure 1 - Le processus de maintenance [Retaille 2005]

Pour la gestion des anomalies, appelée maintenance corrective, le processus de maintenance est très proche de celui d'une recette. La principale différence est que la détection et la déclaration ne sont pas réalisées par des testeurs mais directement par les utilisateurs finals. De ce fait, la tâche de correction est plus difficile. Les déclarations sont généralement moins précises, et la reproductibilité, qui est une condition nécessaire à la correction, est complexifiée faute d'un scénario précis à rejouer. À cela s'ajoutent les impératifs de production en termes de délais de réaction, de contraintes de diffusion, etc [Retaille 2005].

Pour la gestion des évolutions mineures, appelée maintenance évolutive, le processus de maintenance s'apparente à la gestion d'un petit projet de développement. La principale différence réside dans la phase d'analyse, où une demande peut être rejetée si elle constitue une évolution majeure devant être prise en compte dans la prochaine version du logiciel.

Signalons aussi les deux autres types de maintenance suivants [Retaille 2005] :

- *Maintenance préventive* : il s'agit de réaliser des développements pour prévenir des dysfonctionnements futurs.

- *Maintenance adaptative* : il s'agit d'adapter le logiciel aux changements survenant dans son environnement d'exécution.

En conclusion, sur l'ensemble de la vie d'un logiciel, la durée de la maintenance est généralement supérieure à celle du développement. Le cycle développement mise en production est beaucoup plus court pour la maintenance et comprend de nombreuses itérations [Retaille 2005].

I.2. Le refactoring

Le *refactoring* est une opération de maintenance du code informatique. Elle consiste à retravailler le code source non pas pour ajouter une fonctionnalité supplémentaire au logiciel mais pour améliorer sa lisibilité, simplifier sa maintenance, ou changer sa généricité (on parle aussi de remaniement). Le refactoring consiste à refondre le code source d'une application existante ou en cours de développement pour en améliorer la qualité, avec pour objectif une réduction des coûts de maintenance et une meilleure évolutivité [Retaille 2005]. Il s'agit non pas d'une tâche de correction des anomalies ou de réalisation de petites évolutions pour les utilisateurs mais d'une activité plus proche de la maintenance préventive et adaptative dans le sens où elle n'est pas directement visible de l'utilisateur.

Le refactoring est une activité d'ingénierie logicielle consistant à modifier le code source d'une application de manière à améliorer sa qualité sans altérer son comportement du point de vue de ses utilisateurs. Son rôle concerne essentiellement la pérennisation de l'existant, la réduction des coûts de maintenance et l'amélioration de la qualité de service au sens technique du terme (performance et fiabilité) [Retaille 2005]. Mises au point très tôt sur des langages orientés objet comme Smalltalk ou C++, les techniques de refactoring reposent essentiellement sur les meilleures pratiques de développement objet et sont donc généralisables à tous les langages reposant sur ce paradigme [Retaille 2005].

I.2.1. Les techniques de refactoring

Les techniques suivantes forment la base du refactoring : Renommage, Extraction d'une méthode, Extraction de variable locale, Extraction de constant, Extraction d'interface, Changement de signature d'une méthode, Généralisation d'un type, Déplacement d'éléments, Déplacement d'éléments dans le graphe d'héritage, Remplacement d'une variable locale par une expression, Remplacement d'une méthode par son corps, Remplacement d'une constante par sa valeur littérale.

Nous ne prétendons pas fournir un catalogue exhaustif des techniques existantes, mais nous nous concentrons sur celles qui sont supportées par les environnements de développement logiciel tels qu'Eclipse, JBuilder ou IntelliJ IDEA. Ce choix est justifié par le fait qu'une mise en œuvre manuelle, sans l'aide d'un outil, d'une technique de refactoring est généralement lourde, fastidieuse et sujette à erreur stupide, comme des fautes de frappe. Martin Fowler [Fowler 99], auteur du livre précurseur Refactoring: *Improving the Design of Existing Code*, met à la disposition des internautes sur son site Web un catalogue des techniques de refactoring formalisées à ce jour [Retaille 2005].

Les techniques plus avancées du refactoring sont les suivantes : Refactoring avec les design patterns, Refactoring avec la programmation orientée aspect, Refactoring de base de données.

I.2.2. Le processus de refactoring

À l'instar de tout projet logiciel, le refactoring s'inscrit dans un processus comportant les étapes fondamentales suivantes [Retaille 2005]:

1. La préparation, qui consiste à mettre en place, si ce n'est déjà fait, les outils permettant de gérer les changements et les valider.
2. L'analyse du logiciel, qui consiste à identifier les éléments du logiciel nécessitant un refactoring et à sélectionner ceux qui sont pertinents.
3. La réalisation des opérations de refactoring (technique de refactoring).
4. La validation du refactoring, qui consiste à vérifier la non-régression du logiciel du point de vue des utilisateurs, tant pour les aspects fonctionnels que pour la qualité de service.
5. Si le logiciel en cours de refactoring subit des maintenances correctives en parallèle, une fusion de la version de maintenance et de la version de refactoring est nécessaire. Des points de synchronisation doivent être mis en place.

Le processus que nous venons de décrire considère implicitement que le refactoring est un projet en lui-même. Cependant, du fait de sa nature quelque peu ésotérique pour les utilisateurs, puisqu'il s'agit d'une série d'opérations techniques, le refactoring est rarement appliqué en dehors des projets d'évolution ou de maintenance des logiciels. Cette intégration du refactoring au sein d'un projet ayant un périmètre plus large ne remet pas en cause les étapes fondamentales ci-dessus, car celles-ci sont propres à tout projet d'évolution, qu'il soit technique ou fonctionnel. Ces étapes doivent simplement intégrer les problématiques spécifiques des autres types d'évolution. Dans le cas des méthodes agiles, l'étape de

refactoring est clairement identifiée et s'inscrit de manière visible dans la démarche projet [Retaille 2005].

I.3. L'Interface Homme Machine

Les interfaces homme machine (IHM) se retrouvent dans la quasi-totalité des systèmes informatiques ; ainsi, selon [Mye 95] 97% des applications possèdent une interface utilisateur. Dans plusieurs cas, ces IHM constituent le seul point d'entrée de l'utilisateur dans ces systèmes. Elles ont pour but d'établir un lien entre les fonctionnalités brutes du système (la sémantique de l'application, regroupée dans un composant appelé noyau fonctionnel) et l'utilisateur. L'IHM agit donc comme un élément médiateur (voir figure 2).

Au cours des vingt dernières années, les interfaces utilisateurs sont devenues de plus en plus complexes. S'il existe toujours des interfaces de type formulaire où l'interaction se limite au déplacement dans un masque de saisie par l'intermédiaire de touches « fonctions », on trouve couramment aujourd'hui des interfaces qui utilisent des interactions plus évoluées (manipulation directe, utilisation de la voix, du geste…) rendant ces interfaces de plus en plus complexes. La prise en compte de cette complexité se traduit par une part importante du code dévolue à la programmation de l'IHM et donc un effort de développement également important. En conséquence le risque d'erreurs qui pourraient être introduites lors des étapes de conception augmente [Baron 2003]. Selon [MR 92], le développement lié à l'interface représente 48% du code d'une application et monopolise 50% du temps imparti au processus de développement et 37% pour la phase de maintenance [Baron 2003]

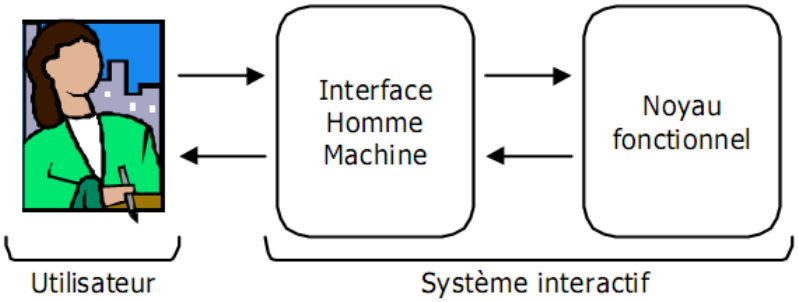


Figure 2 - L'interface, un intermédiaire entre l'homme et la machine [Baron 2003]

Une application interactive ou système interactif est un logiciel qui communique avec un utilisateur (Figure 2). Les échanges s'effectuent grâce à une interface homme-machine (IHM), qui prend en charge [Kinan s. 2006] :

- La communication de l'utilisateur humain vers le noyau fonctionnel de l'application (entrée d'information) ;
- La communication du noyau fonctionnel vers l'utilisateur humain (sortie d'information);
- La traduction entre les variables informatiques du monde de l'application et celles psychologiques et/ou réelles de l'utilisateur.

L'analyse de la littérature met en évidence plusieurs types de modèles mettant l'accent sur différents aspects de la conception des interfaces et faisant intervenir plusieurs acteurs dans le processus de construction des systèmes interactifs [Kinan s. 2006] :

- les *modèles de tâches* : destinés au concepteur d'application, auquel ils fournissent un schéma général de l'activité de l'utilisateur du système ;

- les *modèles de dialogue* : destinés au concepteur de dialogue pour décrire la structure des échanges entre l'homme et l'application. Ils identifient les objets du dialogue ;

- les *modèles d'architecture* : ils fournissent aux réalisateurs une structure générique de l'application à partir de laquelle il est possible de construire un système interactif particulier.

De nombreux modèles décrivent les cycles de développement et sont particulièrement adaptés au processus de développement des IHM : le modèle en spirale qui introduit un processus itératif, le modèle en couches, ou le modèle en étoile. Mais nous nous limiterons au modèle en V comme support pédagogique pour la description des différents modèles, notations et outils (figure 3) qui servent à l'analyse, la spécification et la conception des systèmes interactifs [Baron 2003].

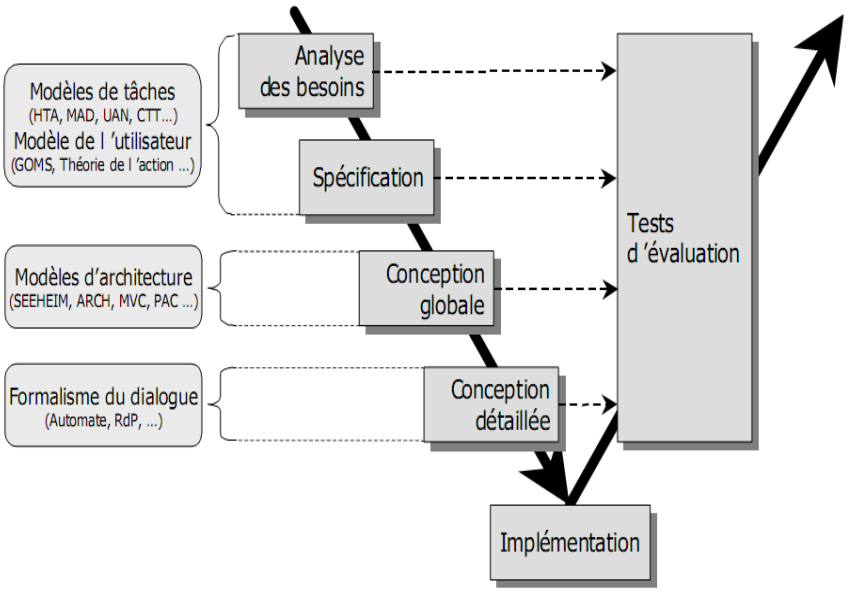


Figure 3 - Modèles intervenant dans le cycle de développement en V pour les systèmes interactifs [Baron 2003]

Le domaine de l'interface homme-machine est bien plus qu'une préoccupation d'interface. L'interface homme-machine doit satisfaire à des critères de qualité, par exemple celui d'utilisabilité du système interactif, c'est-à-dire être en adéquation avec les besoins et les capacités de l'utilisateur, afin de permettre à cet utilisateur d'atteindre ses objectifs à travers des trajectoires d'interactions intuitives et sûres. Dans le but d'assurer ces critères de qualité, l'interface doit [Baron 2003]:

- Refléter exactement et fidèlement le comportement de l'application : c'est l'aspect informatique ;
- Faciliter son utilisation : c'est l'aspect ergonomique ;
- Etre accessible au plus grand nombre d'utilisateur ayant des comportements différents : c'est l'aspect psychologique.

Pour ce faire, la conception d'une IHM nécessite la collaboration de spécialistes issus de différents métiers. Elle fait intervenir non seulement des informaticiens pour la conception et le développement mais aussi des ergonomes pour faciliter son utilisation, et des psychologues pour capturer les comportements des utilisateurs. Cette pluridisciplinarité contribue également à l'accroissement de la complexité du développement par l'allongement du processus de développement avec les phases de tests devant prendre en compte ces disciplines [Baron 2003].
Le recours à des modèles et des méthodes s'avère indispensable afin d'assurer l'utilisabilité du système interactif. Ces modèles et méthodes proviennent des différentes communautés citées précédemment (informaticiens, ergonomes, psychologues…) et constituent aujourd'hui un champ de recherche très actif [Baron 2003].

I.4. Architecture d'Interface Homme Machine

Les architectures logicielles ou modèles constituent un des éléments indispensables sur lequel s'appuie le génie logiciel moderne. Il systématise à la fois l'élaboration et l'utilisation des logiciels et joue un rôle primordial dans leur qualité et leur efficacité. Garlan et Perry soulignent le fait que l'utilisation de modèles peut avoir des effets bénéfiques sur au moins cinq aspects du développement des logiciels [Garlan et al 1995] :

- La *compréhension* est simplifiée en présentant la structure de grands systèmes à un niveau d'abstraction adapté. Les contraintes et choix de conception peuvent ainsi être mieux explicités et expliqués ;
- La *réutilisation* est favorisée. Bien sûr, la réutilisation des composants est largement encouragée par la définition d'architectures, mais il est également possible de réutiliser

des structures plus complexes définies sous forme de motifs de conception ou design patterns ;

- L'*évolution* et la *maintenance* profitent largement de la définition d'architectures logicielles. En comprenant les ramifications et les connexions de chaque composant, il est beaucoup plus facile de déterminer a priori les coûts potentiels de toute modification ;
- L'*analyse* des systèmes est simplifiée. Suivant les modèles utilisés, on peut envisager des formes de vérification de cohérence, de respect de styles, de satisfaction de critères de qualité voire d'adaptation à des domaines spécifiques d'application ;
- La *gestion* des projets industriels doit s'appuyer sur cet élément essentiel pour déterminer au plus tôt la viabilité des options choisies, les prérequis indispensables et les perspectives d'évolution.

Les modèles sont nés du constat que la conception des IHM est un processus complexe, donc itératif, et par conséquent la possibilité d'apporter des modifications au logiciel d'IHM est un élément important du cahier des charges. Dans ce sens, la décomposition de l'application interactive en différents éléments directeurs est rendue nécessaire. Les modèles définis pour les IHM reposent sur un principe commun : la séparation du modèle sémantique de l'application (noyau fonctionnel) de l'interface fournie à l'utilisateur pour interagir avec le modèle. Les modèles décrivent également la manière dont ces deux composants communiquent entre eux. Ainsi, ils offrent une structure générique destinée au concepteur de l'application [Baron 2003].

Les apports d'une telle organisation modulaire sont importants. D'une part, cette organisation permet une conception plus simple dans la mesure où chaque module peut être réalisé de manière plus ou moins indépendante. D'autre part, les modifications apportées aux modules s'en trouvent amoindries et leur fiabilité accrue [Baron 2003].

Les modèles des applications interactives peuvent être classés en deux grandes catégories. Les modèles globaux tels que le modèle de SEEHEIM [Pfaff 1985] et modèle ARCH [UIMS 1992] s'intéressent à la séparation de la partie interface du reste de l'application, afin de rendre cette dernière indépendante de tout élément spécifique de l'application. Ces modèles sont également appelés modèles logiques, ou sont qualifiés d'abstraits. La seconde catégorie contient les *modèles multi-agents*, ou encore *modèles orientés objet* tels que le modèle MVC [Krasner et al 1988] et le modèle PAC [Coutaz 1990] ont été mis en place avec l'avènement des « *techniques orientées-objets* » pour répondre aux exigences liées à la modularité,

l'encapsulation et au parallélisme. Ces modèles proposent une décomposition modulaire de l'interface en un ensemble d'agents communicants et réagissant à la réception de certains événements. Par ailleurs, des *modèles mixtes* tels que PAC-Amodeus [Nigay et al 1993] ou H4 [Guittet 1995] tentent de tirer parti des avantages respectifs de ces deux approches en les combinant.

I.5. L'architecture MVC

L'architecture ou modèle MVC (Modèle-Vue-Contrôleur) est issu du langage orienté-objet Smalltalk-80 [Krasner et al, 1988]. De fait, dès le début des années 80, Smalltalk appuyait son organisation IHM sur ce modèle. Depuis, le modèle s'est largement répandu et est désormais connu et reconnu comme un modèle de conception très mature.

Le modèle MVC s'appuie essentiellement sur la séparation en deux couches verticales regroupant d'un côté les objets métiers (Modèle) et de l'autre les objets IHM, ces derniers étant eux-mêmes regroupés en objets chargés de l'acquisition d'informations en provenance de l'utilisateur (Contrôleur) et en objets chargés de la restitution d'informations vers l'utilisateur (Vue) ; voir figure 4. Il est recommandé comme modèle pour plate-forme J2EE [Zango et Ouattara 2006].

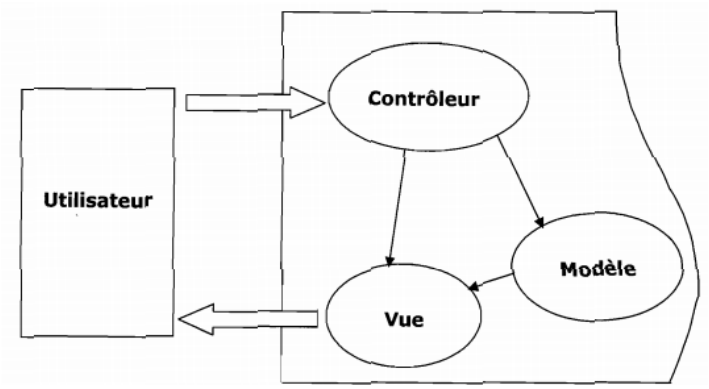


Figure 4 - Le modèle MVC [Zango et Ouattara 2006]

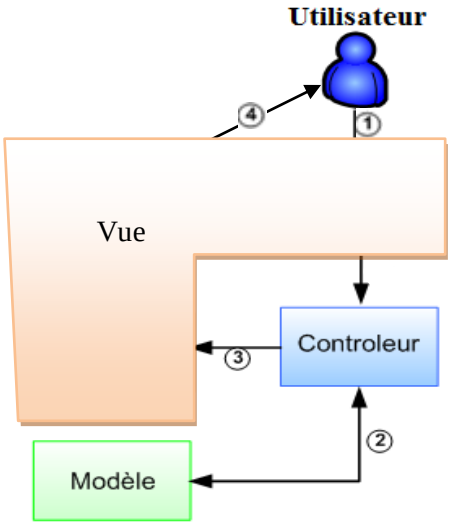
I.5.1. Fonctionnement du MVC

MVC est un modèle de conception qui impose la séparation entre données, traitements et présentation. C'est pour cette raison que l'application est divisée en trois composants fondamentaux: le modèle, la vue et le contrôleur. Chacun de ces composants tient un rôle bien défini.

Ainsi, ce modèle se décompose en trois composants [Zango et Ouattara 2006]:

- *Le Modèle*: Le modèle représente le comportement de l'application : traitements des données, interactions avec la base de données, etc. Il décrit ou contient les données manipulées par l'application. Il assure la gestion de ces données et garantit leur intégrité. Dans le cas typique d'une base de données, c'est le modèle qui la contient. Le modèle offre des méthodes pour mettre à jour ces données (insertion, suppression, changement de valeur). Il offre aussi des méthodes pour récupérer ces données. Les résultats renvoyés par le modèle sont dénués de toute présentation;
- *La Vue*: La vue correspond à l'interface avec laquelle l'utilisateur interagit. Sa première tâche est de présenter les résultats renvoyés par le modèle. Sa seconde tâche est de recevoir toutes les actions de l'utilisateur (clic de souris, sélection d'une entrée, boutons, etc). Ces différents événements sont envoyés au contrôleur. La vue n'effectue aucun traitement, elle se contente d'afficher les résultats des traitements effectués par le modèle et d'interagir avec l'utilisateur;
- *Le Contrôleur*: Le contrôleur prend en charge la gestion des événements de synchronisation pour mettre à jour la vue ou le modèle et les synchroniser. Il reçoit tous les événements de l'utilisateur et enclenche les actions à effectuer. Si une action nécessite un changement des données, le contrôleur demande la modification des données au modèle, ce dernier avertit la vue que les données ont changée pour qu'elle se mette à jour. Certains événements de l'utilisateur ne concernent pas les données mais la vue. Dans ce cas, le contrôleur demande à la vue de se modifier. Le contrôleur n'effectue aucun traitement, ne modifie aucune donnée. Il analyse la requête du client et se contente d'appeler le modèle adéquat et de renvoyer la vue correspondant à la demande.

Le schéma suivant présente les relations structurelles entre les trois objets:



Cinématique

1. L'utilisateur émet une requête depuis la vue
2. Le contrôleur intercepte la requête de l'utilisateur, détermine quelle partie du modèle est concernée. Le modèle traite les interactions avec les données, applique les règles métier et renvoie les données au contrôleur
3. Le contrôleur sélectionne la vue et lui renseigne les données
4. La vue présente les données à l'utilisateur

Il est important de noter que la vue et le contrôleur dépendent (le sens des flèches) tous deux du modèle. En revanche, le modèle ne dépend ni de la vue ni du contrôleur. C'est l'un des principaux avantages de la séparation. Cette séparation des tâches permet de créer et de tester le modèle indépendamment de la représentation visuelle. La séparation entre la vue et le contrôleur est secondaire dans la plupart des applications clientes riches et, en effet, de nombreuses infrastructures d'interface utilisateur implémentent les rôles comme un seul objet [Zango et Ouattara 2006].

I.5.2. Les principaux avantages du MVC

La plupart des applications Web sont créées à l'aide de langages procéduraux tels que ASP ou PHP ce qui entraîne qu'on est rapidement tenté de mélanger traitements, données et présentation. Alors MVC impose la séparation entre les différentes parties.

La *Vue* correspond à l'interface avec laquelle l'utilisateur interagit. Auparavant, dans le cas des applications Web, il s'agissait d'une interface HTML et la gestion de toutes ces interfaces dans l'application devenait alors de plus en plus difficile. MVC permet de gérer l'utilisation de différentes vues. Cependant, aucun traitement n'est effectué dans celles-ci. Elles servent uniquement à afficher les données et permettre à l'utilisateur d'agir sur ces données [Zango et Ouattara 2006].

Le deuxième composant du MVC, *le Modèle*, représente les données et les règles métiers. C'est à ce niveau que s'effectuent les traitements. Les bases de données et les objets tels que les EJB (Enterprise Java Beans, composant J2EE pour la gestion des transactions et aussi de la persistance dans la couche M du modèle MVC) en font partie. Les données renvoyées par le modèle sont indépendantes de la présentation, c'est-à-dire que le modèle ne réalise aucune mise en forme. Les données d'un seul modèle peuvent ainsi être affichées dans plusieurs vues. Cette capacité permet de factoriser le code, car le code du modèle n'est écrit qu'une seule fois puis réutilisé par toutes les vues [Zango et Ouattara 2006].

De plus, comme la vue est séparée du modèle et qu'il n'y a pas de dépendance directe entre eux, l'interface utilisateur peut afficher simultanément plusieurs vues des mêmes données. Ainsi, plusieurs pages d'une application Web peuvent utiliser les mêmes objets de modèle. L'application Web permettant à l'utilisateur de modifier l'apparence des pages constitue un autre exemple des avantages de cette séparation. Ces pages affichent les mêmes données à partir du modèle partagé mais ces données s'affichent de différentes manières. Aussi, la duplication du code est réduite puisqu'on a séparé les données et la logique métier de la présentation [Zango et Ouattara 2006].

En outre, comme le modèle est autonome et séparé du contrôleur et de la vue, il est beaucoup plus facile de modifier la couche de données ou les règles métier. Par exemple si on change de bases de données et qu'on passe de MySQL à Oracle, il suffira simplement de revoir le modèle. Le fonctionnement interne des trois parties d'une application MVC est masqué aux autres parties. On peut ainsi élaborer des interfaces bien définies et des composants autonomes.

Un autre avantage important de MVC est bien entendu l'adaptation aux changements. Les exigences en matière d'interface utilisateur ont tendance à changer beaucoup plus rapidement que les règles métiers. Les utilisateurs peuvent préférer utiliser différentes couleurs, polices, disposition à l'écran et plusieurs niveaux de prise en charge des nouveaux périphériques tels que les téléphones mobiles ou les assistants personnels. Comme le modèle ne dépend pas des vues, l'ajout de nouveaux types de vues au système ne l'affecte généralement pas. En conséquence, le changement porte uniquement sur la vue.
Le rôle du *Contrôleur* est également important puisqu'il interprète les requêtes de l'utilisateur. Ainsi, lorsque l'utilisateur clique sur un lien ou soumet un formulaire HTML, le contrôleur ne produit rien et n'effectue aucun traitement. Il intercepte la requête et détermine uniquement quels modèles et quelles vues doivent être associés.

I.5.3. Quelques inconvénients

MVC introduit de nouveaux stades d'indirection ce qui augmente légèrement la complexité de la solution. Il accroît également la nature événementielle du code de l'interface utilisateur. Il est indispensable de consacrer du temps à réfléchir aux interactions entre les différentes parties de l'application. De plus, la stricte séparation entre le modèle et la vue peut parfois rendre plus difficile le débogage. Chaque élément devra être soumis à des tests rigoureux avant son utilisation. La gestion des fichiers est aussi plus fastidieuse. Cela peut sembler

évident vu que l'intérêt principal de MVC consiste à séparer les trois couches. Toutefois, la charge de travail supplémentaire induite par la gestion de multiples fichiers ne doit pas être sous-estimée [Zango et Ouattara 2006].

Par conséquent, il en résulte que MVC est beaucoup trop complexe pour les petites applications. Une autre contrainte de l'utilisation de MVC est le coût des mises à jour fréquentes. En effet, dissocier le modèle de la vue ne signifie pas que les développeurs du modèle peuvent ignorer la nature de ces vues. Par exemple, si le modèle subit des modifications fréquentes, il peut envahir les vues de demandes de mise à jour. Et de ce fait, certaines vues, telles que les affichages graphiques, peuvent prendre du temps à s'afficher. La vue peut alors être décalée par rapport aux demandes de mise à jour. Il est donc important de penser à la vue lors du codage du modèle. Le modèle pourra, par exemple, regrouper plusieurs requêtes en une seule notification à la vue [Zango et Ouattara 2006].

I.6. L'architecture MVC 2

Le MVC très pratique, peut se révéler lourd à mettre en place dans les grandes applications. Ceci à cause de la multitude de contrôleur à implémenter comme le montre la figure 5 et cela peut rapidement devenir confus :

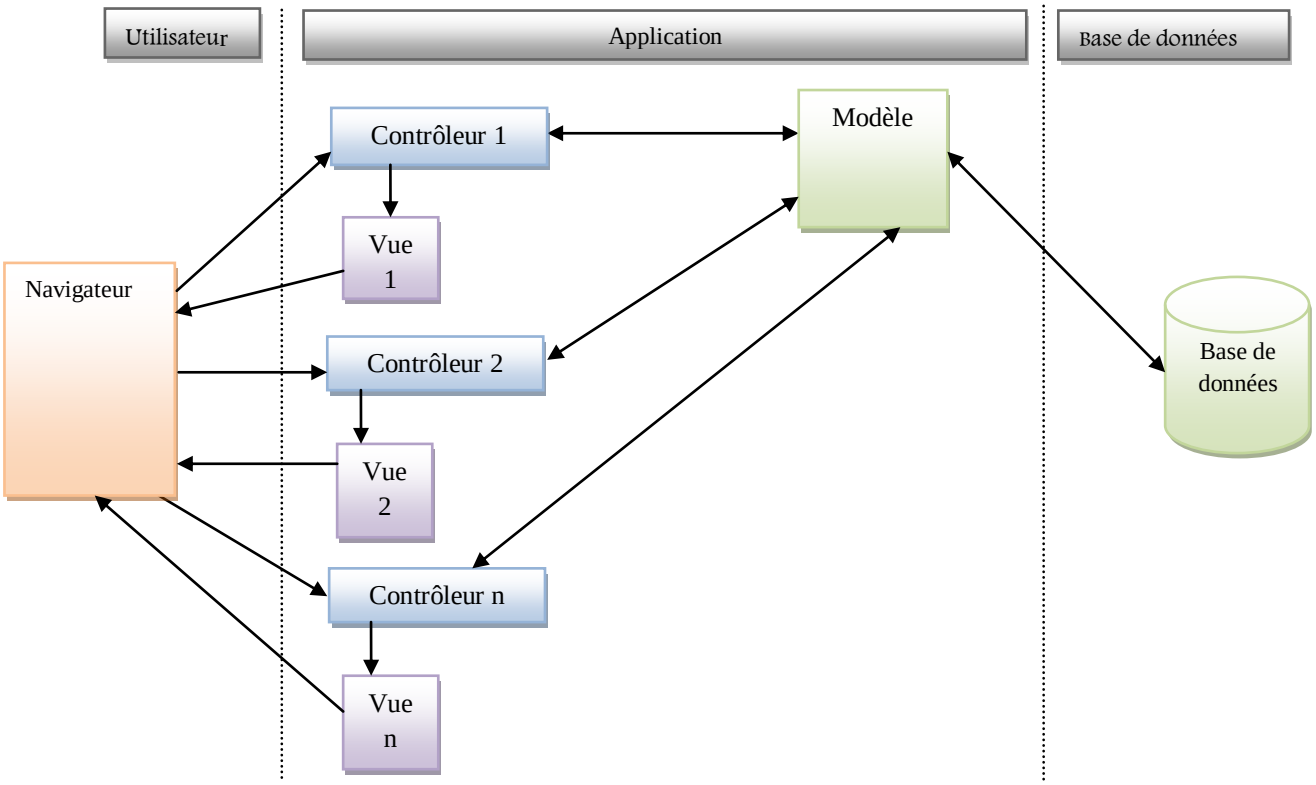


Figure 5 - L'architecture MVC dans une application

Pour simplifier les choses, il conviendra d'utiliser des architectures plus adaptées à des traitements aussi riches et variés. Ainsi le modèle *MVC model 2 ou MVC 2* propose de n'utiliser qu'un seul contrôleur. Ce contrôleur se charge de rediriger la requête vers le bon traitement en fonction d'un *paramétrage généralement sous la forme d'un fichier au format XML* [Jean Michel D. 2011] (figure 6):

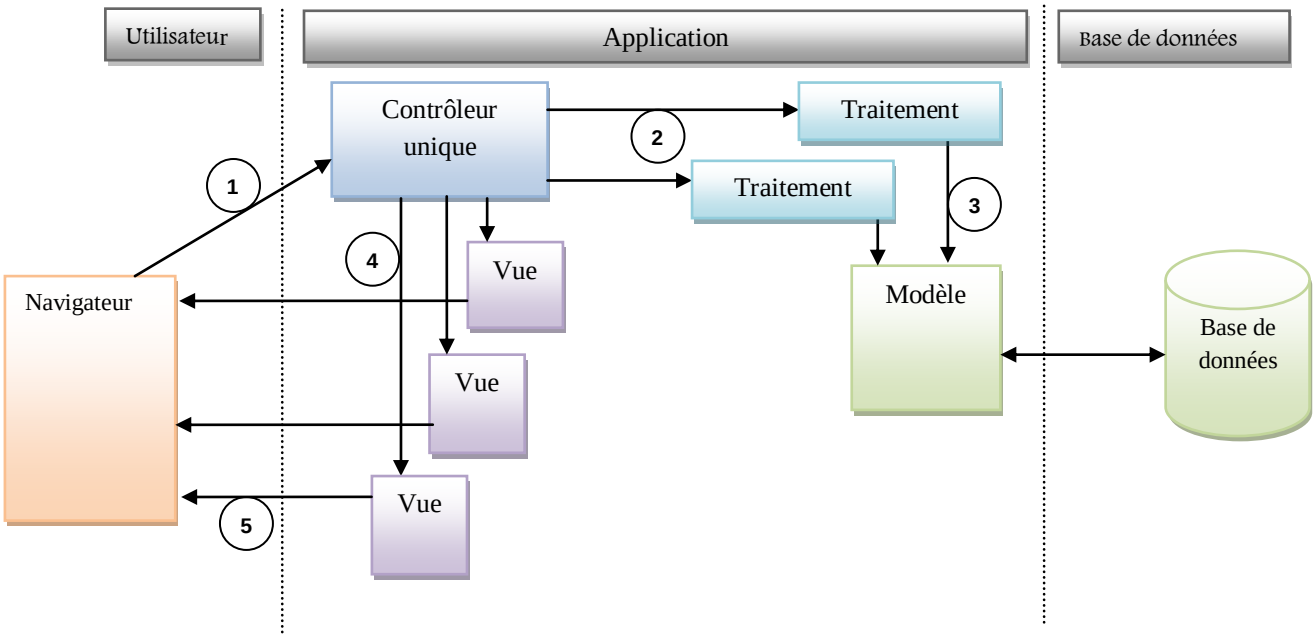


Figure 6 - L'architecture MVC 2 dans une application

Le *modèle MVC 2* conserve les principes du modèle MVC, mais il divise le contrôleur en deux parties en imposant un point d'entrée unique à toute l'application (première partie du contrôleur) qui déterminera à chaque requête reçue les traitements à invoquer (seconde partie du contrôleur).Une application implémentant le modèle MVC 2 utilise *un unique* contrôleur traitant les requêtes. En fonction de la requête, il appelle les traitements dédiés généralement encapsulés dans une classe [Jean Michel D. 2011].

Dans ce modèle, le cycle de vie d'une requête est le suivant (voir figure 6):

1. *Le client envoie une requête à l'application qui est prise en charge par le contrôleur unique.*
2. *Le contrôleur unique analyse la requête et appelle la classe dédiée contenant les traitements.*
3. *Cette classe exécute les traitements nécessaires en fonction de la requête notamment en faisant appel aux objets métiers (Modèle).*
4. *En fonction du résultat des traitements, le contrôleur unique redirige la requête vers la page vue.*

5. *La vue génère la réponse qui est renvoyée au client.*

I.7. Migration informatique

Une migration est, en informatique, le passage d'un état existant d'un système d'information ou d'une application vers une cible définie dans un projet ou un programme. La migration de données est généralement réalisée par programmation pour parvenir à un traitement automatisé, en libérant des ressources humaines de tâches embarrassantes. La migration est nécessaire lorsque des organisations ou des individus procèdent au changement des systèmes informatiques ou à leur mise à niveau [MI 2011].

Les différents types de migrations applicables aux applications informatiques sont les suivantes [Roland O. 2005]:

- *Maintenance* **:** C'est la transformation minimum d'un logiciel pour qu'il reste opérationnel. Cette opération consiste à corriger des anomalies qui apparaissent dans des cas particuliers non encore traités ou lorsque les conditions d'utilisations changent (on parle alors de maintenance corrective). L'exemple le plus connu est le bug de l'an 2000 liés que de nombreux programmes qui notaient l'année sur 2 chiffres allaient considérer que l'année 01 était antérieure à l'année 99.
- *Transformation de système* **:** Certaines évolutions nécessitent une transformation systématique des systèmes d'information. Par exemple pour utiliser un même logiciel pour plusieurs sociétés, il peut être nécessaire d'ajouter systématiquement un code société devant toutes les données et d'utiliser ce code dans tous les traitements. Cette transformation doit être effectuée globalement dans toutes les parties de l'application concernées. Toute l'application, ou au moins des sous-ensembles cohérents doivent être installés en même temps pour continuer à fonctionner correctement.
- *Migration de système* **:** Cette transformation vise à transporter un logiciel (avec ses données) d'un modèle d'ordinateur à un autre (d'un ordinateur MVS vers Unix). Les données auront cette fois la même organisation mais les programmes devront être adaptés pour prendre en compte le mode d'emploi de la future machine. On peut noter qu'une migration de système comprend aussi une migration de données mais qui est un simple transport des données sans réorganisation (migration technique de données). Cette opération est intéressante si les fonctionnalités conviennent mais que l'on souhaite changer d'ordinateur pour regrouper 2 applications, pour disposer de plus de puissance ou pour faire des économies en couts de maintenance. Elle peut aussi être

intéressante comme solution d'attente d'un objectif plus long à atteindre comme par exemple le développement d'une autre application.

- *Rétro conception (re-engineering)* **:** Pour l'informatique il s'agit d'analyser les programmes d'une application informatique dans le but de la réécrire. Il s'agit en général de la modifier en même temps, *ou d'en améliorer la structure*. Elle s'appuie généralement sur des outils d'analyse du code.
- *Migration de données* **:** Cette transformation vise à adapter un ensemble de données pour l'utiliser sur un autre système informatique. Les données initiales (les données sources) doivent subir des transformations pour former les données cibles adaptées aux nécessités du nouveau système. Par exemple les données sources peuvent se baser sur un fichier client contenant les noms et adresses de facturation avec des adresses de livraison rappelées sur chaque bon de livraison. Le système cible peut lui prévoir un fichier d'adresse indépendant auquel il faut se référer par un numéro.

CHAPITRE II : METHODOLOGIE DE MIGRATION

Migrer l'architecture d'une IHM vers une autre architecture implique une démarche rigoureuse car cette opération ne doit en rien modifier au fonctionnement de départ de l'application. La nouvelle solution devra conserver les mêmes propriétés ou caractéristiques que la solution de départ. Ce chapitre est consacré à notre méthodologie de migration de l'architecture MVC d'une IHM vers l'architecture MVC 2. Il aborde successivement les points suivants, les outils scientifiques utilisés, présentation de la méthodologie proprement dite et l'illustration sur l'application web de l'ERP ADempiere.

II.1. Les outils scientifiques utilisés lors de la migration

Nous présentons, quelques techniques de refactoring mise en œuvre lors de la migration : Renommage, Extraction d'une méthode, Extraction d'interface, Changement de signature d'une méthode, Généralisation d'un type, Déplacement d'éléments et enfin le design pattern ou modèle de conception réutilisable Façade (une technique plus avancée du refactoring avec les design patterns).

II.1.1. Renommage

Le renommage consiste à modifier le nom de un ou plusieurs éléments du code. Pour Java, les éléments susceptibles d'être renommés sont les suivants :

- packages ;
- classes ou interfaces ;
- méthodes ;
- attributs de classe ;
- variables locales ;
- paramètres d'une méthode.

La figure 7 illustre le fonctionnement de cette technique

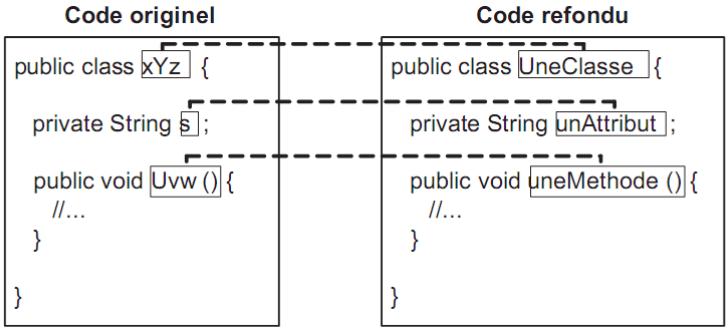


Figure 7- Renommage d'éléments de code [Retaille 2005]

Le renommage est considéré comme une technique de refactoring, car l'absence de règles de nommage ou de mauvaises règles peuvent rendre le code source du logiciel rapidement illisible. L'absence de règles de nommage est préjudiciable au logiciel en ce qu'elle ne favorise pas le travail en équipe et rend difficile l'intégration de nouveaux développeurs [Retaille 2005].

Le principe fondamental à respecter est qu'un nom doit être représentatif de la nature de l'entité qui le porte. Chaque langage comporte certaines conventions, qui doivent aussi être respectées, au-delà du strict respect des règles imposées par sa grammaire. Par exemple, en Java le nom d'une classe doit commencer par une majuscule. S'il s'agit d'un nom composé, chaque composante est démarquée par sa première lettre en majuscule, par exemple ContratAssuranceAuto [Retaille 2005].

Le renommage est une opération de refactoring fastidieuse, même avec l'assistance d'un outil de type rechercher/remplacer, mais peu risquée du point de vue de l'intégrité du logiciel. En effet, la majorité des problèmes liés au renommage sont détectés lors de la compilation [Retaille 2005].

II.1.2. Extraction d'une méthode

L'extraction d'une méthode consiste à sélectionner un bloc de code consistant, dont l'exécution peut se faire de manière indépendante aux variables près, et à le transformer en une méthode voir figure 8.

L'extraction d'une méthode est utile principalement dans les deux cas suivants [Retaille 2005]:

- La méthode contenant le bloc de code à extraire est complexe ou longue. En effectuant l'extraction, nous isolons une partie du traitement et rendons ainsi la méthode source moins complexe et plus lisible. Généralement, les méthodes extraites sont privées, car leurs détails d'implémentation ne sont, par principe, pas visibles de l'extérieur.
- Le bloc de code constitue un élément réutilisable au sein de la méthode source ou au niveau de la classe, voire au-delà. Il est donc intéressant de le transformer en une méthode réutilisable. Si la réutilisation est strictement interne à une méthode ou à une classe, la méthode doit être privée, sinon elle peut être soit protégée, soit publique, en fonction du mode de réutilisation (réutilisation par les descendants ou par des classes externes).

Dans les deux cas, il est important de veiller à ce que la méthode extraite soit pertinente, c'est-à-dire que son contenu ait une signification en lui-même afin de sauvegarder la lisibilité du

code, et qu'elle n'ait aucune adhérence vis-à-vis de la méthode source afin de faciliter sa réutilisation éventuelle.

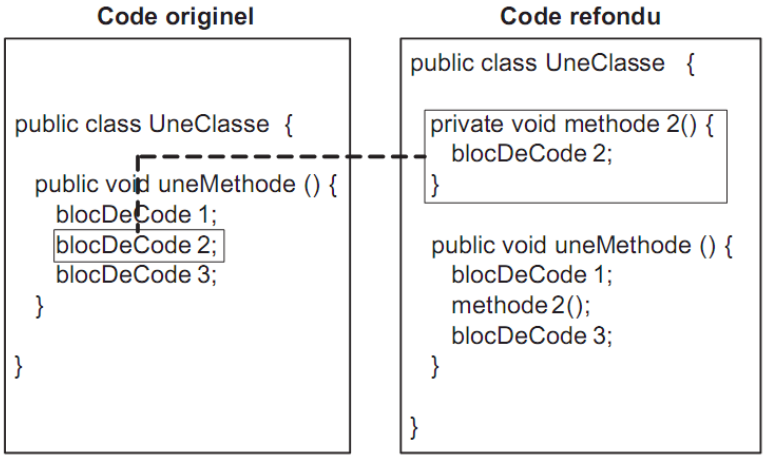


Figure 8 - Extraction d'une méthode [Retaille 2005]

L'extraction de méthode comporte peu de risques, hormis une régression du fonctionnement du bloc de code extrait et de la méthode source. Même s'il est tentant de réaliser une extraction de méthode strictement technique afin de réduire la complexité de la méthode source, il est important de conserver la sémantique du bloc extrait. En cas d'extraction de méthode en vue de sa réutilisation, il faut veiller à ce que toutes les occurrences du bloc de code soient bien remplacées par la nouvelle méthode afin de maximiser les gains de la réutilisation. Plus il y a d'occurrences à remplacer, plus le risque de régression est important [Retaille 2005].

II.1.3. Extraction d'interface

L'extraction d'interface consiste à créer une interface à partir d'une classe existante (voir figure 9). Il est possible de remplacer toutes les références à cette classe par l'interface nouvellement créée lorsque cela est possible, c'est-à-dire quand les références ne dépendent pas de méthodes ou d'attributs absents de l'interface.

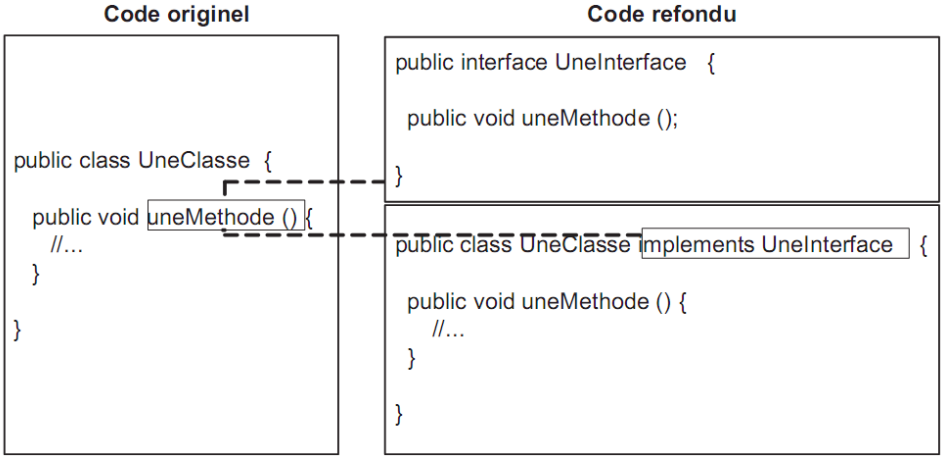


Figure 9 - Extraction d'interface [Retaille 2005]

Le principal objectif d'une extraction d'interface est de dé corréler les services fournis par une classe de leur implémentation. En effet, il peut être utile d'avoir la capacité de modifier la classe d'implémentation sans impacter l'ensemble du logiciel du fait des références statiques à la classe en question. Nous pouvons, par exemple, reporter le choix de l'implémentation à utiliser au moment où le logiciel est lancé en utilisant un fichier de configuration donnant la classe à charger. L'extraction d'interface peut aussi se révéler utile lorsqu'un ensemble de classes partagent un certain nombre de méthodes communes sans pour autant justifier la mise en place d'un arbre d'héritage pour les lier. Grâce à cette interface, des opérations complexes peuvent être définies indépendamment des détails d'implémentation.

Les risques sont nuls pour la classe dont est extraite l'interface, car sa seule modification revient à déclarer qu'elle implémente cette dernière. Le remplacement des références à la classe par l'interface est peu complexe et consiste essentiellement en un rechercher/ remplacer, sauf au moment de la création de l'instance, où il faut faire un cast. Le seul risque ici est de remplacer une référence reposant sur des spécificités de la classe originelle non reprises par l'interface. Dans ce cas, la compilation détecte immédiatement l'anomalie [Retaille 2005].

II.1.4. Changement de signature d'une méthode

Le changement de signature d'une méthode consiste à modifier un ou plusieurs des éléments suivants (voir figure 10) :

- nom de la méthode ;
- portée de la méthode ;
- type de la valeur de retour de la méthode ;
- liste des paramètres de la méthode (nombre de paramètres, type des paramètres, nom des paramètres) ;
- liste des exceptions générées potentiellement par la méthode (nombre et type des exceptions générées).

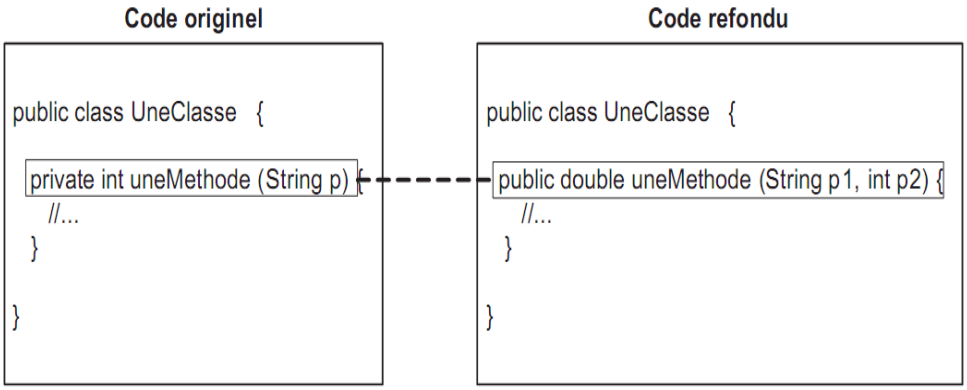


Figure 10 - Changement de signature d'une méthode [Retaille 2005]

Le changement de signature d'une méthode peut intervenir dans l'une ou l'autre des situations suivantes [Retaille 2005]:

- Le nom de la méthode ne respecte pas les conventions de nommage ou n'est pas significatif.
- La portée de la méthode n'est pas adaptée. Une portée trop grande, publique, par exemple, nuit à l'encapsulation en dévoilant les détails d'implémentation de la classe.
- Une portée trop courte, privée, par exemple, nuit à la réutilisation.
- La valeur de retour ou certains paramètres sont inutiles. Leur présence nuit à la compréhension du code. Il est donc utile de les supprimer afin d'alléger la méthode.
- La liste des paramètres est trop longue. Une méthode ayant plus de quatre ou cinq paramètres devient rapidement difficile à manipuler. Il est donc intéressant de chercher à réduire cette liste, soit en réduisant le périmètre de la méthode (voir l'extraction de méthode), soit en encapsulant plusieurs paramètres dans un seul paramètre plus complexe.
- La liste des paramètres est trop courte. Par exemple, une méthode peut être rendue plus facilement réutilisable en ajoutant quelques paramètres supplémentaires.
- Le type d'un paramètre ou de la valeur de retour est inadapté. Par exemple, une méthode effectuant des calculs utilisant des entiers courts peut bloquer sa réutilisation dans des contextes où des nombres plus grands doivent être manipulés.

La modification de la portée d'une méthode est une opération peu risquée puisqu'une compilation permet de détecter immédiatement les problèmes. Dans le cas de valeurs de retour ou de paramètres inutiles, les risques sont faibles, car les traitements effectués par la méthode concernée ne sont pas impactés et n'entraînent donc pas de régression. Pour les autres cas de figure, le changement de signature est une opération lourde et risquée, qui rompt le contrat liant la méthode à ses utilisateurs et nécessite de repenser leur relation [Retaille 2005].

II.1.5. Généralisation d'un type

Le type d'un attribut ou d'une variable locale peut être remplacé soit par un de ses ancêtres, soit par une des interfaces qu'il implémente. Cette opération est possible lorsque l'utilisation d'un attribut ou d'une variable ne repose que sur les fonctionnalités offertes par l'ancêtre ou l'interface remplaçant le type originel.

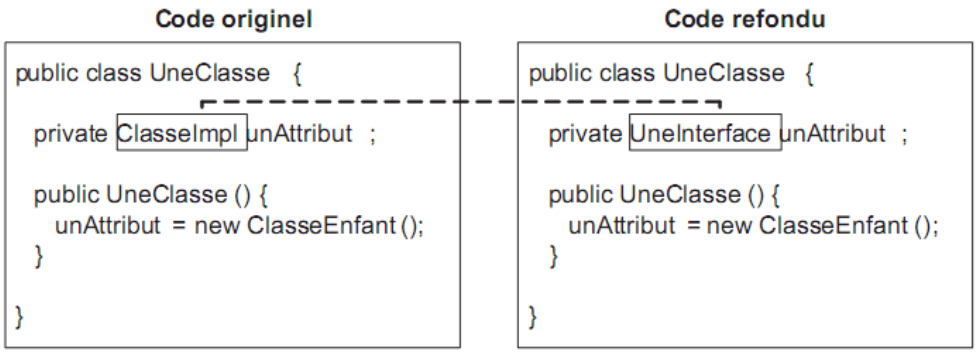


Figure 11 - Généralisation d'un type [Retaille 2005]

Le remplacement d'un type donné par un de ses ancêtres ou par une des interfaces qu'il implémente permet de rendre le code plus générique. Dans le cas d'un paramètre d'une méthode, par exemple, l'utilisation d'un ancêtre permet à cette dernière d'accepter les objets du type ancêtre, mais aussi les objets des types descendants. Il en va de même pour l'utilisation d'une interface en lieu et place d'un type donné. Tous les objets qui implémentent cette interface peuvent être utilisés. L'utilisation des classes ascendantes ou des interfaces permet donc d'améliorer la réutilisation.

Les risques liés à une généralisation de type sont assez faibles, à la condition que vous preniez soin de vérifier qu'aucune spécificité de la classe descendante ou de la classe d'implémentation à remplacer n'est utilisée [Retaille 2005].

II.1.6. Déplacement d'éléments

Le déplacement d'éléments consiste à :

- Déplacer une classe ou une interface d'un package à un autre ;
- Déplacer une méthode d'une classe ou d'une interface à une autre ;
- Déplacer un attribut d'une classe ou d'une interface à une autre.

Les déplacements d'éléments restructurent la conception ou le code de manière à les rendre plus compréhensibles. Dans le cas d'une classe ou d'une interface déplacée d'un package à un autre, il s'agit souvent de regrouper au sein d'un même package des classes ou interfaces ayant une même nature ou une finalité commune. Dans le cas d'une méthode ou d'un attribut, il s'agit souvent de corriger un problème de conception. Le déplacement d'une classe ou d'une interface d'un package à l'autre est très peu risqué, car les classes ou les interfaces déplacées ne sont pas modifiées. Il faut toutefois prendre garde à d'éventuels chargements dynamiques de l'élément déplacé. Le déplacement d'une méthode ou d'un attribut est beaucoup plus risqué, car il s'agit d'une modification de la conception de la classe ou de l'interface initialement propriétaire [Retaille 2005].

II.1.7. Le pattern Façade

Les design patterns, ou modèles de conception réutilisables, apportent des solutions génériques à des problèmes récurrents rencontrés dans le développement de logiciels. Ces modèles sont indépendants des langages mais reposent pour la plupart sur l'approche orientée objet [Retaille 2005].

Au sein d'un logiciel, les différents traitements pour rendre les services attendus sont répartis au sein d'une multitude d'objets. Les traitements répondent à des problématiques très variées, pouvant être fonctionnelles ou techniques. Généralement, chaque problématique est adressée par un sous-système du logiciel. Un service pouvant impliquer plusieurs problématiques, les liens entre les différents sous-systèmes peuvent devenir très nombreux et difficilement gérables [Retaille 2005].

Le design pattern Façade (en Annexes) vise à simplifier l'accès à un ensemble d'objets formant un sous-système en fournissant à l'extérieur une interface unifiée, sous forme d'une ou de plusieurs classes, masquant la complexité liée à la manipulation de cet ensemble. Ce design pattern est particulièrement utile pour marquer clairement les limites de chaque sous-système. Typiquement, nous l'utilisons pour isoler les différentes couches techniques et fonctionnelles du logiciel. Pour que le sous-système soit correctement isolé, les dépendances doivent être unidirectionnelles, de l'extérieur vers la façade [Retaille 2005].
Le schéma UML de la figure 12 illustre l'utilisation de ce design pattern dans le cadre d'une refonte.

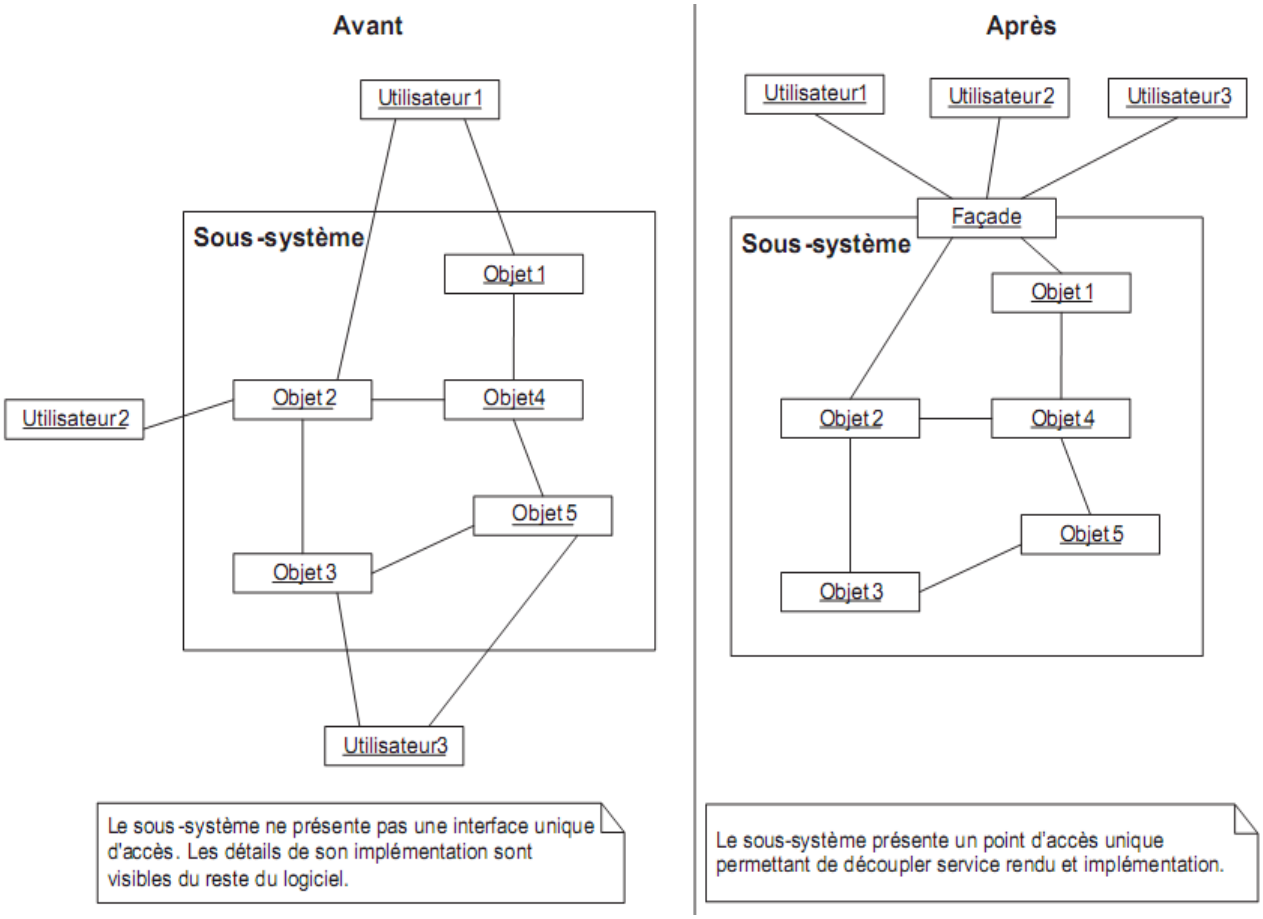


Figure 12 - Refonte avec le design pattern façade [Retaille 2005]

Gains attendus et risques à gérer

En améliorant la séparation des sous-systèmes entre eux, nous facilitons la compréhension de l'organisation du logiciel et la maintenance de chaque sous-système puisque la façade masque les détails de l'implémentation de ce dernier.

L'application de ce design pattern est cependant une opération lourde, car les relations entre les objets sont directement impactées. Dans le cadre d'un projet de refactoring, il est préférable d'utiliser ce design pattern de manière chirurgicale et progressive, sous-système par sous-système.

II.2. Présentation de la méthodologie de migration

Pour la migration de l'architecture MVC d'une IHM vers l'architecture MVC 2 :

1- Mettre sur pied un contrôleur unique qui déterminera à chaque requête reçue les traitements applicatifs à invoquer. Le contrôleur devra implémenter le design pattern ou modèle de conception réutilisable *Façade* comme le montre la *figure 13 – b ci-dessous* (*un exemple d'implémentation du pattern Façade en langage Java est en Annexes*):

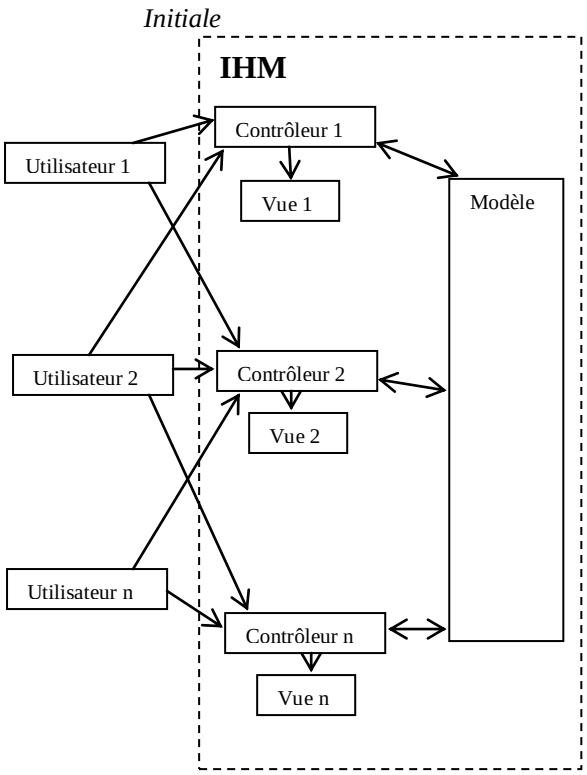


Figure 13 - a - IHM basée sur MVC

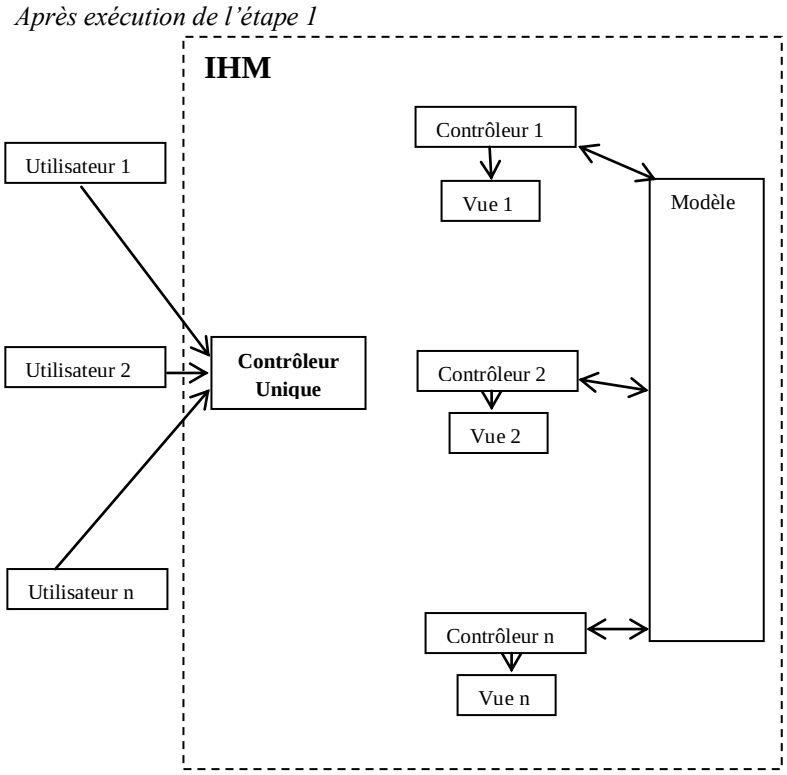


Figure 13 - b – Migration étape 1

2- Extraire les traitements effectués par les différents contrôleurs puis les encapsuler dans des classes à l'aide des techniques de refactoring (*voir Figure 13 - c ci-dessous*).

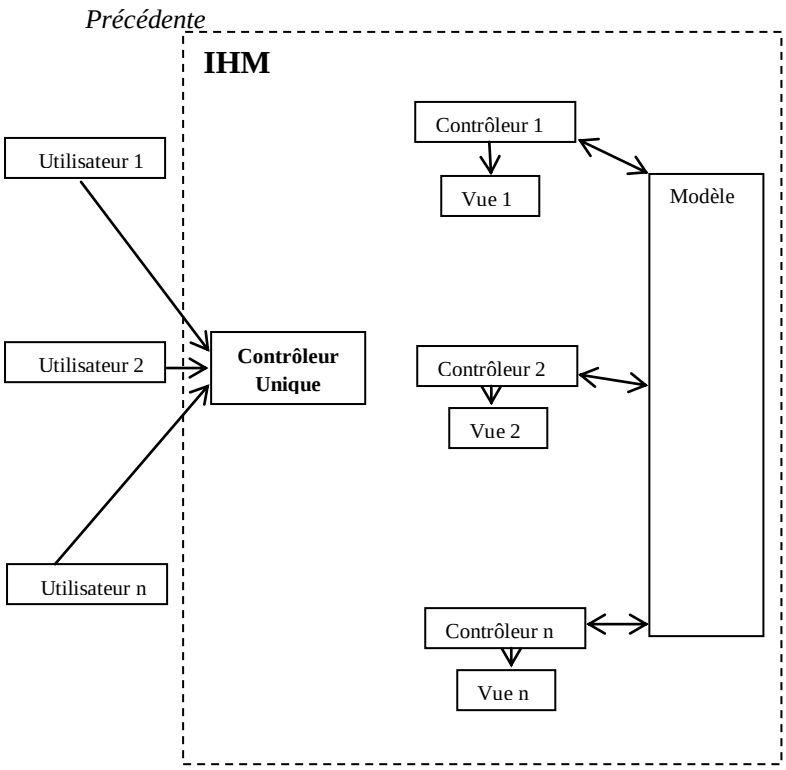


Figure 13 - b – Migration étape 1

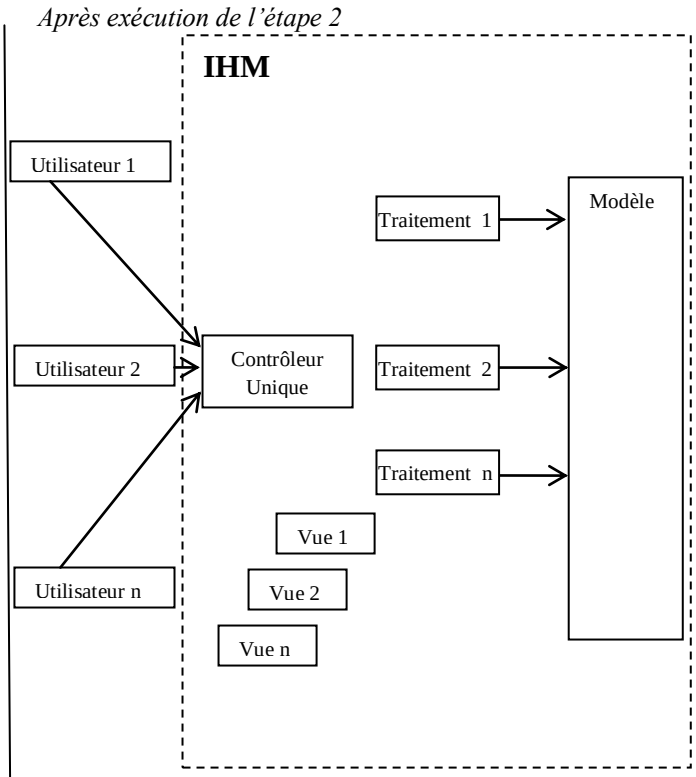


Figure 13 - c – Migration étape 2

3- Créer un fichier de configuration au format xml (*Config.xml*) qui contiendra les informations mettant en relation les différents composants (contrôleur unique > traitements, contrôleur unique > vues) (*voir Figure 13 - d ci-dessous*) C'est à partir de ce fichier que le contrôleur unique va pouvoir:
- Router les requêtes vers le traitement approprié (dispatch).
- Rediriger l'exécution vers différentes vues (forward).

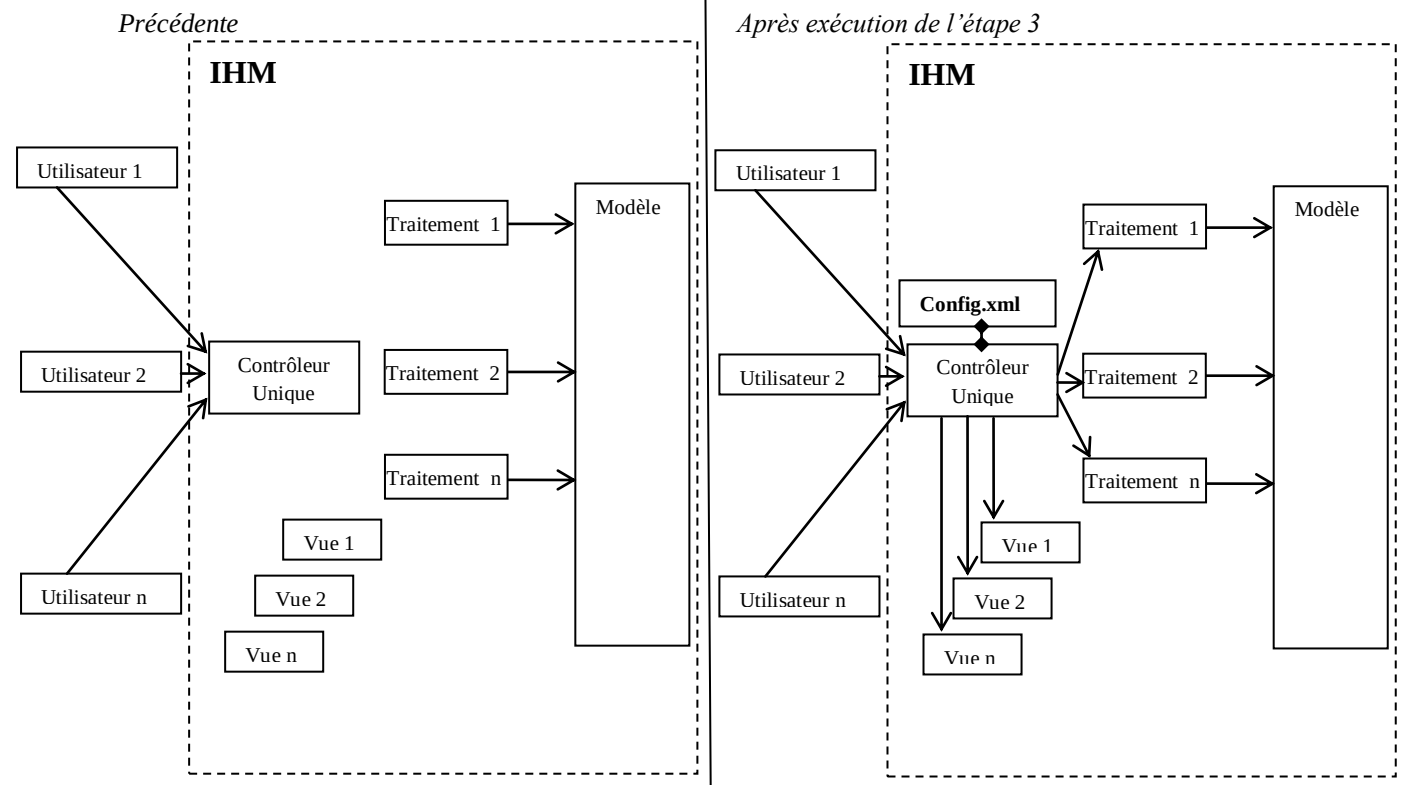


Figure 13 - c – Migration étape 2

Figure 13 - d – IHM basée sur MVC2

II.3. Illustration sur l'application web de l'ERP ADempiere

II.3.1. Généralités

Cette sous- section présente la plate-forme java EE en abordant successivement les composants de Java EE, le modèle d'architecture MVC appliqué à Java EE, le framework de développement d'application web Struts 2 [ApacheStruts] sur lequel sera basée l'application web de l'ERP ADempiere après migration.

a) Présentation de la plate-forme Java EE

J2EE est l'acronyme de Java 2 Entreprise Edition. Cette édition est dédiée à la réalisation d'applications pour entreprises.

J2EE est basé sur J2SE (Java 2 Standard Edition) qui contient les *API* (Application Programming Interface) de base de Java. Depuis sa version 5, J2EE est renommé Java EE (Java Enterprise Edition).

La plate-forme Java offre de très nombreuses possibilités de développement d'applications. La plate-forme Java EE est probablement la plus riche des plates-formes Java, en offrant un environnement standard de développement et d'exécution d'applications d'entreprise multi tiers. La complexité des architectures informatiques d'entreprise étant grandissante, les plates-formes de développement de systèmes informatiques ont dû prendre en compte cette complexité pour l'intégrer. La plate-forme JEE fait partie de celles qui ont le mieux réussi cette intégration, en offrant des possibilités d'interconnexion entre les systèmes et les applications, auparavant inexistantes [Étienne L. 2008].

Java EE est une plate-forme fortement orientée serveur pour le développement et l'exécution d'applications distribuées. Elle est composée de deux parties essentielles [Jean Michel D. 2011]:

- Un ensemble de spécifications pour une infrastructure dans laquelle s'exécutent les composants écrits en Java : un tel environnement se nomme serveur d'application (Exemple : Tomcat, JBoss,….).
- un ensemble d'API qui peut être obtenu et utilisé séparément. Pour être utilisées, certaines nécessitent une implémentation de la part d'un fournisseur tiers.

En tirant parti des avantages de Java, tels que l'indépendance de la plate-forme, ou bien son or ientation objet qui lui confère une très grande réutilisabilité, Java EE simplifie la conception de systèmes d'entreprise, en fournissant [Étienne L. 2008]:

- Une infrastructure d'exécution pour héberger les applications ;
- Des modèles de composants pour développer les applications, livrés sous forme de bibliothèques de programmation ;
- Une plate-forme de service intégrée par les infrastructures d'exécution, et utilisée par les composants.

Java EE permet une grande flexibilité dans le choix de l'architecture de l'application en combinant les différents composants. Ce choix dépend des besoins auxquels doit répondre l'application mais aussi des compétences dans les différentes API de Java EE. L'architecture d'une application se découpe idéalement en au moins trois tiers [Jean Michel D. 2011]:

- La partie cliente : c'est la partie qui permet le dialogue avec l'utilisateur. Elle peut être composée d'une application standalone, d'une application web ou d'applets.
- La partie métier : c'est la partie qui encapsule les traitements.
- La partie donnée : c'est la partie qui stocke les données.

b) Les composants Java EE

Le modèle de développement d'application Java EE préconisé par Sun Microsystems fait intervenir trois types de composants logiciels. L'objectif est de mieux séparer les traitements et donc les découpages, l'équipe de développement peut mieux structurer et modéliser son application avant de commencer le codage [Étienne L. 2008].

❖ Servlet

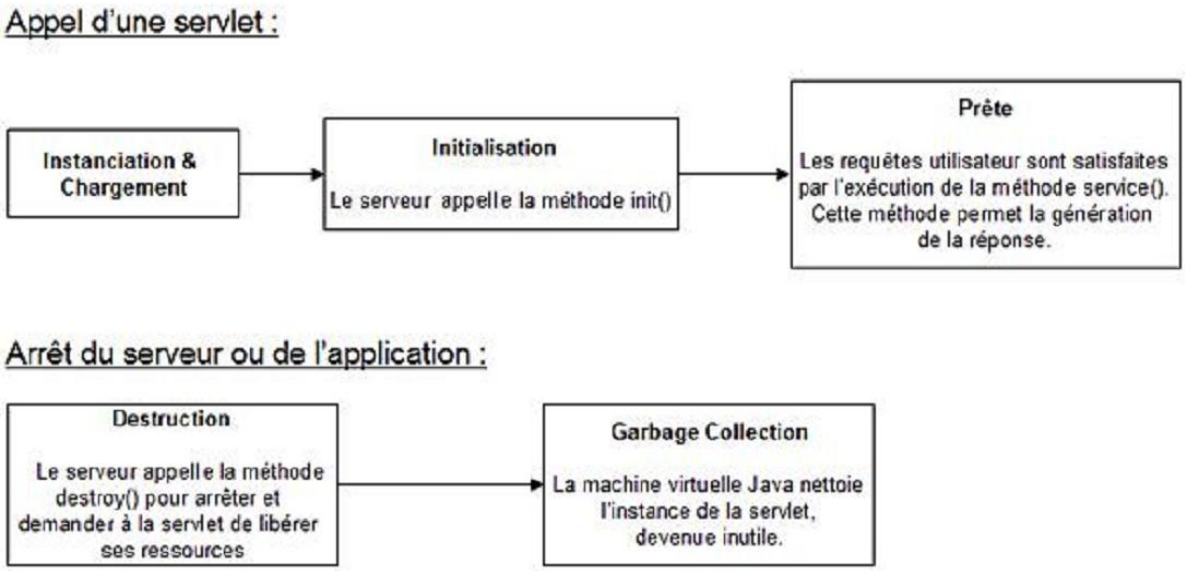
Les serveurs web sont de base uniquement capables de renvoyer des fichiers présents sur le serveur en réponse à une requête d'un client. Cependant, pour permettre l'envoi d'une page HTML contenant par exemple une liste d'articles répondant à différents critères, il faut créer dynamiquement ces pages HTML. Plusieurs solutions existent pour ces traitements. Les servlets java sont une de ces solutions. Mais les servlets peuvent aussi servir à d'autres usages [Jean Michel D. 2011]. Sun Microsystems fourni des informations sur les servlets sur son site [SunServlet].

Les servlets sont des composants logiciels entièrement écrits en Java, ce ne sont pas des p rogrammes autonomes, et elles doivent être hébergées dans un serveur d'application pour pouvoir être appelées. Les servlets sont des composants orientés requête/réponse, c'est-à-dire qu'une servlet s'exécute suite à une requête, en général http, et fournit une réponse à cette requête, ce fonctionnement est donc très proche de celui d'un serveur Web. Une servlet possède des caractéristiques intéressantes, comme elle est écrite en Java, elle est portable et évolutive, mais elle est également très performante car elle est chargée en mémoire lors de son premier appel, et en sera déchargée à l'arrêt de l'application ou du serveur d'applications. De plus, il n'y a toujours qu'une seule instance d'une servlet en mémoire, le serveur d'applications utilisant un thread pour traiter chaque requête émise par les clients. Un thread étant une unité d'exécution faisant partie d'un programme [Étienne L. 2008].

Cycle de vie d'une servlet

Une servlet étant avant toute chose une classe Java, cette classe doit être chargée puis interprétée par une machine virtuelle Java, en l'occurrence ici, celle du serveur

d'application. Ensuite, le serveur va initialiser la servlet pour lui faire charger des informations de configuration, par exemple, la servlet est maintenant prête à recevoir des requêtes et à renvoyer des réponses. Lorsque l'application ou le serveur s'arrête, la servlet est détruite, puis son instance est nettoyée par la machine virtuelle Java [Étienne L. 2008]



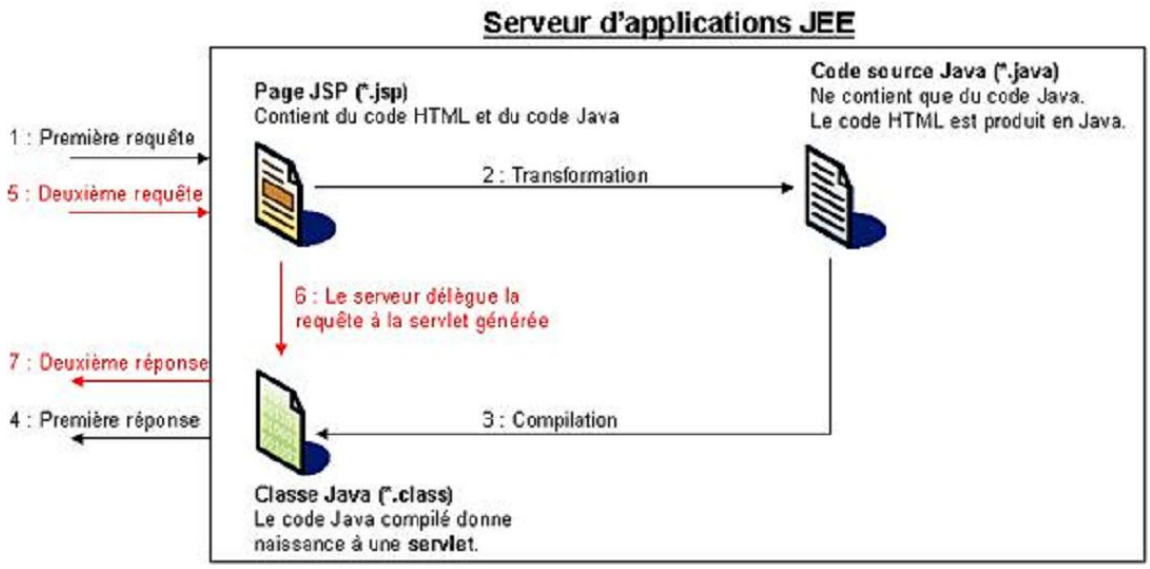
Le rôle d'une servlet dans une application JavaEE, est de recevoir les requêtes des clients, ainsi que d'en extraire les informations, ces informations une fois éventuellement formatées, pourront être utilisées pour appeler les traitements. La servlet est aussi responsable de la préparation des données nécessaires à la génération de la réponse, sans nécessairement fournir cette réponse au client [Étienne L. 2008].

- Java Server Pages : JSP

La technologie Java Server Pages permet le développement de pages Web dynamiques. D'un point de vue de sa structure, une page JSP est très proche d'une page PHP ou bien ASP, si ce n'est que le langage utilisé pour la partie dynamique n'est pas le même. Une page JSP est un fichier qui porte l'extension *.jsp*. Une page JSP est constituée d'un squelette de code HTML pour la partie statique, et de code Java pour permettre l'intégration de données obtenues en résultat des traitements. La structure d'une JSP est donc hétérogène, et elle devra être traitée par le serveur d'application avant d'être envoyée dans la réponse au client. Ce traitement est réalisé par le serveur d'applications au premier appel de la page et à chaque fois que cette page est

modifiée par un programmeur. Un serveur d'applications transforme une JSP en classe Java, puis la compile en servlet, c'est ensuite l'exécution de cette servlet particulière qui provoquera la génération de la réponse. Chaque autre requête vers cette JSP est en fait directement prise en charge par la servlet générée. [Étienne L. 2008]

Cycle de vie d'une JSP :



Le rôle d'une JSP dans une application Java EE est de prendre en charge la partie visuelle de cette application en présentant les données au client.

❖ Enterprise JavaBeans : EJB

Les composants Enterprise JavaBeans sont des composants métier distribués, c'est-à-dire qu'ils sont invocables par le réseau. Les EJB sont à très grande valeur ajoutée car ils prennent en charge les transactions, la sécurité, et sont naturellement amenés à être réparti sur des fermes de serveurs. Il existe deux types d'EJB [Étienne L. 2008] :

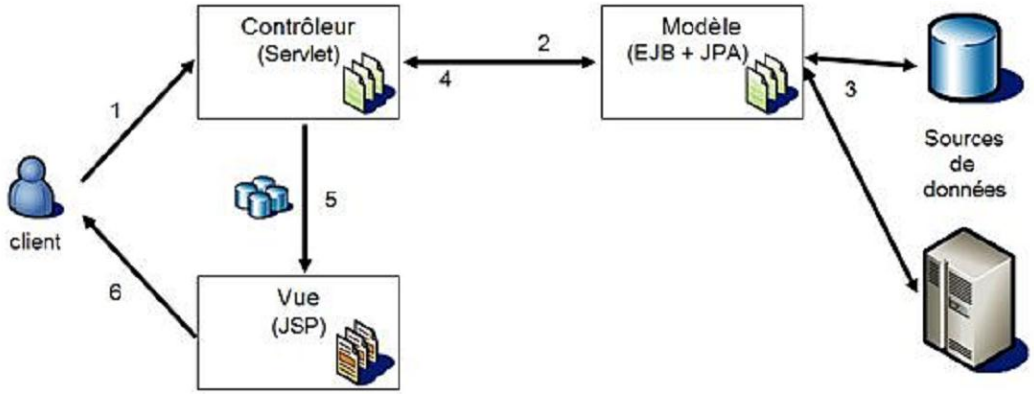
- *Les EJB session* **:** Ils encapsulent l'ensemble des fonctionnalités métier nécessaires à l'application, ils peuvent, selon leurs configurations, maintenir ou non des informations sur les clients et les traitements qu'ils réalisent (ils sont dit avec ou sans état).
- *Les EJB pilotés par message* : Appelés également MDB (Message Driven-Bean), ils sont comparables aux EJB session, mais sont invoqués différemment. Les EJB session sont directement exploités par des appels de fonctionnalités sur ces composants. Par contre, pour déclencher un traitement sur un EJB MDB, il faut au contraire, lui envoyer un message applicatif en utilisant l'API JMS (Java Message Service).

- Cette nouvelle version de la plate-forme JavaEE a vu disparaître les *EJB entités*. Un *EJB entité* est un composant persistant, c'est-à-dire que son état est sauvegardé et restauré dans une base de données. Ils matérialisent les données nécessaires à l'application, et sont en général appelés par les EJB session. Les EJB entités sont aujourd'hui remplacés par la nouvelle API de persistance Java : Java Persistence API (JPA).

Le rôle d'un EJB dans une application Java EE est donc variable en fonction de son type. Les différents types d'EJB sont en général utilisés conjointement dans une même application.

c) Le modèle d'architecture MVC appliqué à Java EE

L'architecture de développement MVC (Model View Controller) ou encore modèle d'architecture MVC, trouve ses origines avec le langage SmallTalk au début des années 1980, ce n'est donc pas un concept nouveau spécifiquement lié au développement Java EE. Son objectif principal est d'apporter une séparation de la logique métier et de la logique d'affichage, et pour ce faire, elle divise l'application en trois parties distinctes: le *modèle*, la *vue* et enfin le *contrôleur*. Initialement utilisé pour le développement des interfaces graphiques, appliqué aux technologies Java EE, ce modèle MVC peut se transposer sous la forme d'une architecture dite 3-tiers : le modèle est représenté par les composants EJB, le contrôleur par les servlets, et la vue par les JSP [Étienne L. 2008].



Collaboration dans le modèle MVC :

1. *Le client émet une requête HTTP à destination de l'application, c'est en général une servlet qui reçoit la requête et qui en extrait les informations.*
2. *Les informations sont utilisées pour appeler les traitements métier.*
3. *Les composants du modèle manipulent les données du système d'information (lecture, enregistrement, mise à jour,…).*

4. *Les traitements métier retournent les données résultats à la servlet, qui stocke ces don nées pour les rendre accessible aux JSP.*
5. *La servlet appelle la JSP adéquate.*
6. *La JSP s'exécute, inclut les données transmises par la servlet, et génère la réponse au client.*

L'utilisation du modèle MVC pour le développement des applications Java EE est une préconisation de Sun Microsystems, et l'utiliser apporte un certain nombre d'avantage. D'abord, les responsabilités étant clairement définies au sein de l'application, les composants sont plus nombreux, mais également plus simples, leurs spécificités font qu'ils peuvent être développés par des spécialistes, les servlets et EJB par des développeurs Java, les JSP par des webdesigners. Ainsi un changement de charte graphique, ou encore de logo par l'entreprise, n'aura un impact que sur la partie vue, le modèle et le contrôleur ne subiront en rien ces changements. Pour simplifier et systématiser l'utilisation de MVC dans les architectures Java EE, des Framework de développement entièrement basé sur ce modèle MVC, tel *Apache Struts* [ApacheStruts] ou encore la nouvelle API *Java Server Faces* [SunJSF], ont vu le jour [Étienne L. 2008].

d) Framework Struts 2

- *La définition d'un framework*

Avant d'aborder plus en détail le framework Struts 2, il convient de définir brièvement ce qu'est un framework. Un framework est un ensemble de bibliothèques, d'outils et de règles à suivre, qui aident au développement d'applications. Les frameworks sont développés par des programmeurs système. Un framework est composé de plusieurs briques/composants qui sont en interaction les uns avec les autres. Les applications peuvent être écrites de manière plus efficace si nous utilisons un Framework adapté au projet au lieu d'être obligé de réinventer à chaque fois la roue. Un framework Java fournit un ensemble de fonctionnalités à partir d'une implémentation objet. Lors de développement à grande échelle et de conception par équipe, les frameworks sont alors très utiles, voire indispensables. Actuellement, différents types de frameworks sont disponibles [Jérôme L. 2009] :

- Les frameworks d'infrastructure système, qui permettent de développer des systèmes d'exploitation, des outils graphiques et des plates-formes Web (Struts, Spring...).
- Les frameworks communicants (appelés intergiciels).

- Les frameworks d'entreprise (développements spécifiques).
- Les frameworks de gestion de contenu (type CMS).

L'objectif d'un framework est de faciliter la mise en œuvre des fonctionnalités de son domaine d'activité. Il doit permettre au développeur de se concentrer sur les tâches spécifiques à l'application à développer plutôt qu'à des tâches techniques récurrentes dans le développement d'applications tel que : l'accès aux données, l'internationalisation, la journalisation des événements (logging), la sécurité (authentification et gestion des rôles) [Jean Michel D. 2011] …

La mise en œuvre d'un framework permet notamment : de capitaliser le savoir-faire sans "*réinventer la roue*", d'accroître la productivité des développeurs une fois le framework pris en main, d'homogénéiser les développements des applications en assurant la réutilisation de composants fiables, donc de faciliter la maintenance notamment évolutive des applications. Cependant, cette mise en œuvre peut se heurter à certaines difficultés : le temps de prise en main du framework par les développeurs peut être plus ou long en fonction de différents facteurs (complexité du framework, richesse de sa documentation, expérience des développeurs, ...), les évolutions du Framework qu'il faut répercuter dans les applications existantes [Jean Michel D. 2011].

- *L'utilité de mettre en œuvre des frameworks*

Le développement d'applications d'entreprise avec Java (Java EE) s'avère relativement complexe. Il est nécessaire d'assimiler de nombreux concepts et API pour pouvoir développer et déployer une application Java EE. Par exemple pour le développement d'applications web, les API et les spécifications de Java EE ne proposent qu'un support de bas niveau au travers des API Servlets et JSP. Le développement d'une application de taille moyenne ou complexe va ainsi rencontrer d'énormes difficultés [Jean Michel D. 2011]. Pour faciliter le développement et ainsi augmenter la productivité, des frameworks ont été développés.

L'intérêt de la mise en œuvre d'un ou plusieurs frameworks a fait ses preuves depuis longtemps. Une des grandes difficultés est de sélectionner le ou les frameworks à utiliser. L'intérêt majeur des frameworks est de proposer une structure identique pour toutes les applications qui l'utilisent et de fournir des mécanismes plus ou moins sophistiqués pour assurer des tâches communes à toutes les applications. Il est possible de développer son propre framework mais cela représente un investissement long, risqué et donc coûteux

qu'il sera quasi impossible de rentabiliser d'autant que de nombreux frameworks sont présents sur le marché dont quelques frameworks open source particulièrement matures. Par exemple, le Framework Struts [ApacheStruts] du groupe Apache Jakarta est devenu un quasi standard adopté par la majorité des acteurs proposant un IDE (Environnements de Développement Intégrés) et sert de base au développement d'autres frameworks open source ou commerciaux [Jean Michel D. 2011].

Il existe de nombreux frameworks open source dont la possibilité de mise en œuvre concrète est très variable en fonction de plusieurs facteurs : fonctionnalités proposées, maturité du projet, évolutions constantes, documentation proposée, ... Plusieurs d'entre eux sont devenus de véritable succès grâce à leur adoption par de nombreux développeurs et à l'apport de nombreux contributeurs (exemple : *Struts* [ApacheStruts], Log4J [LOG4J] ou Spring [Spring]). Ils permettent d'avoir des frameworks relativement complets, fiables et fonctionnels. En plus, comme tout projet open source, les sources sont disponibles ce qui permet éventuellement de faire des modifications pour répondre à ces propres besoins ou ajouter des fonctionnalités [Jean Michel D. 2011].

- *Les frameworks pour les applications web*

Les applications web sont généralement le point d'entrée principal pour les applications développées en utilisant la plate-forme Java EE. Pourtant le développement d'applications web est relativement compliqué à concevoir et à implémenter pour plusieurs raisons [Jean Michel D. 2011]:

- La nature du protocole http (basé sur un modèle simple de request/response sans état).
- Les nombreuses technologies à mettre en œuvre (HTML, XHTML, CSS, JavaScript, ...) ainsi que leurs différentes versions.
- Le support de ces technologies par les différents navigateurs est particulièrement différent.
- HTML est un langage pauvre qui ne permet que le développement de formulaires assez rudimentaires.

A ces inconvénients indépendants de la technologie côté serveur pour les mettre en œuvre, viennent s'ajouter des raisons spécifiques à la plate-forme Java EE [Jean Michel D. 2011]:

- Les API Servlet et JSP sont de bas niveau.
- Le manque de modèles événementiels (cet argument explique le développement de la technologie JSF [SunJSF]).

Dès que l'on développe des applications web uniquement avec les API servlet et JSP comme c'est le *cas avec l'application web de l'ERP ADempiere*, il apparait évident que de nombreuses parties dans des applications différentes sont communes mais doivent être réécrites ou réutilisées à chaque fois. Ceci est en grande partie liée au fait que les API servlets et JSP sont des API de bas niveau. Elles ne proposent par exemple rien pour automatiser l'extraction des données de la requête HTTP et mapper leur contenu dans un objet de façon fiable, assurer les transitions entre les pages selon les circonstances, ... [Jean Michel D. 2011].

L'utilisation d'un framework web permet de faciliter le développement et la maintenance évolutive d'une application web. Les frameworks utilisent ou peuvent être complétés par des moteurs de templates qui facilitent la génération de page web à partir de modèles. Les frameworks les plus récents (tel que Java Server Faces [SunJSF]) mettent en œuvre l'utilisation de composants côté serveur pour faciliter les développements (modèle événementiel et développement graphique). Les Frameworks de développement web permettent généralement une séparation logique d'une application selon le concept proposé par le modèle MVC (Modèle/Vue/Contrôleur). Le framework le plus utilisé dans cette catégorie est Struts [ApacheStruts] [Jean Michel D. 2011].

- *Struts 2*

Struts 2 est un framework libre servant au développement d'applications web J2EE et qui met en application le *modèle MVC 2*. Il utilise et étend l'API Servlet Java afin d'encourager les développeurs à adopter l'architecture Modèle-Vue-Contrôleur. Apache Struts a été créé par Craig McClanahan et donné à la fondation Apache en mai 2000. Struts a fait partie du projet Jakarta de mai 2000 jusqu'en mars 2004.

Cette infrastructure permet la conception et l'implémentation d'applications Web de taille importante par différents groupes de personnes. En d'autres termes, les designers, développeurs de composants logiciels peuvent gérer leur propre part du projet de manière découplée.

Struts 2 permet la structuration d'une application Java sous forme d'un ensemble d'actions représentant des événements déclenchés par les utilisateurs de l'application. Ces actions sont décrites dans un fichier de configuration de type XML (*struts.xml*) décrivant les cheminements possibles entre les différentes actions. En plus de cela, Struts permet d'automatiser la gestion de certains aspects comme par exemple la validation des données

entrées par les utilisateurs via l'interface de l'application. Plus besoin de venir coder le contrôle de chaque donnée fournie par un utilisateur, il suffit de décrire les vérifications à effectuer dans un fichier XML dédié à cette tâche.

En utilisant Struts 2, le développeur simplifie son travail au niveau des vues et des contrôleurs du modèle MVC 2. Mais il serait inadapté d'utiliser ce framework dans des projets de petite taille car il introduit une certaine complexité. Struts 2 montre toute sa puissance dans des applications d'une certaine envergure.

Struts 2 est un *Framework orienté actions*. Les actions sont décomposées en trois rôles :

- Premièrement, les actions jouent le rôle le plus important du framework en encapsulant le traitement et le travail à réaliser par le service.
- Deuxièmement, les actions permettent de manipuler automatiquement les données des requêtes lors des transferts.
- Troisièmement, le framework détermine quel résultat doit être retourné et la vue à afficher en réponse à un traitement.

Les actions Struts 2 implémentent des objets JavaBeans (classes Java simples) pour chaque groupe de données envoyées dans la requête. Chaque paramètre de la requête est déclaré dans la classe d'action avec un nom identique pour réaliser automatiquement le mapping des valeurs. La finalité d'une action étant de retourner une chaîne de caractères permettant de sélectionner le résultat à afficher [Jérôme L. 2009].

Struts2 s'appuie sur une architecture modulaire dont les principaux composants sont sur la figure 14 suivante:

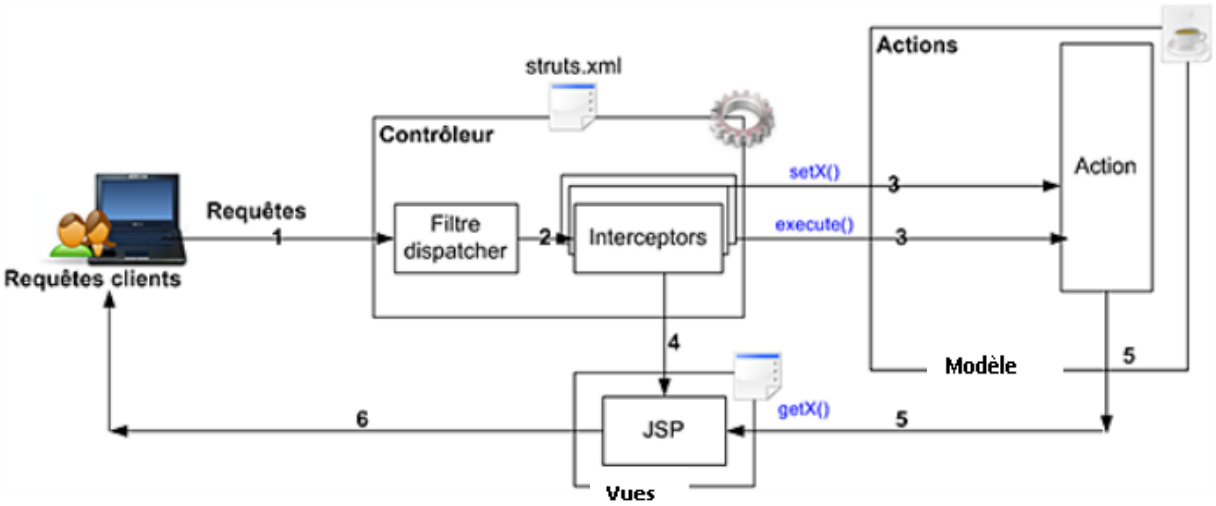


Figure 14 - Architecture de Struts2 [Jérôme L. 2009]

Sur la figure 14 :

1. *Le client envoie des requêtes vers un service de l'application avec des paramètres éventuels*
2. *Le filtre* « DispatcherFilter » *(org.apache.struts2.dispatcher.FilterDispatcher) examine la requête et détermine l'action associée à la requête à partie de la configuration effectuée dans le fichier de configuration struts.xml.*
3. *Si des intercepteurs (interceptors) ont été configurés pour l'action courante alors il y'a exécution de ceux-ci. Les interceptors peuvent traiter des opérations telles que les autorisations sur l'action (exemple : mise en place d'un mécanisme de sécurité consistant à n'autoriser que certains rôles), etc.*
   L'intercepteur params *assigne les valeurs présentes dans la requête à la classe d'action associée par l'intermédiaire de* ses accesseurs *et* exécute la méthode de traitement *(execute() par défaut). . L'action peut être emmenée à invoquer des services de la partie serveur.*
4. *La vue à afficher est sélectionnée en accord avec le fichier de configuration struts.xml.*
5. *La classe d'action transmet les données nécessaires à la vue.*
6. *La vue affiche au client les résultats traités.*

➢ *Avantages du framework Struts 2*

Le choix de Struts 2 comme framework de développement comporte de nombreux avantages liés à la modularité de l'architecture MVC 2 [Veronique L. 2008]:

- La division du travail est facilitée. Si tout le code d'une application était concentré dans un seul fichier, il serait impossible de diviser le travail. Avec Struts 2, le code de l'application est subdivisé en plusieurs actions. Les développeurs peuvent donc travailler en même temps sur des actions différentes puis intégrer le résultat de leur travail sans problème.
- La présence d'un seul et unique servlet contrôleur rend la compréhension et la maintenance de l'application beaucoup plus simple. Quand la logique de l'application est intégrée dans les pages JSP, il faut consulter plusieurs pages JSP pour avoir seulement une idée du fonctionnement de l'application. Au contraire, avec Struts 2, la logique de l'application est centralisée en un seul endroit.

- L'utilisation de fichiers de configuration XML rend les modifications du contrôleur beaucoup plus faciles. Le fichier de configuration évite de devoir modifier directement le code du servlet contrôleur.
- Le framework Struts 2 est open source. Il est donc possible de consulter et même de modifier le code source du framework pour l'adapter à des besoins spécifiques.
- Le framework Struts 2 bénéficie du support d'une communauté très dynamique et d'une visibilité de plus en plus grande dans l'industrie. La communauté contribue à améliorer continuellement la qualité du framework et peut offrir du support en cas de problèmes.
- Struts 2 permet aux développeurs de profiter des avantages d'une architecture MVC 2 sans avoir à la développer eux-mêmes. Les spécialistes s'entendent pour dire que le modèle MVC 2 est une architecture très performante pour la réalisation d'applications web. Cependant, développer les composants d'une architecture MVC 2 à partir de zéro est une tâche complexe et coûteuse. En utilisant Struts, qui est disponible gratuitement, on contribue à réduire le temps et les coûts de développement.
- L'utilisation d'un framework garantit une meilleure stabilité à l'application. Le framework Struts 2 est testé et utilisé par de nombreux développeurs à travers le monde. Le code de Struts 2 a, par le fait même, des chances d'être plus robuste que n'importe quel code développé localement.

II.3.2. Etude de l'existant

L'analyse de l'application de l'ERP ADempiere nous a permis de constater que, la plate-forme Java EE a été utilisée pour développer et exécuter l'application web de l'ERP ADempiere. L'application web de l'ERP ADempiere est composée d'un ensemble des composants Java EE (Servlets, JSP). Par exemple nous avons les servlets *WLogin* (ADempiere Login), *WWindow* (ADempiere window), *WMenu* (ADempiere Menu), *WReport* (ADempiere Report), *WWorflow* (ADempiere Workflow), *WHelp* (ADempiere Help) …

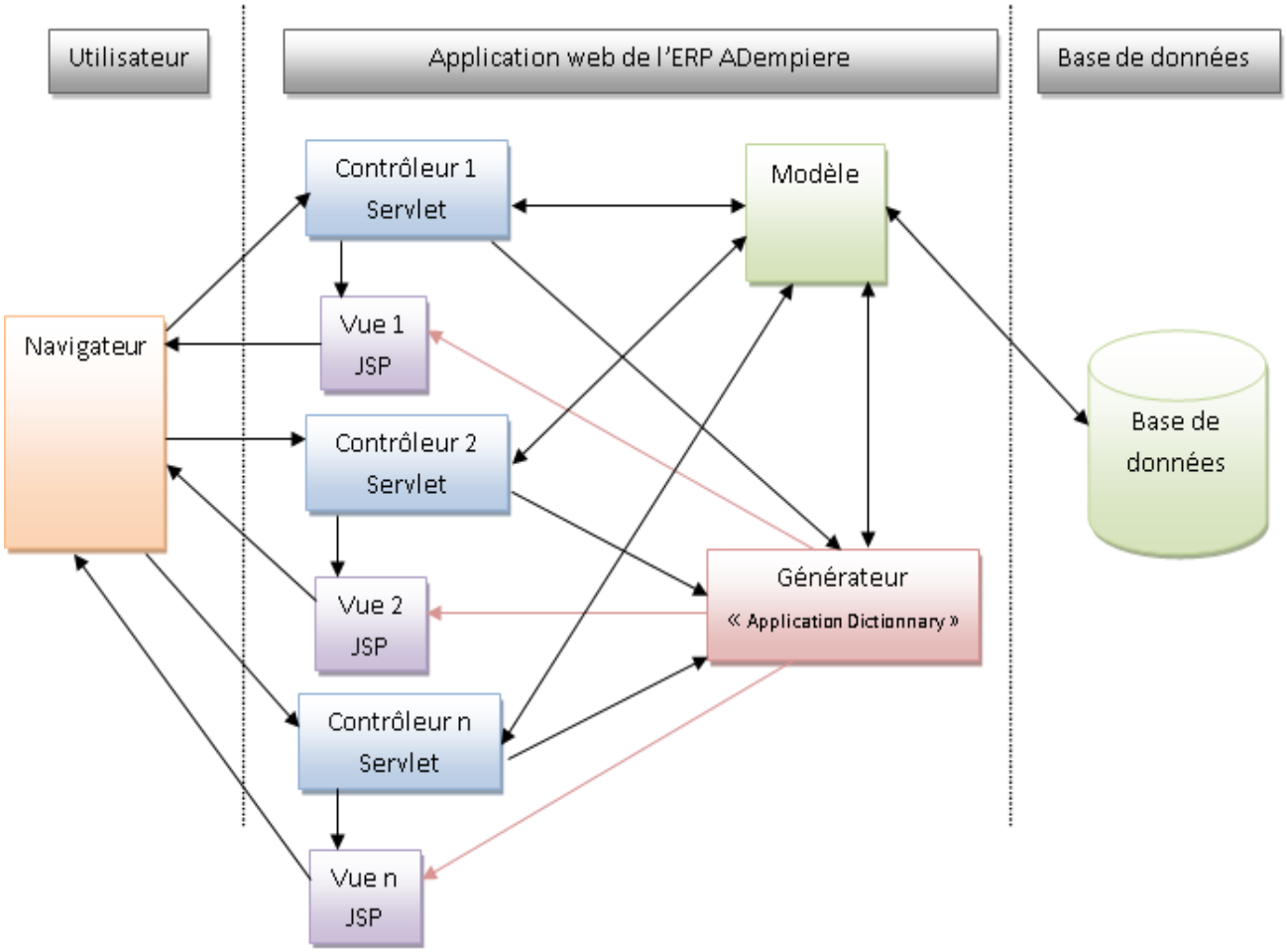


Figure 15 - Architecture de l'application web de l'ERP ADempiere basée sur MVC

En fonction de la requête effectuée par l'utilisateur à travers le navigateur (voir figure) :

La servlet traite la requête en faisant appel aux éléments du *modèle* si nécessaire puis passe la main au générateur « *Application Dictionnary*» [Smile 2008] d'IHM de l'ERP ADempiere ; *Le générateur* génère *la JSP* en faisant appel aux éléments du *modèle* si nécessaire; *La servlet* redirige ensuite la requête vers *la JSP* générée qui se charge de créer la réponse à l'utilisateur.

L'application web l'ERP ADempiere est donc basée sur l'architecture MVC appliquée à la plate-forme Java EE ayant pour inconvénient une multiplication du nombre de servlets (contrôleurs) nécessaire à l'application web : l'implémentation de plusieurs servlets (contrôleurs) nécessite beaucoup de code à produire ; rendant l'application web pas évidente pour une prise en main aisée par un développeur. De plus dans l'application web de l'ERP ADempiere, le code java de ses servlets est mélangé avec les scripts HTML\JSP, SQL. Ce mélange de scripts HTML\JSP, SQL et Java rend le débogage de l'application web de l'ERP ADempiere plus complexe et plus long à réaliser, empêche la réutilisation et visibilité des

structures de contrôle. Affichage et accès aux données étant confondus. Ainsi dans l'application web de l'ERP ADempiere nous avons du mal à distinguer clairement la couche présentation, la couche métier et les données : elles restent trop fortement liées entre elles ; rendant l'application web difficilement maintenable et évolutive.

Nous nous proposons donc de migrer l'architecture MVC de l'application web de l'ERP ADempiere vers l'architecture MVC 2; en le basant sur le framework Struts 2.

II.3.3. Application de la méthodologie de migration sur l'ERP ADempiere

Pour migrer l'architecture MVC de l'application web de l'ERP ADempiere vers l'architecture MVC 2 en la basant sur le framework Struts 2, nous appliquons notre méthodologie précédente :

1. *Mettre sur pied un contrôleur unique qui déterminera à chaque requête reçue les traitements applicatifs à invoquer. Le contrôleur devra implémenter le design pattern ou modèle de conception réutilisable Façade.*

Dans cette étape nous aurons plus besoin de mettre sur pied un contrôleur car comme nous l'avons dit, le Framework Struts 2 étant basé sur l'architecture MVC 2 possèdent déjà en son sein un contrôleur unique qui prend en charge toutes les requêtes du client, les analyses et appelle la classe d'action dédiée contenant les traitements, puis en fonction du résultat de l'exécution (*SUCCESS* ou *ERROR*) redirige la requête vers la page vue. Ceci nous permet ainsi de concentrer nos efforts à la réalisation des différentes actions de l'application web (étape suivant).

2. *Extraire les traitements effectués par les différents contrôleurs puis les encapsuler dans des classes à l'aide des techniques de refactoring.*

Dans cette étape, nous avons extrait à l'aide des techniques de refactoring, les traitements effectués par les différents servlets (contrôleurs) de l'ERP ADempiere et les encapsuler dans les classes d'action pour le framework Struts 2.

<u>Par exemple :</u>

Pour la servlet *WWindow* (ADempiere window) de l'application web l'ERP ADempiere responsable du traitement de tout évènement fonctionnel de l'application web pouvant se produire sur une fenêtre de l'ERP ADempiere tels que le clique sur les boutons : *Enregistrer les données d'un formulaire, Créer un nouveaux formulaire d'enregistrement, Supprimer un élément dans un tableau, Parcourir un tableau (Suivant, Précédant, Début, Fin),…*

Nous avons extrait ses traitements (blocs de codes et méthodes de la classe *WWindow*) et encapsulé ses traitements dans la Classe d'action *WindowAction* (voir figure 17) à l'aide des techniques du refactoring tels que le changement de la signature d'une méthode, l'extraction d'une méthode…

```
public class WindowAction extends KoosseryAdempiereBaseAction{
        //action nouveau enregistrement
        public String nouveau(…){
       …… ;
        }
        //action enregistrer
        public String enregistrer(…){
      ……. ;
        }
        //action supprimer
        public String supprimer(…){ ……. ;
        }
        //action actualiser
        public String actualiser(…){ ……. ;
        }
        //action début
        public String premier(…){ ……. ;
        }
        //action suivant
        public String suivant(…){ ……. ;
        }
        //action précédant
        public String precedant(…){ ……. ;
        }
        //action fin
        public String dernier(…){ ……. ;
        }
……————————————————————————

}
```

Figure 16 - Extrait de la structure de la classe d'action WindowAction

Nous avons ainsi fait de même pour tous les autres servlets de l'application web de l'ERP ADempiere tels que : *WLogin* (ADempiere Login), *WReport* (ADempiere Report), *WWorkflow* (ADempiere Workflow), *WHelp* (ADempiere Help) … En créant respectivement les classes d'action *LoginAction, ReportAction, WorkflowAction, HelpAction* …

En plus de ceci les traitements SQL qui se trouvaient dans ces servlets de l'application de l'ERP ADempiere ont été migrés dans la partie serveur de l'ERP KoosseryADempiere (KoosseryADempiere.server figure 16), exposés sur forme de service puis consommés dans les méthodes (traitements) des classes d'action. Permettant ainsi une séparation claire entre la couche métier et la couche donnée.

3. *Créer un fichier de configuration au format xml qui contiendra les informations mettant en relation les différents composants (contrôleur unique > traitements, contrôleur unique > vues). C'est à partir de ce fichier que le contrôleur unique va pouvoir:*
   - Router les requêtes vers le traitement (action) approprié (dispatch).
   - Rediriger l'exécution vers différentes vues (forward).

Dans cette étape nous avons créé le fichier de configuration *struts.xml* dans lequel il a été renseigné toutes les relations entre les différents composants. Dans ce fichier de configuration struts.xml**,** l'action (traitement) est déclarée par le mot clé *action*, suivi du nom de la classe qui l'interprète (*la classe action*), puis du *nom de la méthode* de la classe d'action, et enfin *des JSP* où seront redirigés les résultats.

Par exemple :

Pour renseigner au contrôleur unique la classe d'action *WindowAction* précédente et ses méthodes à invoquer pour le traitement de la requête du client, nous l'avons fait comme ceci (figure 17 sur recommandation du framework Struts 2):

```
<package name="window" extends="default" namespace="/window">

 <action name="nouveau" method="nouveau" class="com.koossery.adempiere.web.actions.window.WindowAction">
  <result name="error">/jsp/window/windowPageError.jsp</result>
  <result name="success">/jsp/window/windowPageNouveau.jsp</result>
 </action>
<action name="supprimer" method="supprimer" class="com.koossery.adempiere.web.actions.window.WindowAction">
  <result name="error">/jsp/window/windowPageError.jsp</result>
  <result name="success">/jsp/window/windowPageSupprimer.jsp</result>
 </action>
 <action name="actualiser" method="actualiser" class="com.koossery.adempiere.web.actions.window.WindowAction">
  <result name="error">/jsp/window/windowPageError.jsp</result>
  <result name="success">/jsp/window/windowPageActualiser.jsp</result>
 </action>

 <action name="enregistrer" method="enregistrer" class="com.koossery.adempiere.web.actions.window.WindowAction">
  <result name="error">/jsp/window/windowPageError.jsp</result>
  <result name="success">/jsp/window/windowPageEnregistrer.jsp</result>
 </action>
  ..............
</package>
```

Figure 17 - Extrait du fichier struts.xml

Ainsi lorsque le client provoque par exemple l'évènement « *Enregistrer*» dans un formulaire de l'application web (qui correspond à la requête *http://localhost:8080/KoosseryADempiere.we/window/enregistrer.action*), grâce au contenu du fichier de configuration *struts.xml*, le contrôleur unique évoque *l'action enregistrer*. Cette action sera traitée par la méthode « *enregistrer()* » de la classe *WindowAction* et le résultat sera redirigé par le contrôleur unique vers la vue *(/jsp/window/windowPageEnregistrer.jsp)* en

cas de *succès*, sinon sera redirigée vers la vue *(/jsp/window/windowPageError.jsp)* en cas *d'échec* afin de renseigner correctement le formulaire.

Enfin nous avons ainsi fait de même pour toutes les autres classes d'action *LoginAction, ReportAction, WorkflowAction, HelpAction …*

Au final nous aurons une application web pour l'ERP KoosseryADempiere basée sur l'architecture MVC 2 comme le montre la figure 18 suivante :

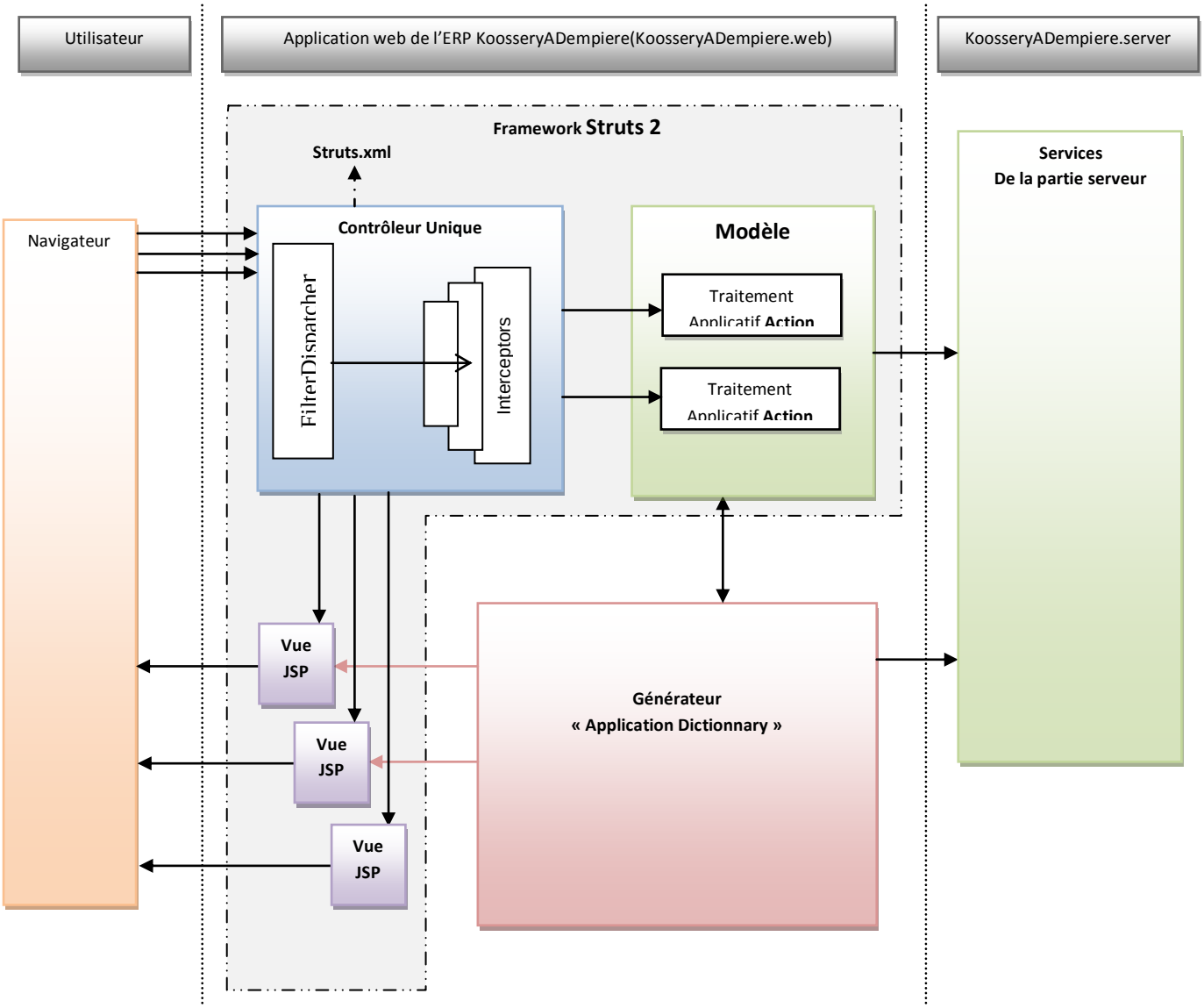


Figure 18 - Architecture de l'application web de l'ERP KoosseryADempiere basée sur MVC 2

CHAPITRE III : REALISATION DE L'APPLICATION WEB DE L'ERP KoosseryADempiere

Ce chapitre est consacré à la mise en œuvre de la migration de l'application web de l'ERP ADempiere en le basant sur le framework Struts 2. La nouvelle application web qui en résulte de cette migration est utilisée comme application web de l'ERP KoosseryADempiere. Il aborde successivement les points suivants, présentation de L'ERP KoosseryADempiere, environnement de développement et outils utilisés, architecture de l'application web de l'ERP KoosseryADempiere, l'arborescence du projet de KoosseryADempiere.web, Migration de l'application web de l'ERP ADempiere.

III.1. Présentation de L'ERP KoosseryADempiere

L'ERP KoosseryADempiere est un ERP avec CRM (Relation Client) intégré, particulièrement bien adapté aux PME et aux filiales des multinationales. Il fournit un ensemble de fonctionnalités entièrement intégrées et faciles d'utilisation grâce auxquelles l'entreprise pourra assurer sa gestion quotidienne de manière performante. Les différents modules fonctionnels (figure 19) de L'ERP KoosseryADempiere sont les suivants :

Comptabilité, immobilisations, Paie, RH, Achats, Ventes, POS (Point Of Sales), Gestions de la relation client, Gestion de la production, Gestion des projets, Gestion financière.

Figure 19 - Fonctionnalité de L'ERP KoosseryADempiere

III.2. Environnement de développement et outils utilisés

Pour nos développements nous avons utilisé les outils suivants :

- *Eclipse Helios Java EE* : Environnement de développement intégré pour application J2EE. Cet environnement de développement Open Source accessible à tous offre un support du refactoring. Dans sa version 3.0, Eclipse supporte l'ensemble des techniques de refactoring que nous avons abordé dans ce chapitre 2.

  L'ensemble des assistants de refactoring est accessible à partir du menu Refactor d'Eclipse, comme illustré à la figure 20.

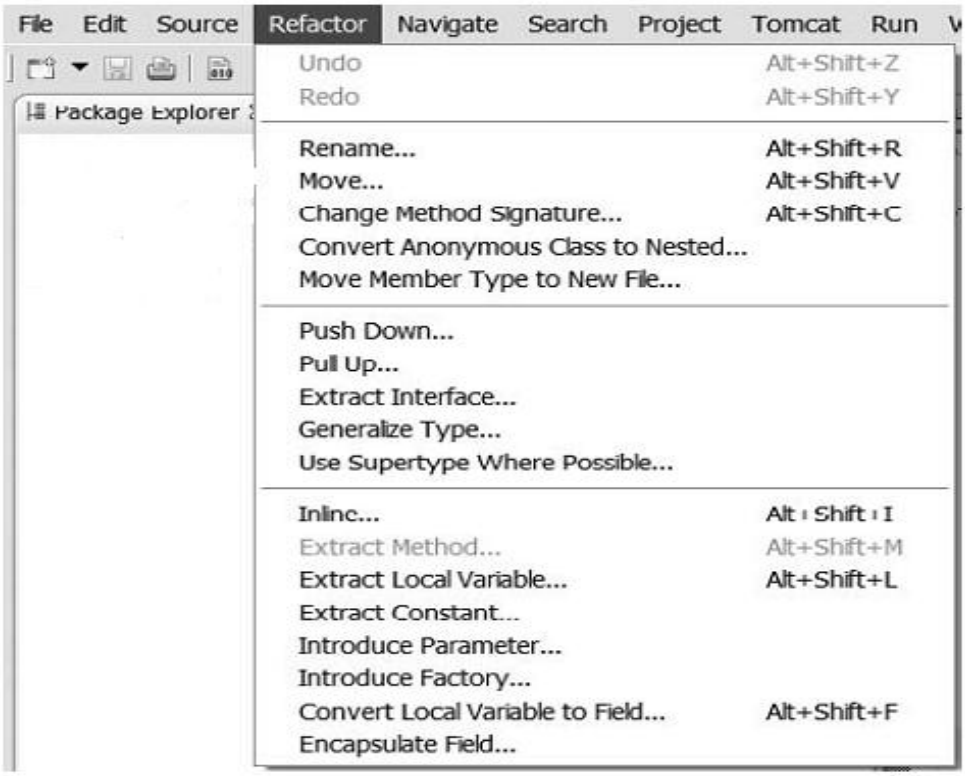


Figure 20 - Le menu Refactor d'Eclipse

- *Maven 2* : Outil Open source de la fondation Apache permettant une gestion des projets Java en général et J2EE en particulier. Il s'intègre facilement à Eclipse et permet notamment de récupérer automatiquement les dépendances des projets qui sont dans le référentiel.

- *Ant :* Outil Open Source de la fondation Apache permettant de compiler les projets sous Eclipse. Il s'intègre facilement à Maven2.

- *TortoiseSVN* : Outil de gestion des versions de projet.

III.3. Architecture de l'application web de l'ERP KoosseryADempiere

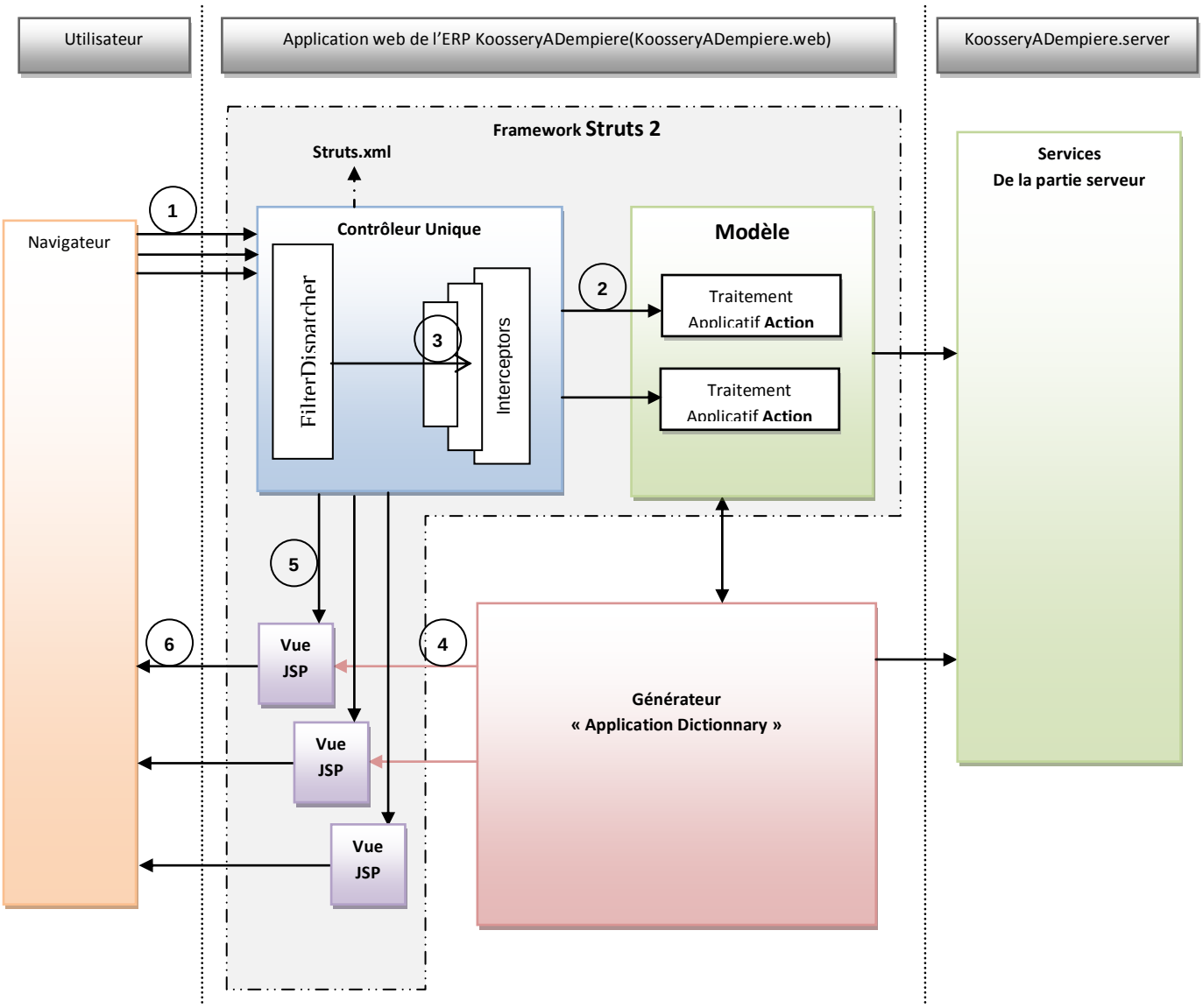


Figure 21 - Architecture de KoosseryADempiere.web

Sur la figure 21 :

1. *L'utilisateur envoie des requêtes à travers le navigateur vers un service de l'application web avec des paramètres éventuels.*
2. *Le filtre* « DispatcherFilter » *(org.apache.struts2.dispatcher.FilterDispatcher) examine la requête et détermine l'action associée à la requête à partie de la configuration effectuée dans le fichier de configuration* struts.xml.
3. *Si des intercepteurs* « interceptors » *ont été configurés pour l'action courante alors il y'a exécution de ceux-ci. Les interceptors peuvent traiter des opérations telles que les autorisations sur l'action (exemple : mise en place d'un mécanisme de sécurité*

consistant à n'autoriser que certains rôles), etc.
L'intercepteur params *assigne les valeurs présentes dans la requête à la classe d'action associée par l'intermédiaire de* ses accesseurs *et* exécute la méthode de traitement *(execute() par défaut). L'action peut être emmenée à invoquer des services de la partie serveur(KoosseryADempiere.server).*

4. *Le générateur «* Application Dictionnary*» d'IHM de l'ERP* ADempiere *génère la vue en faisant appel aux éléments du modèle si nécessaire. Il peut être emmenée à invoquer des services de la partie serveur*
5. *La vue à afficher est sélectionnée en accord avec le fichier de configuration* struts.xml *par le contrôleur.*
6. *La vue affiche à l'utilisateur les résultats traités.*

III.4. Arborescence du projet *KoosseryADempiere.web*

L'application de l'ERP KoosseryADempiere est un projet maven2 multi module constitué d'une couche de services (KoosseryADempiere.server) et d'une couche présentation(koosseryADempiere.web). L'application web de l'ERP KoosseryADempiere est organisée comme suit dans l'environnement de développement Eclipse:

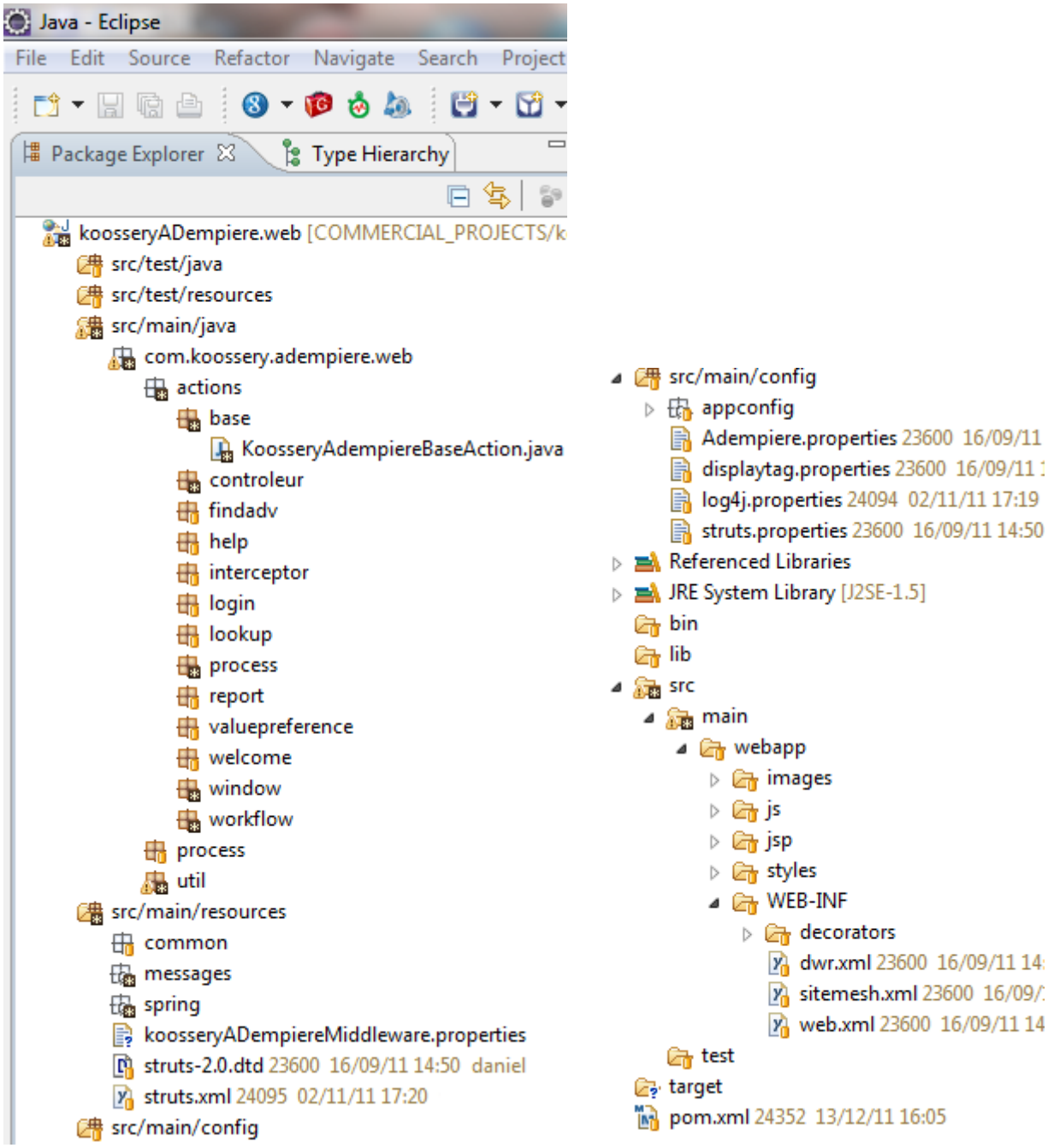


Figure 22 - Arborescence du projet koosseryADempiere.web

Sur la figure 22 :

- *src/main/java* : contient toutes les sources java du projet (

  Les classes d'action : package *com.koossery.adempiere.web.action*,

  Les interceptors : package *com.koossery.adempiere.web.action.interceptor*,

  Les classes utils : package *com.koossery.adempiere.web.util*).

- *src/main/ressources* : contient les configurations figées c'est-à-dire celles qu'on n'a pas besoin d'ajuster pour faire fonctionner l'application web dans un environnement donné.
- *src/main/config* : contient les configurations qui ont besoin d'être ajustées selon l'environnement cible afin de faire fonctionner l'application web. Ces configurations sont constituées essentiellement des valeurs des placeholder présents dans les configurations figées (src/main/ressources).
- *src/main/webapp* : contient toutes les ressources web du projet tel les pages jsp, les scripts, les images, les rapports.
- *src/test/java* : Les tests unitaires.
- *src/test/ressources* : Les ressources nécessaires aux tests unitaires, qui ne seront pas déployées.
- *Referenced Library* : contient les dépendances du projet configurées dans le fichier pom.xml (Maven 2) tels que : les bibliothèques (.jar) du framework struts 2, de KoosseryADempiere.server (pour l'invocation des services), d'ADempiere, Log4j, …

III.5. Migration de l'application web de l'ERP ADempiere

III.5.1. Configuration du projet KoosseryADempiere.web

a) Le fichier pom.xml Maven 2 du projet

Il contient une description détaillée du projet, avec en particulier des informations concernant le visionnage et la gestion des configurations, les dépendances, les ressources de l'application, les tests, les membres de l'équipe, la structure et bien plus encore. Il est placé dans le répertoire de base du projet.

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
        <modelVersion>4.0.0</modelVersion>
        <groupId>com.koosseryADempiere.app</groupId>
       <parent>
          <groupId>com.koosseryADempiere.app</groupId>
          <artifactId>3.5.2-KoosseryADempiere-2.0.0</artifactId>
          <version>${appversion}</version>
       </parent>
        <artifactId>koosseryADempiere.web</artifactId>
        <packaging>war</packaging>
        <version>${appversion}</version>
        <name>koosseryADempiere.web</name>
        <url>http://www.myComp.com</url>
        <description>Struts2 webapplication</description>

      <dependencies>
            <!-- Struts 2 -->
          <dependency>
              <groupId>org.apache.struts</groupId>
              <artifactId>struts2-core</artifactId>
         </dependency>
           <!-- koosseryADempiere.server -->
         <dependency>
             <groupId>com.koosseryADempiere.server</groupId>
             <artifactId>koosseryADempiere.server</artifactId>
             <version>${appversion}</version>
         </dependency>
          <! -- ADempiere -->
         <dependency>
              <groupId>adempiere</groupId>
             <artifactId>Adempiere</artifactId>
         </dependency>
          <!—LOG -->
          <dependency>
              <groupId>log4j</groupId>
              <artifactId>log4j</artifactId>
          </dependency>
           ...................................
     </dependencies>
```

Figure 23 - Extrait du fichier pom.xml

b) Description du fichier de déploiement web.xml du projet

C'est le fichier central du projet. Il contient les paramètres et caractéristiques de l'application web. Il est présent dans le dossier *main/webapp/WEB-INF* (main étant le *répertoire root* de l'application). Voici ses principales caractéristiques :

```
<web-app id="starter" version="2.4" >
        <display-name>koosseryADempiere.web</display-name>
        <context-root>/koosseryadempiere</context-root>
        <context-param>
                <param-name>contextConfigLocation</param-name>
                <param-value>classpath*:spring/applicationContext*.xml</param-value>
        </context-param>
 <!-- Filters struts 2-->
  <filter>
    <filter-name>action2-cleanup</filter-name>
    <filter-class>org.apache.struts2.dispatcher.ActionContextCleanUp</filter-class>
  </filter>
  <filter>
    <filter-name>sitemesh</filter-name>
    <filter-class>com.opensymphony.module.sitemesh.filter.PageFilter</filter-class>
  </filter>
  <filter>
    <filter-name>action2</filter-name>
    <filter-class>org.apache.struts2.dispatcher.FilterDispatcher</filter-class>
  </filter>
 <filter-mapping>
    <filter-name>action2-cleanup</filter-name>
    <url-pattern>/*</url-pattern>
  </filter-mapping>
  <filter-mapping>
    <filter-name>sitemesh</filter-name>
    <url-pattern>/*</url-pattern>
    <dispatcher>FORWARD</dispatcher>
  </filter-mapping>
  <filter-mapping>
    <filter-name>action2</filter-name>
    <url-pattern>/*</url-pattern>
  </filter-mapping>
   <!-- Listeners -->
 <listener>
    <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
  </listener>
   <listener>
   <listener-class>org.springframework.web.context.request.RequestContextListener</listener-class>
  </listener>
   <servlet>
    <servlet-name>dwr</servlet-name>
    <servlet-class>uk.ltd.getahead.dwr.DWRServlet</servlet-class>
    <init-param>
      <param-name>debug</param-name>
      <param-value>true</param-value>
    </init-param>
  </servlet>
  <servlet>
        <servlet-name>jspSupportServlet</servlet-name>
        <servlet-class>org.apache.struts2.views.JspSupportServlet</servlet-class>
        <load-on-startup>5</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>dwr</servlet-name>
    <url-pattern>/dwr/*</url-pattern>
  </servlet-mapping>
  <welcome-file-list>
    <welcome-file>jsp/index/index.jsp</welcome-file>
  </welcome-file-list>
</web-app>
```

Figure 24 - Extrait du fichier web.xml

- **Les listener**

❖ Le contexte listener *org.springframework.web.context.ContextLoaderListener* écoute l'évènement de chargement du contexte et suite à cet évènement, *Spring* crée tous les beans (objets) qui se trouvent dans le fichier *applicationContext.xml* dont le chemin dans le classpath est indiqué via le paramètre de contexte *contextConfigLocation*.

```
<listener>
   <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
</listener>
<context-param>
  <param-name>contextConfigLocation</param-name>
  <param-value>classpath*:spring/applicationContext*.xml</param-value>
</context-param>
```

❖ Le request listener *org.springframework.web.context.request.RequestContextListener* Écoute les événements de traitement de requête et l'expose au thread courant.

```
<listener>
  <listener-class> org.springframework.web.context.request.RequestContextListener  </listener-class>
</listener>
```

- **Les filtres**

Les filtres présents au niveau du descripteur de déploiement sont : *action2-cleanup, sitemesh, action2 et ResponseOverrideFilter*.

- *action2-cleanup* : Ce filtre a pour rôle de déterminer les paramètres obsolètes (qu'il faudra nettoyer) et quels sont les paramètres qu'il ne faut pas nettoyer. Par exemple si la requête contient le paramètre *decorator*, *action-cleanpup* n'autorisera pas le nettoyage de ce paramètre par le filtre *Dispatcher*. En outre action-cleanpup précisera à *action2* quand nettoyer les requêtes et quand il ne faut pas sinon le *Dispatcher* les nettoiera toutes.
- *sitemesh* : Son rôle est de décorer les pages à partir d'un template.
- *action2* : c'est le *Dispatcher* et son rôle est d'analyser la requête afin de savoir quelle est l'action à exécuter ; en général l'action à exécuter est indiquée dans la requête ex : *http://localhost:8080/KoosseryADempiere.we/window/enregistrer.action*.
- *ResponseOverrideFilter* : c'est un filtre relatif aux actions manipulant les tags *DisplayTag*.

NB : En réalité, le Dispatcher remet même le nettoyage des requêtes au filtre action-cleanpup. Ainsi, l'ordre des filtres est très important. On a donc par ordre dans le fichier web.xml : *ActionContextCleanUp*, *sitemesh* et *FilterDispatcher*. Et notons que la requête est traitée dans cet ordre.

- **Les servlets**
  - *dwr :* permet l'intégration d'AJAX dans les pages. Cette servlet traite les requêtes du type /dwr/*
  - *jspSupportServlet* : Cette servlet ne traite aucune requête car elle ne possède pas de mapping dans le fichier web.xml. Cependant, elle permet juste d'avoir accès au contexte applicatif dans le code.
- **Déclaration de la welcome-file**

```
<welcome-file-list>
    <welcome-file> jsp/index/index.jsp </welcome-file>
</welcome-file-list>
```

Elle indique la page d'accueil de l'application web. Elle correspond à la page qui s'affichera lors du traitement d'une requête de la forme *http//localhost:8080/koosseryADempiere.web/*

III.5.2. Les classes d'action

Lorsque l'application web reçoit une requête http, le filtre DispatcherFilter de Struts2 (org.apache.struts2.dispatcher.FilterDispatcher) analyse la requête afin de déterminer l'action à invoquer.

Les actions chez Struts2 sont identifiées par un triplé :

- le *nom de l'action* : il s'agit de l'identifiant de l'action
- la *classe java de l'action* : il s'agit la plupart du temps d'une classe POJO
- la *méthode de l'action* : il s'agit de la méthode à invoquer sur la classe de l'action

Dans l'application web de l'ERP KoosseryADempiere, nous avons la classe d'action *KoosseryAdempiereBaseAction* : c'est la classe de base de toutes les autres classes d'action *WindowAction, LoginAction, DefaultLoginAction, ReportAction, WorkflowAction, HelpAction, WelcomeAction, ValuePreferenceAction, FindAdvAction, LookupAction, ProcessAction.*

Ces classes d'action comme nous l'avons dit, encapsulent les traitements effectués par les différents servlets (contrôleurs) de l'application web de l'ERP ADempiere (étape 2 de notre méthodologie de migration).

La classe d'action KoosseryAdempiereBaseAction

C'est la classe d'action de base : toutes les classes d'actions étendent *KoosseryAdempiereBaseAction*.

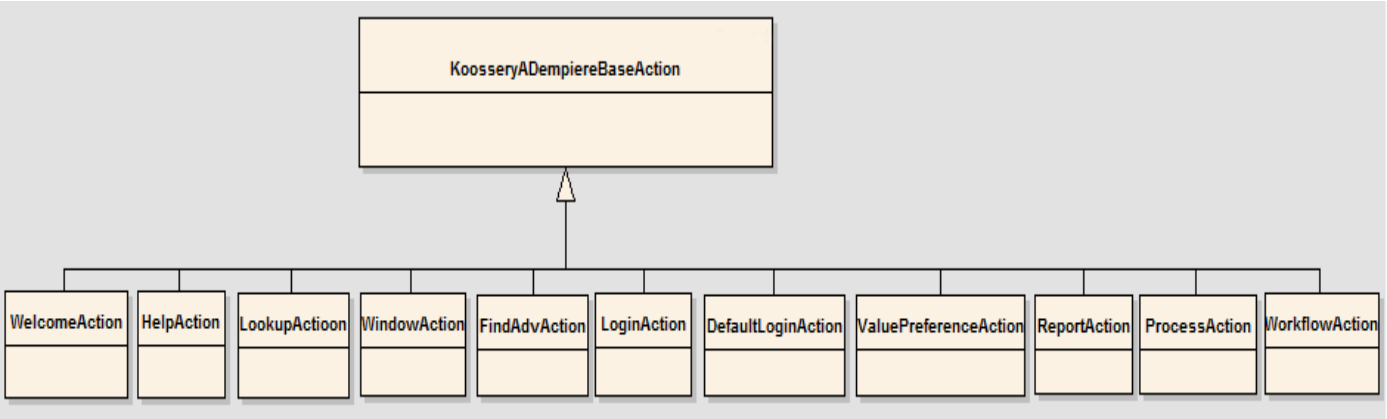


Figure 25 - Les classes d'action

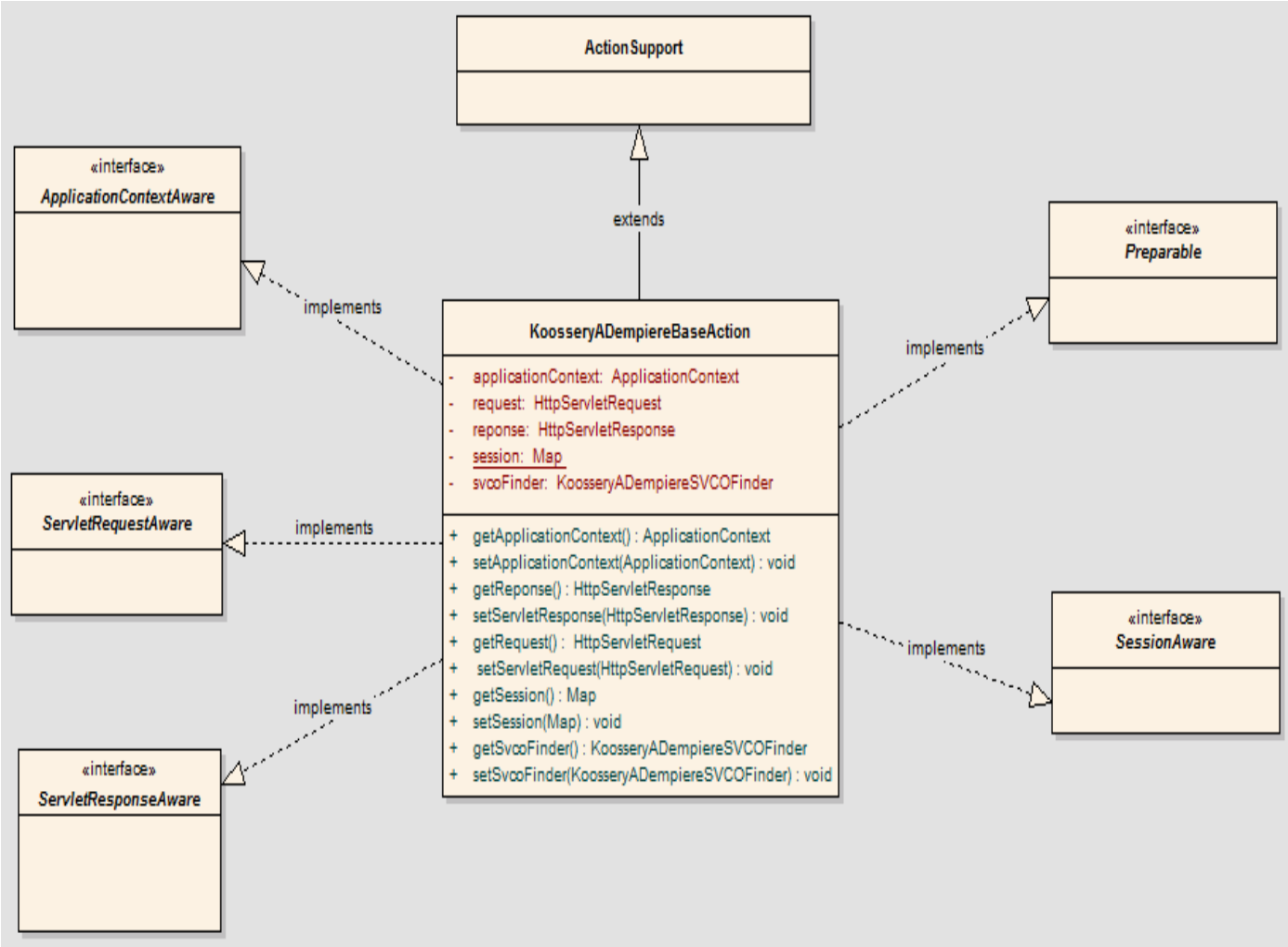


Figure 26 - La classe d'action KoosseryAdempiereBaseAction

Sur la figure 26 :

1. *KoosseryAdempiereBaseAction* étend la classe *ActionSupport* du framework Struts2.
2. *KoosseryAdempiereBaseAction* implémente l'interface *SessionAware* : cela permet aux classes actions de l'application web de l'ERP KoosseryADempiere d'avoir accès à la session http. A cet effet les méthodes setSession() et getSession() sont implémentées.
3. *KoosseryAdempiereBaseAction* implémente l'interface *ApplicationContextAware:* cela permet aux classes actions d'avoir accès aux objets créés via *Spring*. A cet effet une propriété *private ApplicationContext applicationContext;* est ajoutée à la classe.
4. *KoosseryAdempiereBaseAction* implémente l'interface *Preparable* : cette interface permet à Struts2 d'appeler la méthode *prepare* si l'intercepteur *PrepareInterceptor* est appliqué à l'action, ce qui est le cas par défaut. Dans la méthode *prepare()*, sont supprimés les erreurs et les messages d'une action. Ceci est important pour une nouvelle exécution de cette action afin d'éviter d'avoir les erreurs et les messages de l'exécution précédente. Ci-dessous la redéfinition de la méthode *prepare()* :

```
public void prepare() throws Exception {
  // TODO Auto-generated method stub
  clearErrorsAndMessages();
 }
```

5. *KoosseryAdempiereBaseAction* implémente l'interface *ServletRequestAware, et ServletResponseAware* : cela permet aux classes actions de l'application web de l'ERP KoosseryADempiere d'avoir accès aux requêtes et réponses http. A cet effet les méthodes getRequest(), setServletRequest(), getReponse(), setServletResponse() sont implémentées.
6. La propriété *private KoosseryADempiereSVCOFinder svcoFinder;* Elle permet aux classes actions de localiser ou consommer les services exposés par la couche de services (KoosseryADempiere.server)

III.5.3. Fichier de configuration struts.xml

Struts 2 fonctionne par configuration via des fichiers XML. Les actions sont décrites dans un fichier réservé aux actions qui porte par défaut le nom *struts.xml*.

```
<struts>
<package name="window" extends="default" namespace="/window">

  <action name="actualiser" method="actualiser" class="com.koossery.adempiere.web.actions.window.WindowAction">
    <result name="error">/jsp/window/windowPageError.jsp</result>
    <result name="success">/jsp/window/windowPageActualiser.jsp</result>
  </action>
  <action name="nouveau" method="nouveau" class="com.koossery.adempiere.web.actions.window.WindowAction">
    <result name="error">/jsp/window/windowPageError.jsp</result>
    <result name="success">/jsp/window/windowPageNouveau.jsp</result>
  </action>
  <action name="supprimer" method="supprimer" class="com.koossery.adempiere.web.actions.window.WindowAction">
    <result name="error">/jsp/window/windowPageError.jsp</result>
    <result name="success">/jsp/window/windowPageSupprimer.jsp</result>
  </action>

  <action name="enregistrer" method="enregistrer" class="com.koossery.adempiere.web.actions.window.WindowAction">
    <result name="error">/jsp/window/windowPageError.jsp</result>
    <result name="success">/jsp/window/windowPageEnregistrer.jsp</result>
  </action>
    ..............
 </package>
  ......
</struts>
```

Figure 27 - Extrait du fichier struts.xml

Ainsi lorsque le client provoque l'évènement par exemple « *Enregistrer* » dans un formulaire de l'application web (qui correspond à la requête *http://localhost:8080/KoosseryADempiere.we/window/enregistrer.action*), grâce au contenu du fichier de configuration *struts.xml* (figure 27), le contrôleur unique évoque « *l'action enregistrer* ». Cette action sera traitée par la méthode « *enregistrer()* » la classe « *WindowAction* » et le résultat sera redirigé par le contrôleur unique vers la vue « */jsp/window/windowPageEnregistrer.jsp* » en cas de *succès*, mais sera redirigée vers la vue « */jsp/window/windowPageError.jsp* » en cas *d'échec* afin de renseigner correctement le formulaire.

Dans le fichier de configuration struts2 de la figure 27, apparaissent un certain nombre d'éléments importants :

- les namespaces
- les packages
- les results

a) Namespace

Les *namespaces* permettent de regrouper de façon logique les actions. Dans l'application web de l'ERP KoosseryADempiere, les namespaces sont surtout utilisés pour différencier 2 actions portant le même nom : cela nous permet d'uniformiser les noms des actions. Par exemple nous avons l'action « *init* » dans les packages *Window* et *Login*. Nous utilisons les namespace pour différencier c'est-à-dire que pour faire appel à l'action « *init* » du package Window nous utilisons l'url */KoosseryAdempiere.web/window/init.action* alors que pour l'action « init » du package Login c'est l'url */KoosseryADempiere.web/login/init.action*.

Ce principe est illustré comme suit:

```
<package name="window" extends="default" namespace="/window">
    <action name="init" method="init" class="com.koossery.adempiere.web.actions.window.WindowAction">
        <result name="error">/jsp/window/windowPageError.jsp</result>
        <result name="success">/jsp/window/windowPage.jsp </result>
    </action>
</package>
<package name="login" extends="default" namespace="/login">
    <action name="init" method="init" class=" com.koossery.adempiere.web.actions.login.LoginAction">
        <result name="error">/jsp/login/loginPageError.jsp</result>
        <result name="success">/jsp/login/loginPage.jsp </result>
    </action>
</package>
```

b) Packages

Les configurations sont regroupées par package. Un package correspondant à une fonctionnalité du projet. Chaque package contient un ensemble d'actions (traitements).

```
<package name="window" extends="default" namespace="/window">
    <action name="nouveau" method="nouveau" class="com.koossery.adempiere.web.actions.window.WindowAction">
        <result name="error">/jsp/window/windowPageError.jsp</result>
        <result name="success">/jsp/window/windowNewPage.jsp </result>
    </action>
    <action name="enregistrer" method="enregistrer" class="com.koossery.adempiere.web.actions.window.WindowAction">
        <result name="error">/jsp/window/windowPageError.jsp</result>
        <result name="success">/jsp/window/windowEnregistrerPage.jsp  </result>
    </action>
……................................................
  </package>
```

c) Results : type de retour des actions

Les results sont le type de retour d'une action. Dans l'application web de l'ERP KoosseryADempiere il est mis en œuvre les types de results suivant :

- Types standards : success, error, input

```
  <action name="enregistrer" method="enregistrer"
class="com.koossery.adempiere.web.actions.window.WindowAction">
      <result name="error">/jsp/window/windowPageError.jsp</result>
      <result name="success">/jsp/window/windowEnregistrerPage.jsp</result>
      <result name="input">/jsp/window/windowEnregistrerPage.jsp</result>
  </action>
```

- Type redirect-action
  Ce type permet de rediriger une action vers une autre action appartenant (ou pas) au même package.

```
<package name="login" extends="default" namespace="/login">
    <action name="login" method="login" class="com.koossery.adempiere.web.actions.login.LoginAction">
        <result name="error">/jsp/login/loginPageError.jsp</result>
        <result name="success" type="redirectAction">
            <param name="namespace">/welcome</param>
            <param name="actionName">init</param>
        </result>
    </action>
</package>
<package name="welcome" extends="default" namespace="/welcome">
    <action name="init" method="init" class=" com.koossery.adempiere.web.actions.welcome.WelcomeAction">
        <result name="error">/jsp/login/welcomePageError.jsp</result>
        <result name="success">/jsp/login/welcomePage.jsp </result>
    </action>
</package>
```

III.5.4 Spring pour l'IoC et la création des objets

La création des objets est déléguée à *Spring*. Pour cela les fichiers *struts.properties* et *web.xml* sont configurés :

- *web.xml* : le context listener *org.springframework.web.context.ContextLoaderListener* écoute l'évènement de chargement du contexte et suite à cet évènement, *Spring* crée tous les beans qui se trouvent dans le fichier *applicationContext.xml* dont le chemin dans le classpath est indiqué via le paramètre de contexte *contextConfigLocation*

```
<listener>
    <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
</listener>
<context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>classpath*:spring/applicationContext*.xml</param-value>
</context-param>
```

- *struts.properties* : la création des beans (objets) est déléguée à Spring via la configuration ci-dessous : *struts.objectFactory* = spring

Les objets des classes d'action sont créés via *Spring*. Voici quelques illustrations de configurations dans le fichier */spring/applicationContext.xml* .

```
<bean id="com.koossery.adempiere.web.actions.base.KoosseryAdempiereBaseAction"
class="com.koossery.adempiere.web.actions.base.KoosseryAdempiereBaseAction" abstract="true" scope="session">
    <property name="svcoFinder" ref="SVCOFinder" />
</bean>
<bean id="com.koossery.adempiere.web.actions.login.LoginAction"
        class="com.koossery.adempiere.web.actions.login.LoginAction"
        parent="com.koossery.adempiere.web.actions.base.KoosseryAdempiereBaseAction"
        scope="session" />
```

III.5.5. Les Intercepteurs

Les intercepteurs permettent d'exécuter un code avant ou après qu'une action soit invoquée. Dans le projet, les intercepteurs sont mis en œuvre pour :

- Sécuriser l'invocation des actions
- Appliquer des validations sur des données du formulaire lors de l'invocation d'une action

a) Sécurisation de l'invocation des actions

Il s'agit d'empêcher qu'un utilisateur de l'application web de l'ERP KoosseryADempiere non authentifié puisse invoquer une action quelconque. Pour cela nous utilisons un intercepteur pour intercepter chaque invocation afin de vérifier si l'utilisateur est déjà authentifié. S'il ne l'est pas, alors il est redirigé vers la page de connexion avec un message l'invitant à s'authentifier. Cet interceptor s'appelle « *authentificate* », sa classe est *com.koossery.adempiere.web.actions.interceptor.AuthentificationInterceptor* et il étend la classe Struts2 *com.opensymphony.xwork2.interceptor.AbstractInterceptor*. Ci-dessous un extrait du code de la méthode *intercept()* de notre intercepteur « *authentificate* »:

```
public String intercept(ActionInvocation invocation) throws Exception {
                    // on recupère l'action demandée
                    action = (KoosseryAdempiereBaseAction) invocation.getAction();
                    // on recupère la session
                    sessionMap=action.getSession();
                    // recupère le Bean dans la session
                     user= (String) sessionMap.get("User");
                    if(user==null){
                      //si le Bean n'existe pas dans la session l'utilisateur  doit s'authentifier
                      addActionError(invocation,FrontEndConstants.getMessageCtx().getProperty("LOGIN-005"));
                      return "loginPage";
                    }
                    else {
                       // si le Bean est trouvé alors on laisse passer
                       return invocation.invoke();
                    }
}
```

Nous configurons notre interceptor dans le fichier struts.xml au travers de :

- sa déclaration :

```
<interceptor name="authentification"
class="com.koossery.adempiere.web.actions.interceptor.AuthentificationInterceptor" />
```

- son insertion dans une pile d'intercepteurs qu'on définit par défaut**:**

```
<interceptor-stack name="myStack">
<interceptor-ref name="servletConfig" />
<interceptor-ref name="authentification" />
<interceptor-ref name="defaultStack" />
```

- <interceptor-ref name="servletConfig" />: cet intercepteur permet de gérer la session. Il permet de mettre les objets en session et de les accéder. Il faut noter que cet intercepteur vient avant l'intercepteur « authentification » car ce dernier a besoin d'accéder aux objets en session.
- <interceptor-ref name="authentification" /> c'est notre interceptor. Cet intercepteur vient avant le « defaultStack » parce qu'on veut qu'on veut que l'authentification des invocations vient avant tout autre traitement.
- <interceptor-ref name="defaultStack" />: c'est la pile d'intercepteurs par défaut présente dans le fichier *Struts-default.xml* de Struts2.

- Pour finir nous avons déclaré notre pile d'intercepteurs par défaut par :

```
<default-interceptor-ref name="myStack" />
```

<u>Remarque :</u> l'ordre des intercepteurs est très important et par conséquent l'ordre de traitement est respecté. Les invocations des actions sont tout d'abord traitées par l'intercepteur servletConfig, ensuite par authentificate et enfin la pile d'intercepteur par défaut présente dans le fichier *struts-default.xml* du framework Struts2 prend le relais.

Ensuite nous configurons le *result* global vers la login page :

```
<global-results>
        <result name="loginPage">/jsp/login/sessionInvalide.jsp</result>
</global-results>
```

b) <u>Mise en œuvre des validations</u>

Les validations dans struts 2 reposent sur le framework des validations fourni par XWork.

Struts fournit une extension de l'intercepteur de XWork (com.opensymphony.xwork2.validator.ValidationInterceptor) au travers de :

```
<interceptor name="validation" class="org.apache.struts2.interceptor.validation.AnnotationValidationInterceptor"/>
```

L'intercepteur pour les validations s'appuie sur les règles de validation qui peuvent être spécifiées par :

- *classe action* : dans un fichier nommé *NomClasseAction-validation.xml*
- *alias d'action* : dans un fichier nommé *NomClasseAction-alias-validation.xml*
- *Hiérarchie d'héritage et interfaces implémentées par classe action* : XWork recherche l'arbre d'héritage de l'action pour trouver les validations par défaut des classes parents de l'action et les interfaces implémentées

Lorsque les règles de validation sont définies pour une classe d'action, elles s'appliquent à chaque invocation de n'importe quelle méthode de la dite classe d'action. Cependant il est

possible d'explicitement demander que les validations ne soient pas déclenchées pour certaines méthodes de la classe d'action. Pour cela on peut le faire par annotation ou par configuration xml en utilisant la classe *org.apache.struts2.interceptor.validation.SkipValidation*. Struts2 offre deux techniques de validation des données : par annotation et par configuration externalisées dans des fichiers xml. Nous avons opté pour la seconde méthode car les validations effectuées peuvent être modifiées sans toucher au code des classes pour lesquelles elles sont effectuées.

Soit par exemple à valider les données du formulaire de connexion à l'application web de l'ERP KoosseryADempiere lors de l'invocation des actions de la classe *LoginAction.java*. Pour cela dans le projet nous :

1- spécifions les règles de validation dans un fichier *LoginAction-alias-validation.xml* dans le même package que la classe action. Dans le projet, l'alias représente toujours le nom de la méthode de la classe action et par conséquent, la validation ne se fera que sur la méthode (login dans notre cas) de la classe. *LoginAction-login-validation.xml*

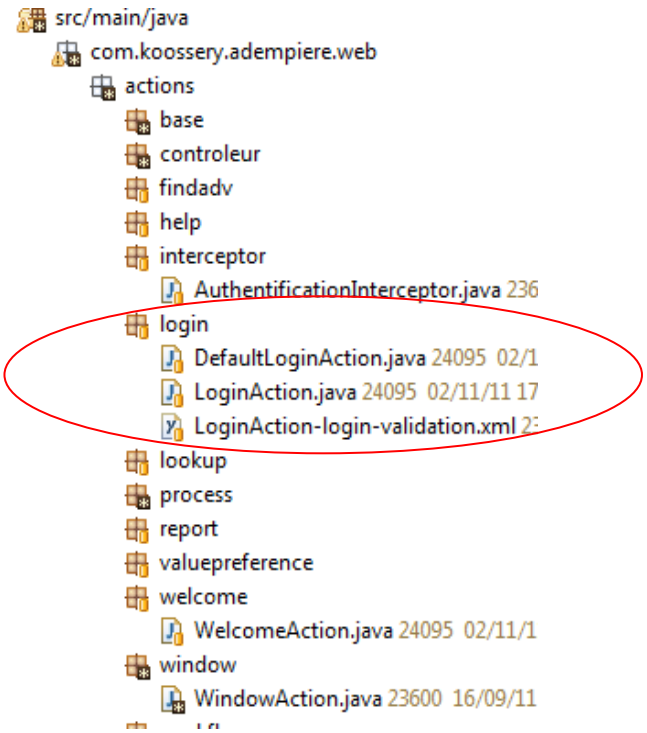


Figure 28 - Package login

Extrait du fichier *LoginAction-login-validation.xml* :

```
<validators>
 <field name="login">
   <field-validator type="requiredstring">
          <param name="trim">true</param>
          <message key="validation.login.jsp.login"/>
   </field-validator>
 </field>
 <field name="passwd">
   <field-validator type="requiredstring">
     <param name="trim">true</param>
     <message key="validation.login.jsp.passwd"/>
   </field-validator>
 </field>
</validators>
```

Dans ce fichier de validation, l'attribut *key* de l'élément message indique la clé du message d'erreur à récupérer dans le bundle correspondant à la langue par défaut. Extrait du contenue du fichier *commonsMessages.properties*

```
############# Messages de validation #############
validation.login.jsp.login=Entrer votre Login
validation.login.jsp.passwd=Entrer votre mot de passe
validation.login.jsp.role=Choisir un Rôle
validation.login.jsp.client=Choisir un Client
validation.login.jsp.org=Choisir une Organisation
validation.login.jsp.warehouse=Choisir un dépôt

############# Messages d'erreur #############
LOGIN-001=LoginAction.init a echoué
LOGIN-002=Login ou mot de passe incorrect! Essayer à nouveau!
LOGIN-003=LoginAction.login a réussi
LOGIN-004=Une erreur s'est produite.%s. %s.
LOGIN-005=Veuillez-vous loguer

############# Welcome #############
WELCOME-001=Login (succès) : {0}
```

III.5.6. Contrainte technique rencontrée

Avant le *déploiement* et *l'exécution* d'une application web Java EE basée sur le framework struts 2, toutes les vues (page JSP) vers lesquelles le contrôleur unique du framework Struts 2 redirige les résultats en cas d'un *success* ou *error* de l'exécution d'une action, sont renseignées dans le fichier de configuration *struts.xml*

```
<action name="init" method="init" class=" com.koossery.adempiere.web.actions.welcome.WelcomeAction">
    <result name="error">/jsp/login/welcomePageError.jsp</result>
    <result name="success">/jsp/login/welcomePage.jsp </result>
</action>
```

Cependant dans l'application web de l'ERP ADempiere certaines vues (page JSP) sont générées en cours d'exécution de l'application par le générateur d'IHM Application Dictionnary (AD) de l'ERP.

Pour résoudre cette contrainte dans l'application web de l'ERP KoosseryADempiere, nous avons implémenté pour ce cas, un contrôleur (ControleurAction) qui en cours d'exécution de l'application web :

1. Intercepte toutes les requêtes dont une vue (page JSP) sera générée par l'Application Dictionnary. Ces requêtes sont identifiées dans l'URL de la requête par le paramètre *action (action=WWindow)*.

   http://localhost:8080/koosseryADempiere.web/controleur/init.action?action= WWindow&AD_Menu_ID=228

2. Renseigne dans le fichier de configuration *struts.xml*, le *package* de la vue qui sera générée en fonction du paramètre *AD_Menu_ID* (au cas où le package n'existe pas déjà). Pour le cas de l'URL précédente dont *AD_Menu_ID=228*, le *package* se présentera comme suit :

```
<package name="window228" extends="default" namespace="/window228">
  <action name="init" method="init" class="com.koossery.adempiere.web.actions.window.WindowAction">
   <result name="error">/jsp/login/login.jsp</result>
   <result name="success">/jsp/window/window228.jsp</result>
  </action>
  <action name="window" method="window" class="com.koossery.adempiere.web.actions.window.WindowAction">
   <result name="error">/jsp/window/window228.jsp</result>
   <result name="input">/jsp/window/window228.jsp</result>
   <result name="success">/jsp/window/window228.jsp</result>
  </action>
  <action name="cancel" method="cancel" class="com.koossery.adempiere.web.actions.window.WindowAction">
   <result name="input">/jsp/window/window228.jsp</result>
   <result type="redirectAction">
    <param name="namespace">/welcome</param>
    <param name="actionName">init</param>
   </result>
  </action>
.......................................
</package>
```

Figure 29 - Extrait du Package de la vue window228.jsp générée par l'Application Dictionnary

3. Enfin passe la main au contrôleur unique du framework Struts 2 pour le traitement de la requête.

CONCLUSION GENERALE

Au terme de ce travail dont le but était de savoir comment migrer une Interface Homme Machine de l'architecture MVC vers l'architecture MVC 2 ; nous avons obtenu comme principaux résultats :

- Sur le plan théorique, une méthodologie de migration d'une IHM de l'architecture MVC vers l'architecture MVC 2.
- Sur le plan pratique, une application web pour l'ERP KoosseryADempiere basée sur le framework Struts 2 qui résulte de l'application de la méthodologie de migration.

Dans le premier chapitre nous avons défini les différents concepts qui interviennent dans le sujet, le génie logiciel domaine dans lequel se situe notre travail, le refactoring, l'IHM, architecture d'IHM, l'architecture MVC, l'architecture MVC 2 et la migration informatique. Dans le second chapitre nous avons présente notre méthodologie de migration, les outils scientifiques utilisés et l'illustration sur l'application web de l'ERP ADempiere. Et en fin dans le troisième chapitre qui est entièrement consacré à l'étude de cas : nous avons présenté la réalisation de l'application web de l'ERP KoosseryADempiere.

PERSPECTIVE

Migrer l'architecture d'une IHM vers une autre architecture implique une démarche rigoureuse car cette opération ne doit en rien modifier au fonctionnement de départ de l'application. La nouvelle solution devra conserver les mêmes propriétés ou caractéristiques que la solution de départ. Bien qu'en interne des tests sur les différentes fonctionnalités de l'application web de l'ERP KoosseryADempiere aient été faits pendant la migration, Il faudra à présent effectuer ses tests au sein d'une entreprise car la validation est un examen réalisé par le client durant lequel il vérifie que les fonctions offertes par le logiciel correspondent effectivement.

REFERENCES BIBLIOGRAPHIQUES

BARON Mickaël (2003), *Vers une approche sûre du développement des Interfaces Homme-Machine*, Thèse de doctorat à l'Université de Poitiers, le 15 décembre 2003. Lien (http://www.lisi.ensma.fr/ftp/pub/documents/thesis/2003-thesis-baron.pdf)

Coutaz J. (1990), *Interfaces Homme-Ordinateur, Conception et Réalisation*, Editeur : Dunod, Collection : Dunod Informatique, ISBN : 2040196358, Paris 1990.

Dembski Ionel (2005), *Ré-architecture et migration d'une application standalone vers un serveur applicatif multi tiers dans un contexte JAVA-SAP*, mémoire de Master, 25 novembre 2005 à Hautes Ecoles Spécialisées de Suisse Occidentale. Lien (http://campus.hesge.ch/Daehne/TB/2005/DembskiIonel.pdf)

LANGLET Étienne (2008), *Apache Tomcat 6 Guide d'administration du serveur Java EE sous Windows et Linux*, ouvrage 2008, Editions ENI, ISBN: 2746041626.

Fowler Martin (1999), *Refactoring: Improving the Design Of Existing Cod*e, ouvrage Editeur Addison-Wesley/ Helix Books, 1999.

Garlan David et Dewayne E. Perry (1995), *Introduction to the Special Issue on Software Architecture*, Article Editor: Nancy G., April 1995. Lien (http://users.ece.utexas.edu/~perry/work/papers/tse21-4.pdf)

Guittet L. (1995), *Contribution à l'Ingénierie des Interfaces Homme-Machine*, Théorie des Interacteurs et Architecture H4 dans le système NODAOO. Thèse de doctorat, Université de Poitiers, 1995.

DOUDOUX Jean Michel (2011), *Développons en Java*, Didactiel (version 1.6) du 11/09/2011. Lien (ftp://ftp2.jmdoudoux.fr/jmdoudoux/dej_1_60.zip)

Lafosse Jérôme (2009), *Struts 2 - Le framework de développement d'applications Java EE – Présentation*, ouvrage 2009, Editions ENI, ISBN : 2746050544. Lien (http://sliim-projects.eu/Programmation/Java/Struts%202%20Le%20framework%20de%20d%E9veloppement%20d%27applications%20Java%20EE.pdf)

KADRI Réda (2010), *Une approche pour la Modélisation d'Applications Web à base de Composants Logiciels*, Thèse de doctorat à l'Université de Bretagne Sud, 26 août 2010. Lien (http://hal.archives-ouvertes.fr/docs/00/51/18/28/PDF/kadri.pdf)

Karim Zarour (2000), *Vers un système d'information coopératif pour la prise en charge des soins à domicile : une architecture basée agent*. Mémoire présenté en vue de l'obtention du Magister en Informatique Option : Systèmes d'Information & Intelligence Artificielle Distribués

SAMAAN Kinan (2006), *Prise en Compte du Modèle d'Interaction dans le Processus de Construction et d'Adaptation d'Applications Interactives*, Thèse de doctorat à l'Ecole Centrale de Lyon, le 03 octobre 2006. Lien (http://bibli.ec-lyon.fr/exl-doc/ksamaan.pdf)

AUDIBERT Laurent (2006), *UML 2 - De l'apprentissage à la pratique*, Ouvrage 2009, Editeur : Ellipses, ISBN13 :978-2-7298-5269-6.
Lien (http://laurent-audibert.developpez.com/Cours-UML/html/index.html)

Myers Brad A. (1995). *User interface software tools. ACM Transactions on Computer Human Iteraction*, Article 1995, Editor Dan R., Publisher: ACM New York,
Lien (http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.73.5361&rep=rep1&type=pdf)

Nigay L. et Coutaz J. (1993), *Design Space for Multimodal Systems: Concurrent Processing and Data Fusion*, Article 1993, Publisher: ACM New York, NY, USA ©1993;

Pfaff G. (1985), *User Interface Management Systems*, Book 1985, Publisher: Springer-Verlag New York, Inc. Secaucus, NJ, USA ©1985 Springer-Verlag, NY, ISBN: 038713803X.

Retaillé Jean-Philippe (2005) , *Refactoring des applications Java / J2EE*, ouvrage *Editeur Eyrolles*, juin 2005.

Roland Ollivet (2005), *Transformations des applications informatiques*, article 2005.
Lien (http://rolsys.com/site/page/mlr/rol/prfpb/transformations_applications_informatiques.pdf)

TARBY Jean-Claude (1993), Gestion Automatique du Dialogue Homme-Machine à partir de Spécifications Conceptuelles, *Thèse présentée en vue de l'obtention du Doctorat à l'Université de Toulouse I*, 20 septembre 1993. Laboratoire d'accueil : L.I.S.
Lien (http://hal.archives-ouvertes.fr/docs/00/17/46/43/PDF/JC_Tarby.pdf)

UIMS (1992), *A metamodel for the runtime architecture of an interactive system*, Article 1992, Editor: William, Publisher: ACM New York, NY, USA, ISSN: 0736-6906.
Lien (http://nielsmayer.com/L18623-5228TMP.pdf)

ZANGO Abdoulaye et OUATTARA Yacouba (2006), *Migration vers une architecture 3-tier respectant le modèle MVC de l'application web de gestion des règlements pécuniaires des avocats du Burkina*, Projet de fin de cycle Ingénieur à l'Ecole Supérieure d'informatique au Burkina Faso.
Lien (http://www.pierregavin.ch/documents/Varia/migration-informatique.pdf)

REFERENCES WEBOGRAPHIQUES

ADempiereWIKI, *Site officiel de l'ERP ADempiere*. Lien (http://www.adempiere.com).

ApacheStruts, *Site officiel du Framework de développement web Struts 2*.
Lien (http://struts.apache.org/2.x/)

Laberge Véronique (2008), *Réalisation d'une application Web*, article 2008 sur wikipractice.
Lien (http://fr.wikipractice.org/wiki/Réalisation_d'une_application_Web)

Migration (informatique). (2011, juillet 10). Wikipédia, l'encyclopédie libre. Page consultée le 08:07, janvier 10, 2012 à partir de
Lien (http://fr.wikipedia.org/w/index.php?title=Migration_(informatique)&oldid=67320793).

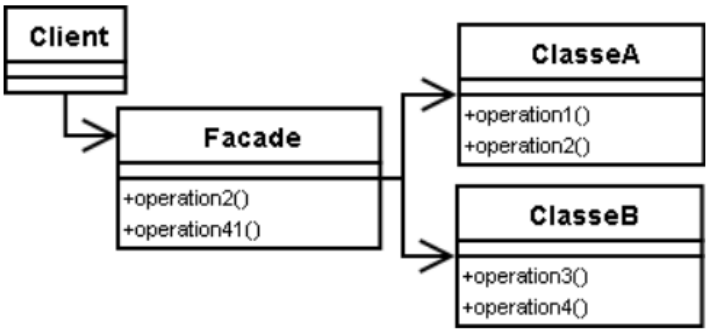
SUN, Site officiel de Sun. Lien (http://java.sun.com/)

SunJSF, Site officiel du Framework JSF. Lien (http://java.sun.com/j2ee/javaserverfaces/)

SunServlet, Site officiel. Lien (http://java.sun.com/products/servlet/index.html).

ANNEXES

Design pattern Façade (*par Régis POUILLER*)



AUTRE RESSOURCE SUR DEVELOPPEZ.COM :

La façade par Sébastien MERIC (*http://smeric.developpez.com/java/uml/facade/)*

OBJECTIFS :

- Fournir une interface unique en remplacement d'un ensemble d'interfaces d'un sous-système (*http://dico.developpez.com/html/2838-Conception-sous-systeme.php*).
- Définir une interface de haut niveau pour rendre le sous-système plus simple d'utilisation.

RAISONS DE L'UTILISER :

Le système comporte un sous-système complexe avec plusieurs interfaces. Certaines de ces interfaces présentent des opérations qui ne sont pas utiles au reste du système.

Cela peut être le cas d'un sous-système communiquant avec des outils de mesure ou d'un sous-système d'accès à la base de données.

Il serait plus judicieux de passer par une seule interface présentant seulement les opérations utiles. Une classe unique, la façade, présente ces opérations réellement nécessaires. *Remarque : La façade peut également implémenter le Design Pattern Singleton (http://rpouiller.developpez.com/tutoriel/java/design-patterns-gang-of-four/?page=page_2#LIV-E)*.

RESULTAT :

Le Design Pattern permet d'isoler les fonctionnalités d'un sous-système utiles à la partie cliente.

RESPONSABILITES :

- *ClasseA* et *ClasseB* : implémentent diverses fonctionnalités.
- *Facade* : présente certaines fonctionnalités. Cette classe utilise les implémentations des objets *ClasseA* et *ClasseB*. Elle expose une version simplifiée du sous-système *ClasseA-ClasseB*.
- La partie cliente fait appel aux méthodes présentées par l'objet *Facade*. Il n'y a donc pas de dépendances entre la partie cliente et le sous-système *ClasseA-ClasseB*.

IMPLEMENTATION EN JAVA :

ClasseA.java

```java
/**
 * Classe implémentant diverses fonctionnalités.
 */
public class ClasseA {

    public void operation1() {
        System.out.println("Methode operation1() de la classe ClasseA");
    }

    public void operation2() {
        System.out.println("Methode operation2() de la classe ClasseA");
    }
}
```

ClasseB.java

```java
/**
 * Classe implémentant d'autres fonctionnalités.
 */
public class ClasseB {

    public void operation3() {
        System.out.println("Methode operation3() de la classe ClasseB");
    }

    public void operation4() {
        System.out.println("Methode operation4() de la classe ClasseB");
    }
}
```

Facade.java

```java
/**
 * Présente certaines fonctionnalités.
 * Dans ce cas, ne présente que la méthode "operation2()" de "ClasseA"
 * et la méthode "operation41()" utilisant "operation4()" de "ClasseB"
 * et "operation1()" de "ClasseA".
 */
public class Facade {

    private ClasseA classeA = new ClasseA();
    private ClasseB classeB = new ClasseB();

    /**
     * La méthode operation2() appelle simplement
     * la même méthode de ClasseA
     */
    public void operation2() {
        System.out.println("--> Méthode operation2() de la classe Facade : ");
        classeA.operation2();
    }

    /**
     * La méthode operation41() appelle
     * operation4() de ClasseB
     * et operation1() de ClasseA
     */
    public void operation41() {
        System.out.println("--> Méthode operation41() de la classe Facade : ");
        classeB.operation4();
        classeA.operation1();
    }
}
```

FacadePatternMain.java

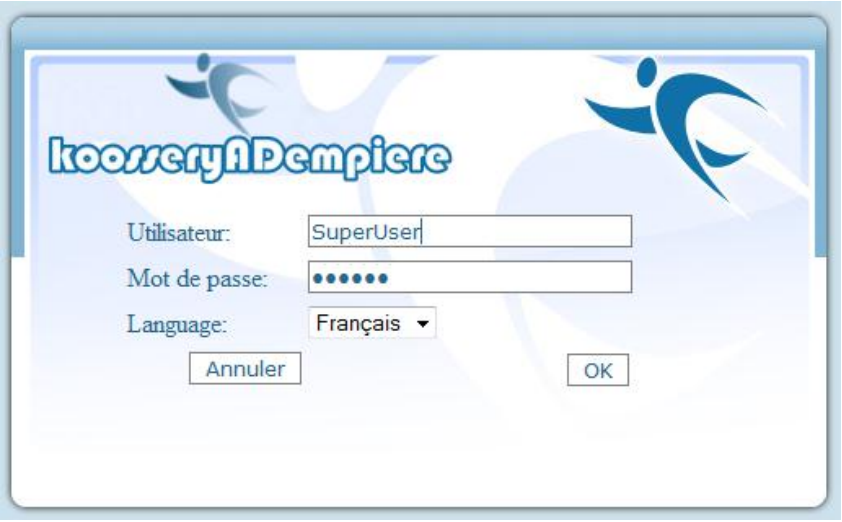
```java
public class FacadePatternMain {

    public static void main(String[] args) {
        // Création de l'objet "Facade" puis appel des méthodes
        Facade lFacade = new Facade();
        lFacade.operation2();
        lFacade.operation41();
```

FacadePatternMain.java

```
        // Affichage :
        // --> Méthode operation2() de la classe Facade :
        // Methode operation2() de la classe ClasseA
        // --> Méthode operation41() de la classe Facade :
        // Methode operation4() de la classe ClasseB
        // Methode operation1() de la classe ClasseA
    }
}
```

POUILLER Régis (2008), *Design Patterns du Gang of Four appliqués à Java*, Article, Date de publication : 16/09/2008, Lien[http://rpouiller.developpez.com/tutoriel/java/design-patterns-gang-of-four/]

Capture d'écran des formulaires de connexion de l'application web de l'ERP KoosseryADempiere



Premier formulaire de connexion

Deuxième formulaire de connexion affiché après validation des informations du premier formulaire

Capture d'écran de la page d'accueil de l'application web de l'ERP KoosseryADempiere

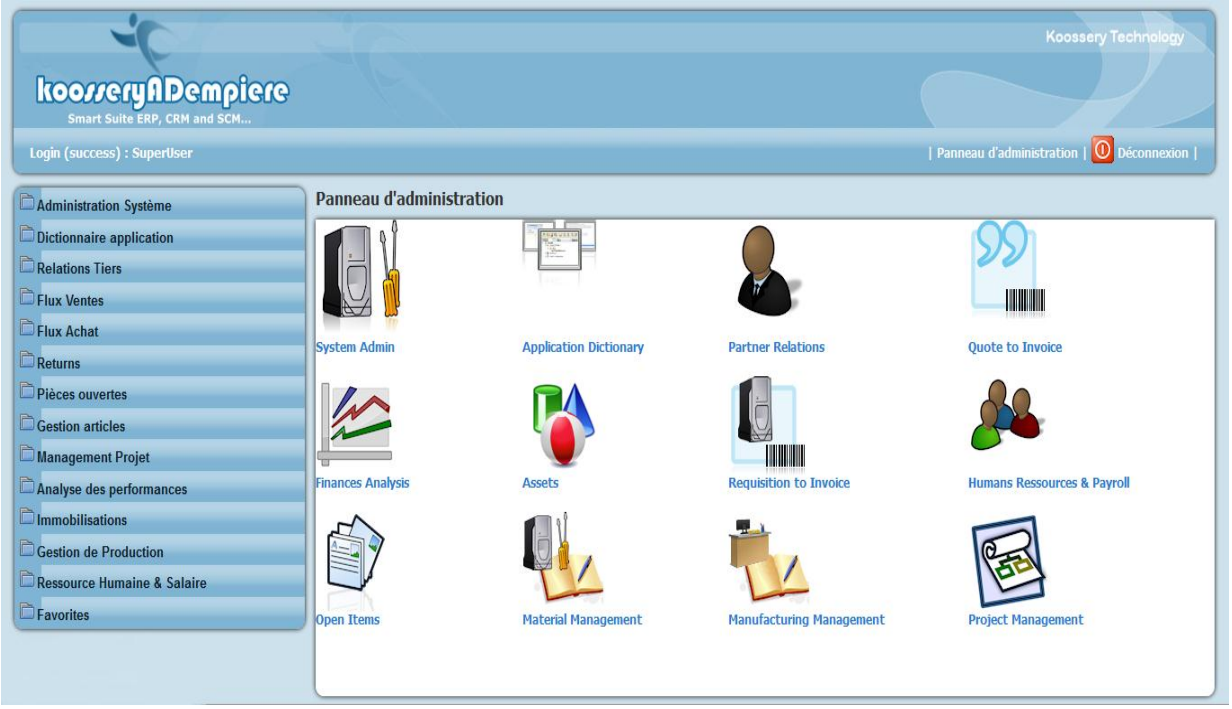


TABLE DES MATIERES

Dédicaces i
Remerciements ii
Abréviations iii
Résumé iv
Abstract iv
Liste des figures v
Sommaire vi
INTRODUCTION GENERALE 1
CHAPITRE I : DEFINITION DES CONCEPTS 5
I.1. Le génie logiciel 5
I.1.1. Les logiciels 5
I.1.2. Le génie logiciel 6
I.1.3. Le rôle de la maintenance dans l'évolution logicielle 7
I.2. Le refactoring 9
I.2.1. Les techniques de refactoring 9
I.2.2. Le processus de refactoring 10
I.3. L'Interface Homme Machine 11
I.4. Architecture d'Interface Homme Machine 13
I.5. L'architecture MVC 15
I.5.1. Fonctionnement du MVC 15
I.5.2. Les principaux avantages du MVC 17
I.5.3. Quelques inconvénients 18
I.6. L'architecture MVC 2 19
I.7. Migration informatique 21
CHAPITRE II : METHODOLOGIE DE MIGRATION 23
II.1. Les outils scientifiques utilisés lors de la migration 23
II.1.1. Renommage 23
II.1.2. Extraction d'une méthode 24
II.1.3. Extraction d'interface 25
II.1.4. Changement de signature d'une méthode 26
II.1.5. Généralisation d'un type 27
II.1.6. Déplacement d'éléments 28
II.1.7. Le pattern Façade 29
II.2. Présentation de la méthodologie de migration 30
II.3. Illustration sur l'application web de l'ERP ADempiere 32
II.3.1. Généralités 32
a) Présentation de la plate-forme Java EE 33
b) Les composants Java EE 34
c) Le modèle d'architecture MVC appliqué à Java EE 37
d) Framework Struts 2 38
II.3.2. Etude de l'existant 44

II.3.3. Application de la méthodologie de migration sur l'ERP ADempiere 46
CHAPITRE III : REALISATION DE L'APPLICATION WEB DE L'ERP KoosseryADempiere 50
III.1. Présentation de L'ERP KoosseryADempiere 50
III.2. Environnement de développement et outils utilisés 51
III.3. Architecture de l'application web de l'ERP KoosseryADempiere 52
III.4. Arborescence du projet *KoosseryADempiere.web* 53
III.5. Migration de l'application web de l'ERP ADempiere 55
III.5.1. Configuration du projet KoosseryADempiere.web 55
a) Le fichier pom.xml Maven 2 du projet 55
b) Description du fichier de déploiement web.xml du projet 56
III.5.2. Les classes d'action 59
III.5.3. Fichier de configuration struts.xml 62
a) Namespace 63
b) Packages 63
c) Results : type de retour des actions 63
III.5.4 Spring pour l'IoC et la création des objets 64
III.5.5. Les Intercepteurs 65
a) Sécurisation de l'invocation des actions 65
b) Mise en œuvre des validations 66
III.5.6. Contrainte technique rencontrée 68
CONCLUSION GENERALE 70
Références bibliographiques 71
Annexes 73
Table des matières 77